



Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE

Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)

Best College Award by University of Mumbai for the Year 2018-2019



DEPARTMENT OF INFORMATION TECHNOLOGY

A PROJECT REPORT ON

Browser Based Discrete – Event Simulator

“JustCmul8 — A No-Code, Browser-Based Discrete Event Simulation Platform”



SUBMITTED BY

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

UNDER THE GUIDANCE OF

Ms. Jinal Gujar

Submitted in partial fulfilment of the requirements for qualifying

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

Semester – V Examination

THAKUR COLLEGE OF SCIENCE AND COMMERCE

(Empowered Autonomous College, Permanently Affiliated to University of Mumbai)

THAKUR VILLAGE, KANDIVALI (EAST), MUMBAI – 400 101, MAHARASHTRA

ACADEMIC YEAR 2026 – 2027



Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019



CERTIFICATE OF APPROVAL

This is to certify that the project entitled, “**Browser Based Discrete – Event Simulator**” has been undertaken at Thakur College of Science and Commerce by **Ms. Khushi Singh (Roll No. 261713)** and **Mr. Mohit Gupta (Roll No. 261715)**, students of “Bachelor of Science (Information Technology)”, and is submitted in partial fulfilment of the requirements for the award of the degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY, Semester–V Examination, as per the syllabus defined by the University of Mumbai in the academic year 2026 – 2027.

It is further certified that the students have completed all the required phases of the project, and that the work presented in this report is their own and has not been submitted elsewhere for the award of any other degree or diploma.

Ms. Jinal Gujar

Project Guide

Prof. Dr. Santosh Kumar Singh

Head of Department

External Examiner

Principal

Date: _____

College Seal / Stamp: _____

CANDIDATES' DECLARATION

We hereby declare that the project work presented in this report entitled “**Browser Based Discrete – Event Simulator**”, submitted to the Department of Information Technology, Thakur College of Science and Commerce, in partial fulfilment of the requirements for the award of the degree of Bachelor of Science in Information Technology (Semester–V), is an authentic record of our own work carried out during the academic year 2026 – 2027 under the guidance of Ms. Jinal Gujar.

We further declare that the analysis, system design, source code, simulation engine integration, testing and documentation presented in this report were developed by us. Wherever the work of other authors, published literature, official technical documentation or open-source libraries has been referred to, due acknowledgement has been made in the text and the corresponding sources have been listed in the References. The matter embodied in this report has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma.

We also understand and accept that any breach of academic integrity detected at any stage would attract disciplinary action as per the rules and regulations of the institution and the University of Mumbai.

Place: Mumbai

Date: _____

Ms. Khushi Singh

Roll No. 261713

Mr. Mohit Gupta

Roll No. 261715

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the guidance, encouragement and practical help we received from several people, and we would like to record our gratitude to them here.

First and foremost, we express our sincere thanks to our project guide, **Ms. Jinal Gujar**, Department of Information Technology, for the consistent direction she provided throughout the development of JustCmul8. Her insistence that we validate the simulation engine before building features on top of it changed the order in which we scheduled our work, and saved us a considerable amount of rework later. The structure of this report also owes a great deal to her review of our early drafts.

We are deeply grateful to **Prof. Dr. Santosh Kumar Singh**, Head of the Department of Information Technology, for creating an academic environment in which an ambitious project of this nature could be attempted, and for his valuable suggestions during the project review sessions.

We extend our thanks to the Principal and the management of Thakur College of Science and Commerce for providing the infrastructure, laboratory facilities and internet resources that this project required. Running a WebAssembly-based Python runtime and testing real-time collaboration across multiple machines simultaneously would not have been feasible without access to the department laboratory.

We thank the faculty members of the Department of Information Technology for the foundation they laid in data structures, database management systems, software engineering and web technologies. Concepts such as priority queues, normalisation and the software development life cycle, taught across earlier semesters, were used directly in this work rather than merely referenced. We are also grateful to the laboratory assistants for their patient help with system access and configuration during the long testing sessions.

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

ABSTRACT

Discrete Event Simulation (DES) is among the most dependable ways of predicting how a queue, a production line or a delivery network will behave before money is spent changing it, yet the technique remains out of reach for most of the people who would benefit from it. Commercial packages such as AnyLogic, Arena and Simul8 carry licence fees running into thousands of dollars a year; the capable open-source alternatives, SimPy chief among them, are programming libraries that expect the user to write and debug Python before a first model will run; and almost all of these tools are desktop applications that must be installed, which rules them out in shared laboratories and on managed machines. Drawing tools such as Visio and Lucidchart sit at the opposite extreme: easy to use, but the diagrams they produce do not execute, so they cannot answer a question as ordinary as “what happens if one more counter is opened?”.

This project closes that gap. **JustCmul8** is a no-code, browser-based Discrete Event Simulation platform on which a user builds a model by dragging nodes onto a canvas and connecting them, configures each node through ordinary form fields, and runs the result without installing anything. The distinguishing architectural decision is that the model is never approximated in JavaScript: the graph is compiled into a genuine Python SimPy script, executed inside the browser by a CPython interpreter compiled to WebAssembly (Pyodide) and hosted in a Web Worker so that the interface stays responsive. Every number the platform reports is therefore produced by the same engine a researcher would use from a terminal.

The platform is implemented as a client-heavy single-page application using Next.js 16, React 19 and TypeScript, with the canvas built on React Flow and shared state held in Zustand. Supabase supplies authentication, persistence and real-time channels, with PostgreSQL Row-Level Security enforcing project isolation at the database rather than in client code. Twelve functional modules and fifteen configurable node types span six application domains: human queues, vehicle traffic, liquid and material flow, manufacturing, logistics and network signalling. An analytics layer computes utilisation, waiting times, throughput, queue lengths, a Little’s Law consistency check and a composite system health score, and flags the bottleneck node; a Monte Carlo facility repeats a run across independent replications and reports 95 per cent confidence intervals instead of a single figure; an animated digital-twin playback, a cost and return-on-investment panel, a Python code inspector with Jupyter export, executive PDF reports and revocable read-only share links complete the workflow. A Google Gemini assistant,

served through a health-aware model router with automatic fall-back, converts a plain-English description into a candidate graph and recommends fixes for the bottleneck it finds.

The engine was verified against closed-form queueing theory. In the verification suite all 28 analytical checks (M/M/1, M/M/c and saturated-queue benchmarks) and all 22 structural invariants of a multi-stage hospital-flow model passed, with simulated utilisation, waiting time and queue length agreeing with theory to within 3.5 per cent. The result is a working platform that removes the cost, coding and installation barriers to simulation without compromising the accuracy of the engine underneath.

Keywords: discrete event simulation, SimPy, WebAssembly, Pyodide, no-code modelling, Monte Carlo analysis, Next.js, Supabase, AI-assisted modelling.

INDEX

1. INTRODUCTION.....	1
1.1 Background and Motivation	1
1.2 Objective and Scope of the Project.....	4
1.3 Problem Definition.....	6
1.4 Limitations	8
2. REQUIREMENT ANALYSIS.....	9
2.1 User Requirements / SRS / Functional Requirements	9
2.2 Feasibility Study	13
2.3 Details of Hardware and Software Used.....	15
2.4 Proposed System and Advantages of the Proposed System	18
3. PROJECT PLANNING.....	20
3.1 List of Activities / Tasks	20
3.2 Timeline for Each Task.....	22
3.3 Gantt Chart and Activity Network Diagram	24
3.4 Resource Planning	28
3.5 Proposed Software Development Model	30
4. SOFTWARE / HARDWARE MODULES	33
4.1 Module 1 — Platform and Data Layer	34
4.2 Module 2 — Visual Graph Editor.....	35
4.3 Module 3 — Node Configuration Panel	37
4.4 Module 4 — Simulation Engine	38
4.5 Module 5 — AI Assistant	39
4.6 Module 6 — AI System Optimiser	40
4.7 Module 7 — KPI Dashboard and Results.....	41
4.8 Module 8 — Monte Carlo and Scenario Comparison	42
4.9 Module 9 — Digital Twin Animated Playback	43
4.10 Module 10 — Simulation Template Gallery	43
4.11 Module 11 — Executive Report and Share	44
4.12 Module 12 — Real-Time Collaboration	45
4.13 Hardware Modules and Considerations	46
4.14 Individual Work Justification	46
5. SYSTEM ANALYSIS AND DESIGN.....	48
5.1 Context Diagram.....	48
5.2 DFD, ERD, Class Diagram and State Transition Diagram.....	49

5.3 Use Case Diagram.....	55
5.4 Architecture Design	58
5.5 Activity, Component and Collaboration Diagrams	60
5.6 Table Design	64
5.7 Architecture / Deployment Diagram.....	67
6. SYSTEM IMPLEMENTATION AND METHODOLOGY	70
6.1 Development Methodology	70
6.2 Technologies Used.....	70
6.3 Algorithms and Techniques Used.....	71
6.4 Module-wise Implementation	74
6.5 Database Implementation.....	75
6.6 User Interface Design	75
6.7 Important Code Snippets.....	79
7. SYSTEM TESTING	82
7.1 Testing Strategy	82
7.2 Testable Units	83
7.3 Test Case Design with Reference to the Project.....	84
7.4 Test Approach.....	88
7.5 Test Results and Validation	89
8. RESULTS AND DISCUSSION	91
8.1 System Output.....	91
8.2 Result Analysis	99
8.3 Performance Evaluation.....	101
9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION.....	104
9.1 Effort Estimation.....	104
9.2 KSLOC Analysis	105
9.3 Function Point Analysis.....	106
9.4 Comparison of Estimates and Cost–Benefit Analysis	108
10. CONCLUSION AND FUTURE SCOPE	111
10.1 Achievements of the Project	111
10.2 Conclusion	112
10.3 Future Scope	112
REFERENCES AND BIBLIOGRAPHY	115

LIST OF FIGURES

Figure 1.1: Functional Modules of JustCmul8 grouped by area	5
Figure 2.1: Execution Pipeline of the Proposed System	18
Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026).....	25
Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle	27
Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8.....	31
Figure 4.1: Module Structure of JustCmul8.....	33
Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8	48
Figure 5.2: Level-1 Data Flow Diagram for JustCmul8	50
Figure 5.3: Level-2 Data Flow Diagram — Decomposition of Process P4 (Simulation Execution)....	51
Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database	52
Figure 5.5: Class Diagram of the JustCmul8 Object Model	53
Figure 5.6: State Transition Diagram for a Node During Simulation Execution.....	54
Figure 5.7: Use Case Diagram for JustCmul8	55
Figure 5.8: Multi-Tier Architecture of JustCmul8.....	59
Figure 5.9: Activity Diagram of the End-to-End User Workflow	61
Figure 5.10: Component Diagram of the JustCmul8 Implementation.....	62
Figure 5.11: Collaboration Diagram for the Simulation Run Interaction	63
Figure 5.12: Deployment Architecture of JustCmul8	68
Figure 6.1: Landing Page.....	76
Figure 6.2: Registration and Login Screens.....	77
Figure 6.3: Project Dashboard	77
Figure 6.4: Workspace with the Visual Graph Editor.....	78
Figure 6.5: Node Configuration Panel	78
Figure 6.6: Template Gallery	79
Figure 7.1: Layered Testing Strategy for JustCmul8.....	82
Figure 8.1: Executive Summary Tab of the Results Dashboard	91
Figure 8.2: Flow and Balance Check (Little’s Law Verification)	92
Figure 8.3: Traffic Heatmap	92
Figure 8.4: Line Buildup Timeline	93
Figure 8.5: Station Counters	93
Figure 8.6: Person / Item Journey (Entity Tracer)	94
Figure 8.7: Cost and Return-on-Investment Panel.....	94
Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands	95
Figure 8.9: Digital Twin Animated Playback	95
Figure 8.10: AI Assistant Generating a Model from a Description	96
Figure 8.11: AI System Optimiser Recommendation	96
Figure 8.12: Python Code Inspector	97
Figure 8.13: Real-Time Presence and Live Cursors	97
Figure 8.14: Export, Share and Executive PDF Report	98

Figure 8.15: Read-Only Shared View	98
Figure 8.16: Profile and Settings.....	99
Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks).....	99
Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance	100

LIST OF TABLES

Table 1.1: Comparison of Discrete Event and Continuous Simulation	2
Table 1.2: Probability Distributions Implemented in JustCmul8.....	2
Table 2.1: User Classes and Access Levels	9
Table 2.2: Functional Requirements	10
Table 2.3: Non-Functional Requirements.....	12
Table 2.4: Indicative Cost Comparison with Commercial Alternatives	14
Table 2.5: Development Hardware and Software Configuration.....	16
Table 2.6: Minimum and Recommended Client Requirements.....	16
Table 2.7: Technology Stack with Justification.....	17
Table 2.8: Comparison of the Proposed System with Existing Approaches.....	18
Table 3.1: List of Activities and Tasks with Dependencies.....	21
Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026).....	22
Table 3.3: Project Milestones and Completion Criteria.....	23
Table 3.4: Human Resources and Responsibilities	28
Table 3.5: Hardware, Software and Service Resources	29
Table 3.6: Planned Effort Allocation by Phase.....	29
Table 3.7: Phases of the Adopted Life-Cycle Model.....	31
Table 3.8: Why the Iterative and Incremental Model Was Preferred	32
Table 4.1: Module Overview and Requirement Coverage.....	33
Table 4.2: The Fifteen Node Types Supported by the Simulation Engine	36
Table 4.3: The Six Simulation Domains and Their Vocabularies	37
Table 4.4: Key Performance Indicators and Their Computation	41
Table 4.5: Hardware Roles in the Deployed System	46
Table 4.6: Division of Work between Team Members.....	46
Table 5.1: Use Case Specification — Run Simulation	56
Table 5.2: Use Case Specification — Generate Model via AI	57
Table 5.3: Use Case Specification — Share Project Read-Only	57
Table 5.4: Use Case Specification — Collaborate in Real Time.....	58
Table 5.5: Table: auth.users (managed by Supabase Auth).....	64
Table 5.6: Table: public.profiles	64
Table 5.7: Table: public.projects.....	65
Table 5.8: Table: public.simulation_runs	65
Table 5.9: Table: public.chat_history	65
Table 5.10: Table: public.project_shares	66
Table 5.11: Row-Level Security Policies	66
Table 6.1: Complexity of Graph Operations on the Adjacency List.....	72
Table 6.2: Module-wise Implementation Summary.....	74
Table 7.1: Testable Units and the Techniques Applied to Them	83
Table 7.2: Engine Verification Test Cases and Results	84
Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6)	85

Table 7.4: Functional Test Case Design	86
Table 7.5: Edge-Case Probes and Observed Behaviour.....	89
Table 7.6: Requirements Traceability	90
Table 8.1: Worked Example: M/M/1 Queue at $\rho = 0.8$ ($\lambda = 0.8$, $\mu = 1.0$, 36,000 s, 5 replications) ...	100
Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds).....	100
Table 8.3: Engine Throughput Measured by the Verification Harness.....	102
Table 8.4: Browser Performance Measurements (to be recorded).....	102
Table 9.1: COCOMO Coefficients by Development Mode.....	104
Table 9.2: COCOMO Estimation Results.....	104
Table 9.3: Source Lines of Code by Layer	105
Table 9.4: Measured Project Size	105
Table 9.5: Standard IFPUG Complexity Weights.....	106
Table 9.6: Function Point Count by Component Type	106
Table 9.7: General System Characteristics and Degrees of Influence	107
Table 9.8: Comparison of Estimates against Planned Effort	108
Table 9.9: Development and Operating Costs	109
Table 9.10: Benefits of the Platform.....	109

LIST OF ABBREVIATIONS

Term	Meaning	Term	Meaning
AI	Artificial Intelligence	NFR	Non-Functional Requirement
API	Application Programming Interface	OAuth	Open Authorization
COCOMO	Constructive Cost Model	PK / FK	Primary Key / Foreign Key
CRUD	Create, Read, Update, Delete	PM	Person-Months
CSV	Comma-Separated Values	PRNG	Pseudo-Random Number Generator
DES	Discrete Event Simulation	Pyodide	CPython distribution for WebAssembly
DFD	Data Flow Diagram	RBAC	Role-Based Access Control
ERD	Entity Relationship Diagram	RLS	Row-Level Security
FIFO / LIFO	First / Last In, First Out	SDLC	Software Development Life Cycle
FP / FPA	Function Point / Analysis	SimPy	Simulation in Python (DES library)
FR	Functional Requirement	SLOC / KSLOC	(Thousand) Source Lines of Code
HCI	Human–Computer Interaction	SPA	Single-Page Application
JSON / JSONB	JavaScript Object Notation / Binary	SRS	Software Requirements Specification
KPI	Key Performance Indicator	TLS	Transport Layer Security
LLM	Large Language Model	UAT	User Acceptance Testing
M/M/1, M/M/c	Markovian queues with 1 or c servers	UFP / VAF	Unadjusted FP / Value Adjustment Factor
ML	Machine Learning	UML	Unified Modeling Language
WASM	WebAssembly	UUID	Universally Unique Identifier

1. INTRODUCTION

Every organisation that runs a queue, a production line, a hospital ward or a delivery route faces the same recurring question: *what will happen if we change this?* Adding a counter, re-sequencing a workstation or re-routing a fleet are expensive to try in the real world and expensive to get wrong. Discrete Event Simulation (DES) answers such questions on a computer before any money is spent, but it has traditionally demanded costly licences, programming skill and a local installation. **JustCmul8** was conceived to remove those three barriers at once. This chapter sets out why the project was undertaken, what it aims to achieve, the problem it solves and the boundaries within which it operates.

1.1 Background and Motivation

1.1.1 Simulation as a Decision-Making Tool

Operational systems are governed by variability and contention. Customers do not arrive at perfectly regular intervals, machines do not take exactly the same time to process every part, and two entities frequently compete for one server. Averages hide these effects: a counter that serves a customer in four minutes on average can still produce a long queue if arrivals come in bursts. Simulation captures the variability directly by generating thousands of random arrivals and service times and observing what the system does with them, so the decision-maker sees not only the expected outcome but also its spread.

Simulation is applied across manufacturing, healthcare, logistics, transport, telecommunications and public services. Its value lies in the fact that experiments which would be disruptive, dangerous or prohibitively expensive in reality become cheap and repeatable on a computer. In a hospital, for instance, the effect of a third triage nurse can be examined without a single patient being inconvenienced.

1.1.2 Theoretical Background

The project draws on several established areas of computer science, operations research and software engineering. The concepts below were applied directly in the design and implementation.

Discrete Event Simulation and continuous simulation. DES represents a system as a sequence of events occurring at distinct instants of simulated time. Between two consecutive events the state of the system does not change, so the simulation clock jumps from one event

to the next instead of ticking through uniform intervals. A customer arriving, a machine finishing a job and a packet reaching a router are all events in this sense. Continuous simulation, by contrast, describes state through variables that change smoothly and are integrated over small fixed steps. Because a discrete event simulation skips the intervals in which nothing happens, it can simulate a full working day in a fraction of a second.

Table 1.1: Comparison of Discrete Event and Continuous Simulation

Aspect	Discrete Event Simulation	Continuous Simulation
Time advance	Jumps to the next scheduled event	Advances in small fixed increments
State change	Instantaneous, at event instants only	Smooth and continuous throughout
Mathematical basis	Probability and queueing theory	Differential and difference equations
Typical application	Queues, production lines, supply chains	Thermal, hydraulic and control systems
Computational cost	Proportional to the number of events	Proportional to the simulated duration
Role in JustCmul8	Primary execution paradigm	Approximated by container-level sampling

Probability distributions. Realistic simulation depends on representing variability correctly. Inter-arrival and service times are therefore sampled from probability distributions. JustCmul8 implements five samplers, each chosen for a distinct modelling purpose.

Table 1.2: Probability Distributions Implemented in JustCmul8

Distribution	Parameters	Characteristics	Typical use
Exponential	mean ($1/\lambda$)	Memoryless; short intervals frequent, long ones rare	Inter-arrival times of independent arrivals
Uniform	min, max (mean $\pm$ 50%)	All values in the range equally likely	A duration known only to lie between two bounds
Normal	mean, std. deviation	Symmetric about the mean; truncated at zero	Machine cycle times with small random error
Poisson	λ (mean rate)	Arrival counts in a fixed interval; exponential gaps	Arrivals per period
Deterministic	fixed value	No variability; repeats exactly	Conveyor transfers and scheduled operations

Queueing theory. Analytical results for simple queues provide an independent yardstick for a simulator. For an M/M/1 queue with arrival rate λ and service rate μ , utilisation is $\rho = \lambda/\mu$, the mean waiting time in the queue is $W_q = \rho/(\mu - \lambda)$, and the mean number waiting is $L_q = \rho^2/(1$

– ρ). For multi-server queues the Erlang-C formula gives the corresponding values. **Little’s Law**, $L = \lambda W$, relates the mean number of entities in a system to their arrival rate and mean time in the system, and holds for virtually any stable queueing system. JustCmul8 uses these results both to verify its engine and, at run time, to check the consistency of every simulation it executes.

Monte Carlo estimation. A single stochastic run is one observation drawn from a distribution of possible outcomes. Monte Carlo analysis executes the model repeatedly with independent random streams and treats the collected outcomes as a sample from which the underlying behaviour can be estimated. The 95 per cent confidence interval for a metric is the sample mean plus or minus 1.96 times the standard error, the standard error being the sample standard deviation divided by the square root of the number of replications.

Supporting concepts. The Software Development Life Cycle (SDLC) organised the work into analysis, design, implementation, testing and deployment. A multi-tier architecture separates the presentation, state, domain, execution and persistence concerns so that each can be tested independently. Graph theory underpins the model itself, since a simulation is a directed graph whose validity is checked by graph traversal. Relational database concepts — primary and foreign keys, referential integrity and row-level security — protect user data. Finally, human–computer interaction principles such as plain-language labelling, low cognitive load and familiar drag-and-drop gestures keep the tool usable by people who have never programmed.

1.1.3 Motivation

Three observations motivated the project. First, the cost of commercial simulators places them beyond the reach of students, small businesses and most educational institutions, precisely the audiences that stand to gain most from a low-cost way to test decisions. Second, the best free engine, SimPy, is a code library, so a student who understands queues but not generator functions cannot use it. Third, modern browsers now run WebAssembly at close to native speed, which for the first time makes it possible to execute a real Python simulation engine on the user’s own device with no server and no installation.

The combination of these three observations defined the project: a free platform that anyone can open in a browser, that speaks the language of the user’s domain rather than the language of simulation software, and that runs an authoritative, verifiable engine underneath. The addition of an AI assistant and real-time collaboration reflects how people now expect to work — describing what they want in plain language and sharing a live workspace with colleagues.

1.2 Objective and Scope of the Project

1.2.1 Objective

The primary objective of JustCmul8 is to design, develop and publish a no-code, free Discrete Event Simulation platform that allows students, researchers, startups and enterprises to build, run, analyse and share simulation models entirely within a web browser, without installing software, learning a programming language or purchasing a licence.

The platform offers an intuitive drag-and-drop modelling environment backed by a real Python SimPy engine running through WebAssembly. Users therefore obtain precise and reliable results while enjoying the convenience of a modern web application. An AI simulation assistant translates plain-English process descriptions into executable models, interactive dashboards present the results, and a digital-twin animation lets users watch the system in action.

The specific objectives of the project are as follows.

- **Eliminate cost and installation barriers** — deliver a platform that needs no licence fee and no local installation beyond a modern browser.
- **Preserve simulation accuracy** — execute a real Python SimPy engine rather than a visual approximation, and verify it against analytical queueing theory.
- **Remove the coding prerequisite** — provide a visual modelling interface that a person who has never programmed can use.
- **Quantify uncertainty** — report confidence intervals from Monte Carlo replications instead of a single deterministic figure.
- **Surface actionable insight** — detect the bottleneck automatically, check results with Little's Law, score overall system health and propose concrete remedial changes with an estimate of their financial effect.
- **Enable collaboration and sharing** — support live presence and cursors in a shared workspace, and revocable read-only share links for stakeholders.
- **Design for extension** — structure the system so that new domains, node types and engines can be added through a registry without redesigning the core.

1.2.2 Scope of the Project

JustCmul8 encompasses the design and development of a no-code, browser-based DES platform that lets users build, run, analyse and share simulation models in one application. The delivered system is divided into twelve functional modules grouped into four areas that together cover the complete workflow, from account creation to the export of results.

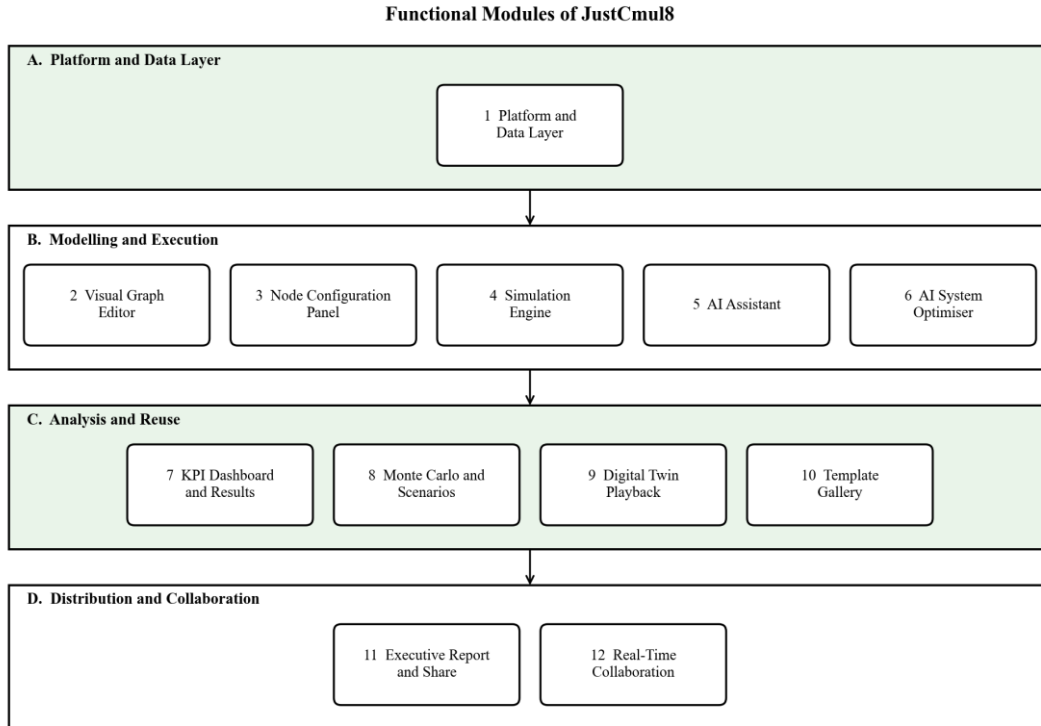


Figure 1.1: Functional Modules of JustCmul8 grouped by area

- **Platform and data layer** — secure registration and login by email or Google/GitHub OAuth, session handling, project creation, renaming and deletion, automatic cloud persistence, and profile and preference management (Module 1).
- **Modelling and execution** — the visual graph editor, the node configuration panel, the Pyodide-hosted SimPy simulation engine, the AI assistant that generates models from natural language, and the AI optimiser that diagnoses a completed run and proposes a fix (Modules 2 to 6).
- **Analysis and reuse** — the KPI dashboard with bottleneck detection and cost analysis, the Monte Carlo and scenario comparison facility, the animated digital-twin playback and the template gallery (Modules 7 to 10).
- **Distribution and collaboration** — export of graphs as JSON and results as CSV, executive PDF reports, Jupyter notebook export, read-only public share links, and real-time presence and collaborative editing (Modules 11 and 12).

Modelling support spans six application domains — human queue systems, vehicle traffic, liquid and material flow, manufacturing processes, logistics and warehousing, and network and signal flow. All six are driven by the same simulation engine and share a library of fifteen configurable node types that together cover a wide class of real-world scenarios.

1.2.3 Out of Scope

Some capabilities were deliberately excluded from this release so that the project remains achievable within the academic programme. They are recorded here so that a considered boundary is not mistaken for an oversight, and several of them appear in the roadmap of Chapter 10.

- **Native mobile applications** — the interface is responsive and works in mobile browsers; no separate Android or iOS application is included.
- **Enterprise services** — single sign-on, LDAP integration, organisation-level administration and subscription billing.
- **Learning-management features** — the platform is a simulation tool, not a courseware system.
- **Offline and self-hosted deployment** — the application is delivered as a cloud-hosted web application.
- **Industrial hardware integration** — direct connection to IoT devices, PLCs, sensors or robotics.
- **Automated optimisation** — the AI optimiser proposes a change for human approval; it does not search the parameter space automatically.
- **Desktop distribution** — no packaged Windows, macOS or Linux build, consistent with the browser-first approach.

1.3 Problem Definition

Discrete Event Simulation is widely used to analyse and improve manufacturing, healthcare, logistics, transportation and service operations, yet its adoption outside large organisations remains narrow. The reasons are practical rather than theoretical, and they fall into four groups.

1.3.1 The Cost of Commercial Simulation Software

Established packages such as AnyLogic, Arena and Simul8 are licensed at prices that run into thousands of dollars per seat per year. For a department teaching operations research to sixty students, or for a restaurant owner wishing to test whether a second service counter would pay for itself, the licence alone can exceed the value of the decision being informed. Simulation therefore becomes available to the organisations that could most easily absorb the cost of guessing wrongly, and unavailable to those that could not.

1.3.2 The Technical Complexity of Free Alternatives

Capable open-source frameworks exist, SimPy foremost among them, but they are programming libraries rather than applications. Before a first model will run, the user must install Python, manage a virtual environment, learn generator functions and the yield statement, understand how resource requests are expressed and debug the inevitable errors. The simulation concepts and the Python idioms must be acquired simultaneously, a considerable demand on someone whose actual question is whether to open another counter.

1.3.3 The Limits of Desktop Distribution

Most simulation software is desktop software that requires installation, configuration and maintenance. In a shared college laboratory, on a managed corporate machine or on a Chromebook, students and staff frequently lack the privileges to install anything. A tool that cannot be opened is of no use whatever its analytical merits, and this obstacle is independent of cost or skill.

1.3.4 The Inadequacy of Static Diagrams and Spreadsheets

Two substitutes are commonly attempted and both fail for the same reason. Diagramming tools such as Visio and Lucidchart produce clear process maps, but those maps are inert: a rectangle labelled “Inspection: 5 min” carries no execution semantics, so the diagram cannot reveal that inspection will accumulate a queue of thirty items by mid-afternoon. Spreadsheet models compute something, but a spreadsheet has no natural representation of an event queue, of entities competing for a resource, or of a server that is busy when the next arrival appears. Neither substitute captures contention, which is the phenomenon that causes queues in the first place.

1.3.5 Statement of the Problem

The combined effect of these barriers is that a technique capable of answering ordinary operational questions cheaply is confined to organisations with both the budget for licences and the staff trained to use them. What is required is a platform that is free to use, needs no installation, demands no programming knowledge, is reachable from any modern browser and — critically — does not trade away the accuracy of the underlying engine in exchange for that accessibility. JustCmul8 was built to meet exactly that requirement.

1.4 Limitations

Every design makes trade-offs. The limitations of the current release are stated plainly below; each is a consequence of a deliberate architectural choice, and each has a corresponding entry in the roadmap of Section 10.3.

- **Client-side resources bound model size** — because execution takes place on the user's own device, the memory available to the browser sets the practical ceiling on the size of a model and the length of a run. This is a memory limit rather than a speed limit.
- **Monte Carlo replications run sequentially** — replications are executed one after another in a single worker, so the elapsed time grows linearly with the replication count even though the runs are independent.
- **First-run runtime download** — the Pyodide runtime is downloaded on first use and cached by the browser afterwards, so the very first simulation in a session takes longer to start.
- **Connectivity is required for cloud features** — persistence, collaboration and AI assistance depend on network access. A loaded model can still be executed without a connection, but nothing is saved.
- **Collaboration is project-scoped** — live presence and graph operations are shared over a private channel authorised by the project's row-level security; role-based invitations with editor and viewer permissions are part of the roadmap.
- **AI output requires human judgement** — generated graphs and recommendations are validated and presented for approval, but are not guaranteed to be a faithful model of the described process.
- **Single simulation paradigm** — the engine is SimPy-based; continuous and agent-based paradigms are approximated (for example, the container node samples level at event boundaries).
- **No native mobile client** — the interface is responsive, but precise node placement on a small touchscreen remains awkward.

2. REQUIREMENT ANALYSIS

This chapter records what the system is required to do and for whom. It opens with the Software Requirements Specification, identifies the classes of user the platform serves and states the functional and non-functional requirements individually, each with an identifier that is referred to throughout the rest of the report. It then examines the feasibility of the project, describes the hardware and software used, and presents the proposed system together with its advantages.

2.1 User Requirements / SRS / Functional Requirements

2.1.1 Purpose and Overall Description

JustCmul8 is a web-based simulation platform accessed through a standard browser on a desktop, laptop, tablet or phone. It follows a multi-tier structure comprising a user-interface layer, an application and domain-logic layer, a client-side execution layer and a managed database layer, and it requires no client installation beyond the browser. The design is deliberately client-heavy: the interface, the application state and the entire simulation computation reside on the client, while the server is responsible only for persistence, authentication, real-time message relay and proxying requests to the AI service.

2.1.2 User Classes and Characteristics

Table 2.1: User Classes and Access Levels

User class	Responsibilities	Access level	Skill required
Registered User (Project Owner)	Creates projects, builds and configures models, runs simulations, uses AI, exports and shares results	Full control of owned projects	None (no-code)
Collaborator	Joins a project's live workspace to view presence, cursors and shared edits	Project-scoped, authorised by row-level security	None
Guest / Public Viewer	Opens a public share link to view a model read-only	Read only; no account required	None

No user class requires programming ability, which follows directly from the no-code objective. The separation of the read-only guest from the owner exists so that a model may be circulated for comment without exposing it to accidental modification.

2.1.3 Operating Environment

- **Client:** a modern web browser — Google Chrome, Mozilla Firefox, Microsoft Edge or Safari — with WebAssembly support, on Windows, macOS, Linux, Android or iOS, with a responsive interface for desktops, tablets and phones.
- **Server:** Supabase provides PostgreSQL, authentication, real-time channels and storage; a Next.js server runtime hosts the pages and a small set of API routes.
- **Simulation:** the SimPy engine executes inside the browser through Pyodide in a Web Worker, so server-side computational requirements are minimal.

2.1.4 Design and Implementation Constraints

- The platform must be fully browser-based, eliminating installation on the client device.
- Access control must be enforced in the database (row-level security) so that a user can reach only their own projects or projects explicitly shared with them.
- The interface must remain intuitive and responsive for users with little or no programming knowledge.
- The architecture must be modular and registry-driven so that new domains and node types can be added without redesigning the core.
- Secrets, in particular the AI service key, must remain on the server and never be shipped in the client bundle.

2.1.5 Functional Requirements

The functional requirements are grouped by module and carry identifiers that are used in the test cases of Chapter 7.

Table 2.2: Functional Requirements

ID	Module	Requirement
FR-01	Platform	The system shall support registration and login by email and password, and by Google or GitHub OAuth, with secure logout.
FR-02	Platform	The system shall maintain an authenticated session across visits and guard protected routes before any project data is fetched.
FR-03	Platform	The system shall allow a user to reset a forgotten password through an emailed link and to set a new password.
FR-04	Platform	The system shall allow an authenticated user to create, open, rename and delete simulation projects, choosing one of six domains.
FR-05	Platform	The system shall persist project graph state to cloud storage automatically, without an explicit save action.

ID	Module	Requirement
FR-06	Platform	The system shall present a dashboard listing the user's projects with domain and last-modified time, and shall permit profile, preference and account management including account deletion.
FR-07	Graph Editor	The system shall provide a drag-and-drop canvas supporting fifteen node types with placement, movement, connection and deletion.
FR-08	Graph Editor	The system shall re-route edges live as nodes move and shall support zoom, pan and fit-to-view navigation.
FR-09	Graph Editor	The system shall validate the graph structure before execution and report the offending node when the graph cannot be run.
FR-10	Node Configuration	The system shall allow the parameters of the selected node — arrival rate, capacity, service-time distribution, routing and costs — to be edited in a side panel showing only the fields relevant to that node type.
FR-11	Node Configuration	The system shall validate parameter values against the node type before they are used by the engine.
FR-12	Simulation Engine	The system shall compile the graph into a Python SimPy script and execute it client-side through Pyodide, performing no simulation computation on the server.
FR-13	Simulation Engine	The system shall accept the simulated duration and its time unit (seconds, minutes, hours or days) before a run.
FR-14	Simulation Engine	The system shall execute the run inside a Web Worker so that the interface remains responsive.
FR-15	Simulation Engine	The system shall report run progress and status, stream log events, permit a run to be stopped, and provide playback speed control of 0.5× to 10×.
FR-16	AI Assistant	The system shall convert a natural-language description of a scenario into a candidate node-and-edge graph.
FR-17	AI Assistant	The system shall validate a generated graph, allow the user to preview and apply it, retain chat history per project, and fall back automatically across AI models when one is unavailable.
FR-18	AI Optimiser	The system shall inspect a completed run, diagnose the bottleneck and propose a concrete corrective change that can be applied in one action.
FR-19	KPI Dashboard	On completion of a run the system shall compute utilisation, average and percentile waiting time, throughput and queue length for each node.
FR-20	KPI Dashboard	The system shall flag the bottleneck node, verify results with Little's Law, and report a composite 0–100 system health score.
FR-21	KPI Dashboard	The system shall present a Cost and ROI analysis based on resource, holding and waiting-penalty cost parameters.
FR-22	Monte Carlo	The system shall execute a model over 10, 20, 50 or 100 replications, report 95% confidence intervals, and compare a baseline against a modified scenario.
FR-23	Digital Twin	The system shall provide animated two-dimensional playback of entity movement with play, pause, step and speed controls.
FR-24	Template Gallery	The system shall offer starter templates across all six domains, each creating an independent project owned by the requesting user.

ID	Module	Requirement
FR-25	Export and Share	The system shall export the graph as JSON, results as CSV, the generated Python script as a Jupyter notebook and an executive report as PDF.
FR-26	Export and Share	The system shall generate and revoke a read-only public share link that never confers edit access and shall display a clear notice for a revoked or invalid link.
FR-27	Collaboration	The system shall show live presence and cursors of users in the same project and broadcast graph operations over an authenticated private channel.

2.1.6 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Canvas interactions such as dragging and connecting nodes shall feel immediate, and the dashboard shall render a completed run without perceptible delay.
NFR-02	Responsiveness	Simulation execution shall never block the interface; runs execute on a background worker thread.
NFR-03	Correctness	Simulated utilisation, waiting time and queue length for reference queues shall agree with analytical queueing theory within a stated tolerance.
NFR-04	Security	Passwords shall be stored only as salted hashes by the authentication provider, all traffic shall use TLS, and secret keys shall never reach the client.
NFR-05	Access Control	Row-Level Security shall be enforced on every table, so that authorisation cannot be bypassed by a modified client.
NFR-06	Usability	The interface shall be usable by a first-time, non-technical user, using domain vocabulary in preference to simulation jargon.
NFR-07	Reliability	Project state shall persist automatically between sessions and across devices.
NFR-08	Portability	The application shall run unmodified on evergreen WebAssembly-capable browsers on Windows, macOS, Linux, Android and iOS.
NFR-09	Maintainability	Simulation logic shall remain isolated behind the worker boundary, and domain and node definitions shall live in a single registry.
NFR-10	Data Integrity	Foreign-key constraints with cascading deletion shall be enforced at the database.
NFR-11	Availability	The hosted application shall target 99 per cent availability across the front end and the managed backend.

2.1.7 User Personas and Operational Workflow

Three personas guided the interface decisions, and each exercises a characteristic path through the system.

- **The student** encounters queueing theory in a lecture and wants to see M/M/1 behaviour rather than derive it. This user opens a template, alters the arrival rate, re-runs and compares the dashboards. The operative requirement is that a first meaningful result is reachable within a minute of arriving on the platform.
- **The lecturer** demonstrates congestion live in a classroom. This user needs the animated playback for visual explanation and the share link so that students can open the same model afterwards on their own devices without accounts.
- **The small-business owner** is deciding whether a second service counter is justified. This user describes the shop to the AI assistant in plain English, accepts the generated graph, runs a Monte Carlo analysis and reads the confidence intervals and the cost panel before committing money.

The common workflow that emerges is: authenticate, create or open a project, construct the graph manually or through the assistant, configure node parameters, validate the structure, execute the run, inspect the analytics, optionally re-run under a modified scenario, and finally export or share the outcome. Chapter 5 models this path formally as an activity diagram.

2.2 Feasibility Study

Before committing to the design the project was examined for viability along five dimensions. The technical question was the one that genuinely mattered, since the whole architecture depended on its answer, and it is treated first.

2.2.1 Technical Feasibility

The technologies selected — Next.js, React, TypeScript, Supabase, React Flow, Pyodide and SimPy — are widely adopted, thoroughly documented and supported by large developer communities, which reduces development risk and the long-term maintenance burden.

The principal technical question was whether a Python-based discrete event simulation could realistically execute inside a browser. Pyodide resolves it by compiling the CPython interpreter to WebAssembly, allowing the SimPy engine to run client-side with nothing installed. This was proved early, in a prototype spike, precisely because the entire architecture depended on the answer. Two constraints emerged and were designed around rather than ignored. First, the Pyodide runtime is a download of several megabytes, so it is fetched once and cached. Second, WebAssembly executes within the browser's memory allocation, which places a practical ceiling on model size; hosting execution in a Web Worker keeps the interface responsive while a large model runs.

Canvas rendering posed a smaller but real question, since manipulating hundreds of interactive nodes in the DOM can degrade sharply. React Flow, which is built for node-based interfaces, was adopted for the editor, and Pixi.js, which renders through WebGL, for the animated playback where many entities move at once.

2.2.2 Economic Feasibility

The project is economically viable because it rests almost entirely on open-source software and on cloud services with generous free tiers. React, Next.js, TypeScript, SimPy, Pyodide, React Flow, Zustand, Recharts and Pixi.js carry no licence fee, in contrast with the commercial packages discussed in Section 1.3.

The decision to run simulations on the client has a direct economic consequence. Because the user's own browser performs the computation, there is no server-side compute cost that grows with the number or size of simulations: a thousand users each running an hour-long simulation impose no more server load than a thousand users reading a static page. The only recurring costs are the managed database tier and the metered AI quota, which are modest because an AI request is issued only when the user explicitly asks for generation or diagnosis.

Table 2.4: Indicative Cost Comparison with Commercial Alternatives

Item	Commercial DES tool	JustCmul8
Software licence	Typically thousands of USD per seat per year	None — open-source stack
Installation and IT effort	Required on every machine	None — runs in the browser
Simulation compute	Local workstation or licensed server	User's browser — no server cost
Backend hosting	Not applicable or self-managed	Managed free tier during the academic phase
AI assistance	Generally unavailable	Gemini API, metered per explicit request
Training	Formal instruction usually necessary	Not required — no-code interface

The figures for commercial tools are indicative, being drawn from vendor price lists that change frequently; the qualitative comparison is what matters for the argument.

2.2.3 Operational Feasibility

Operationally the platform is straightforward to adopt. The drag-and-drop interface, supported by the AI assistant, allows a model to be constructed without programming or prior simulation

experience, and because everything is reached through a browser no software need be installed or maintained. The intended users — students, teachers, researchers and small-business owners — are already familiar with modern web applications, so formal training and dedicated technical support are unnecessary. Deliberate interface choices reinforce this: palettes are labelled in the vocabulary of the chosen domain, so a user modelling a bank sees “Teller” rather than “Resource”, and result tabs carry plain-language names such as “Line Buildup Timeline” and “Station Counters”.

2.2.4 Schedule Feasibility

The development plan is realistic within the academic timeframe. The work is divided into discrete modules sequenced by their dependencies, with core capability delivered before advanced features such as AI assistance and collaboration. Testing, debugging and integration run alongside development rather than after it. The most consequential scheduling decision was to begin the simulation engine in the second week of July, in parallel with backend work, so that the hardest technical uncertainty was resolved while there was still time to respond to it. Chapter 3 sets out the twenty-four-week schedule in full.

2.2.5 Legal and Ethical Feasibility

Every dependency is used under an open-source licence or within the terms of a free service tier, so no licensing obligation is outstanding. User data is protected by PostgreSQL Row-Level Security, confining each user to their own projects, and passwords are never stored in recoverable form. Account deletion is implemented as a genuine cascading removal of the user’s data. Public share links are revocable at any time, after which the circulated URL shows an explanatory notice instead of the project. AI-generated graphs are always presented for human review before they alter a project, so the assistant advises but never acts unilaterally.

2.3 Details of Hardware and Software Used

Two configurations are relevant and easily confused. The first is the environment in which the platform was developed, which is comparatively demanding because it runs an editor, a development server and the browser runtime at once. The second is what an end user actually needs, which is considerably less, and it is the second that determines how widely the platform can be adopted.

2.3.1 Development Environment

Table 2.5: Development Hardware and Software Configuration

Component	Specification	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or higher	Runs the development server, browser and Pyodide runtime concurrently
Memory	8 – 16 GB RAM	Editor, browser with several tabs and local build processes together
Storage	256 GB SSD or larger	Sources, Node.js dependency tree and build artefacts
Display	1920 × 1080 or higher	Canvas, configuration panel and dashboard side by side
Operating system	Windows 11	Primary development platform
Network	Stable broadband connection	Package installation, Supabase and AI API access
Editor	Visual Studio Code with TypeScript and ESLint	Primary development environment
Version control	Git with a GitHub remote	Source history, branching and collaborative development
API testing	Postman	Verification of API routes
Runtime	Node.js LTS with npm	Dependency management and the Next.js build pipeline
Verification	Python 3 with SimPy	Native harness used to verify the engine against queueing theory
Browser tooling	Chrome DevTools	Profiling, memory inspection and performance measurement

2.3.2 Target Client Requirements

Because JustCmul8 is a web application, end users require no specialised hardware. Any reasonably current desktop, laptop, tablet or smartphone with a modern browser is sufficient. One qualification follows from the architecture: since the engine executes client-side, devices with limited memory will run very large models more slowly, and the Web Worker execution path exists specifically to keep the interface usable when that happens.

Table 2.6: Minimum and Recommended Client Requirements

Requirement	Minimum	Recommended
Browser	Current Chrome, Edge, Firefox or Safari with WebAssembly	Latest evergreen release of Chrome or Edge
Memory	4 GB RAM	8 GB RAM or more for large models

Requirement	Minimum	Recommended
Processor	Dual-core	Quad-core or better
Display	1366 × 768	1920 × 1080 or higher
Network	Stable connection for loading and persistence	Broadband for real-time collaboration
Local installation	None	None

2.3.3 Software and Technology Stack

Table 2.7: Technology Stack with Justification

Layer	Technology	Reason for selection
Frontend framework	Next.js 16 (App Router)	File-based routing, server rendering and API routes in one framework
UI library	React 19	Component model suited to a complex, highly interactive editor
Language	TypeScript	Static typing across the code base, catching parameter mismatches at compile time
Styling	Tailwind CSS v4	Utility-first styling giving consistent spacing and a coherent theme
Graph editor	React Flow (@xyflow/react)	Production-grade node-based canvas with handles and edge routing
Graph layout	Dagre	Automatic layered layout for generated and template graphs
Simulation engine	Python SimPy via Pyodide (WebAssembly)	A real DES engine executing client-side
State management	Zustand	Lightweight store with slice-based separation
Charts	Recharts	Declarative, composable charts for the KPI dashboard
2D animation	Pixi.js	WebGL rendering for the digital-twin playback
AI service	Google Gemini (via a model router)	Structured generation of graphs and diagnoses from natural language
Database and auth	Supabase (PostgreSQL, Auth, Realtime)	Managed Postgres with Row-Level Security, OAuth and WebSocket channels
Reporting	jsPDF, html-to-image	Client-side generation of executive PDF reports
Animation and scroll	Framer Motion, Lenis	Transition choreography and smooth scrolling on the landing page
Icons	Lucide React	Consistent icon set covering all node types

2.4 Proposed System and Advantages of the Proposed System

2.4.1 The Proposed System

The proposed system is JustCmul8, a browser-native discrete event simulation platform in which the user drags nodes onto a canvas, sets their parameters in forms, presses **Run** and reads the answer in plain language. The graph is translated into a genuine SimPy program and executed by CPython compiled to WebAssembly inside a Web Worker. The output flows through an analytics layer that computes performance indicators, verifies them, ranks the constraint and translates them into recommendations, cost figures and an animated replay.

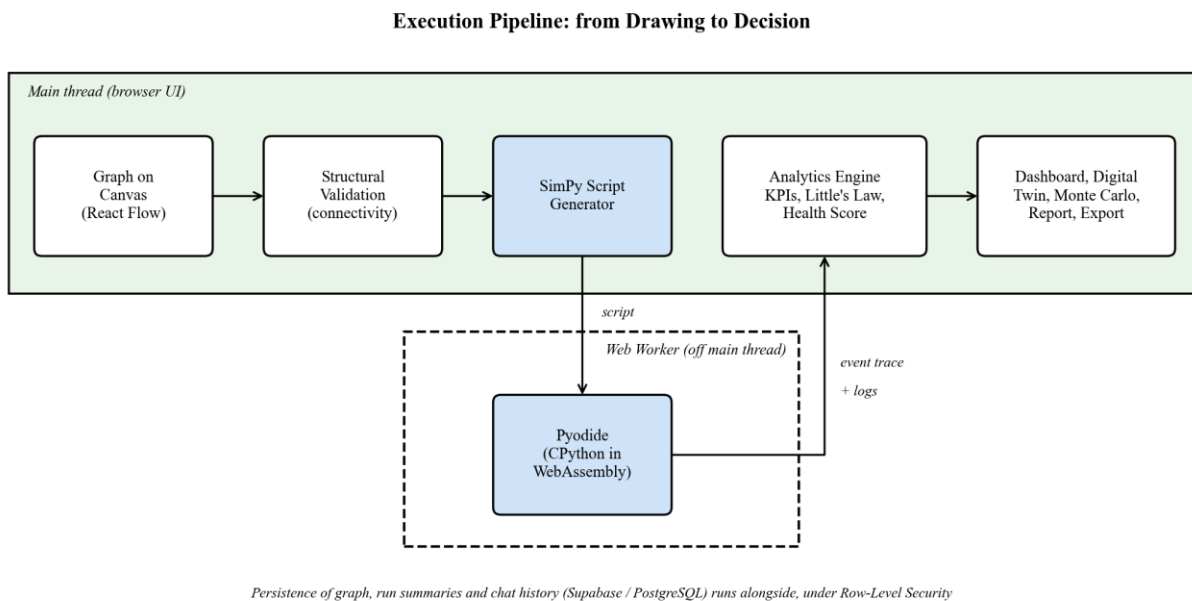


Figure 2.1: Execution Pipeline of the Proposed System

2.4.2 Comparison with Existing Approaches

Table 2.8: Comparison of the Proposed System with Existing Approaches

Capability	Commercial DES tools	SimPy library	Diagram tools / spreadsheets	JustCmul8
Free to use	No	Yes	Partly	Yes
No installation	No	No	Yes	Yes
No programming needed	Partly	No	Yes	Yes
Executes real DES	Yes	Yes	No	Yes (SimPy)
Runs in the browser	Rarely	No	Yes	Yes
AI-assisted modelling	Rarely	No	No	Yes
Confidence intervals (Monte Carlo)	Yes	Manual coding	No	Yes, one click

Capability	Commercial DES tools	SimPy library	Diagram tools / spreadsheets	JustCmul8
Live collaboration and sharing	Limited	No	Yes	Yes
Readable generated code	No	Yes	No	Yes (inspector and notebook)

2.4.3 Advantages of the Proposed System

- **Zero cost and zero installation** — free to use, and reachable from any laptop, lab machine or Chromebook through a URL.
- **Authoritative results** — numbers come from SimPy, not from an imitation, and the engine has been verified against queueing theory.
- **No-code modelling** — domain vocabulary, form-based configuration and a template gallery let a first-time user obtain a result within a minute.
- **AI-assisted workflow** — a plain-English description becomes a runnable model, and a completed run becomes a specific, costed recommendation.
- **Decisions with uncertainty attached** — Monte Carlo confidence intervals show whether an apparent improvement is larger than the noise.
- **Transparent and reproducible** — the generated Python is visible in the code inspector and exportable as a Jupyter notebook, so any result can be re-run independently.
- **Understandable output** — bottleneck flags, a system health score, a flow-balance check, a traffic heatmap, an entity tracer and an animated digital twin translate statistics into pictures.
- **Secure by design** — authorisation is enforced by Row-Level Security in the database; share links are opaque, read-only and revocable.
- **Cheap to scale** — because computation happens on the client, hosting cost is essentially independent of how heavily the platform is used.
- **Extensible** — a single registry defines each domain’s vocabulary, palette, layout and templates, so new domains and node types add without touching the engine.

3. PROJECT PLANNING

Good planning turns an ambitious idea into a sequence of achievable steps. This chapter describes how the development of JustCmul8 was organised: the activities that make up the project, the time allotted to each, the graphical schedule, the resources required, and the development model that governs the way work is carried out.

3.1 List of Activities / Tasks

The development of JustCmul8 was structured to follow the systematic phases of the Software Development Life Cycle: initiation and analysis, design, development, testing, documentation and deployment. The work was divided into twenty-seven well-defined activities, each with a realistic duration and a recorded dependency, so that no activity could begin before the work it relied upon was complete. The complete schedule spans twenty-four working weeks, from the first week of July 2026 to the last week of December 2026.

Dependency mapping served two purposes. It prevented any module from being implemented before its prerequisites existed and, equally important, it identified which activities were independent and could therefore run in parallel, shortening the overall duration without compromising quality. The most consequential decision concerned the **Simulation Engine**, which starts in the second week of July and proceeds in parallel with backend development. Running Python SimPy inside a browser through Pyodide and WebAssembly is the principal technical risk of the project, and every dependent module — the AI assistant, the KPI dashboard, Monte Carlo analysis and the digital twin — presumes it will work. Starting early leaves time to resolve compatibility problems, verify accuracy against known analytical results and stabilise the interface that the other modules will call.

A note on module numbering is necessary. The schedule below was drawn against the eleven-module breakdown of the project synopsis, which is the decomposition in force when planning began: Authentication, Project Management, Visual Graph Editor, Node Configuration Panel, Simulation Engine, AI Assistant, KPI Dashboard, Template Gallery, Export and Share, User Profile and Settings, and Real-Time Collaboration. During implementation the delivered system was reorganised into **twelve modules** (Chapter 4): Authentication, Project Management and Profile were consolidated into a single Platform and Data Layer, while three capabilities that grew beyond their original scope — the AI System Optimiser, Monte Carlo and Scenario Comparison, and the Digital Twin Playback — became modules in their own right. The activity

list therefore records the plan as it was made, and Chapter 4 records the system as it was delivered.

Table 3.1: List of Activities and Tasks with Dependencies

Sr.	Task / Activity	Phase	Duration	Depends on
1	Project Initiation	Initiation / Analysis	2 weeks	—
2	Requirement Gathering	Initiation / Analysis	3 weeks	—
3	Literature Survey	Initiation / Analysis	2 weeks	Project Initiation
4	System Analysis	Initiation / Analysis	2 weeks	Requirement Gathering
5	UI/UX Design	Design	3 weeks	System Analysis
6	Database Design	Design	3 weeks	System Analysis
7	Module: Authentication	Development	3 weeks	Database Design
8	Module: Project Management	Development	3 weeks	Authentication
9	API Development	Development	5 weeks	Database Design
10	Backend Development	Development	8 weeks	API Development
11	Module: Visual Graph Editor	Development	4 weeks	UI/UX Design, Project Management
12	Module: Node Configuration Panel	Development	3 weeks	Visual Graph Editor
13	Frontend Development	Development	9 weeks	UI/UX Design
14	Module: Simulation Engine	Development	8 weeks	Early parallel track; runs ahead of backend
15	Module: AI Assistant	Development	4 weeks	Simulation Engine
16	Integration	Development	5 weeks	Modules 1 – 6 of the plan
17	Module: KPI Dashboard	Development	3 weeks	Simulation Engine
18	Module: Template Gallery	Development	3 weeks	Visual Graph Editor
19	Module: Export and Share	Development	3 weeks	KPI Dashboard
20	Module: User Profile and Settings	Development	3 weeks	Authentication
21	Module: Real-Time Collaboration	Development	4 weeks	Visual Graph Editor
22	Unit Testing	Testing	5 weeks	Modules in progress
23	Integration Testing	Testing	3 weeks	Integration complete
24	Bug Fixing	Testing	3 weeks	Integration Testing
25	User Acceptance Testing (UAT)	Testing	2 weeks	Bug Fixing
26	Documentation	Documentation	23 weeks	Continuous, alongside every module
27	Final Review and Deployment	Deployment	4 weeks	User Acceptance Testing

The phases are described below.

- **Initiation and Analysis** — objectives, intended users and requirements were established; the commercial and open-source tools discussed in Section 1.3 were surveyed; and the findings were consolidated into a specification before any code was written.
- **Design** — interface design produced the workspace layout, node palette organisation and configuration panel behaviour, while database design produced the schema set out in Chapter 5.
- **Development** — the largest share of the schedule, run as three concurrent streams: backend and API (authentication, project management, data layer), simulation (engine, AI assistant, KPI dashboard, templates) and frontend (graph editor, configuration panel, export, profile and collaboration).
- **Testing** — unit testing begins while modules are still under construction so that defects in earlier modules are corrected while later ones are being written; integration testing, bug fixing and user acceptance testing follow in sequence.
- **Documentation** — written continuously for twenty-three of the twenty-four weeks, so that the record stays accurate.
- **Deployment** — a final review followed by release to the hosting platform.

3.2 Timeline for Each Task

The table below gives the planned start and finish week of each activity. Weeks are expressed relative to the calendar month, W1 denoting the first week of that month.

Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026)

Sr.	Task / Activity	Duration	Start	End
1	Project Initiation	2 weeks	Jul W1	Jul W2
2	Requirement Gathering	3 weeks	Jul W1	Jul W3
3	Literature Survey	2 weeks	Jul W2	Jul W3
4	System Analysis	2 weeks	Jul W3	Jul W4
5	UI/UX Design	3 weeks	Jul W4	Aug W2
6	Database Design	3 weeks	Jul W4	Aug W2
7	Module: Authentication	3 weeks	Aug W1	Aug W3
8	Module: Project Management	3 weeks	Aug W2	Aug W4

Sr.	Task / Activity	Duration	Start	End
9	API Development	5 weeks	Aug W1	Sep W1
10	Backend Development	8 weeks	Aug W1	Sep W4
11	Module: Visual Graph Editor	4 weeks	Aug W2	Sep W1
12	Module: Node Configuration Panel	3 weeks	Aug W3	Sep W1
13	Frontend Development	9 weeks	Aug W2	Oct W2
14	Module: Simulation Engine	8 weeks	Jul W2	Sep W1
15	Module: AI Assistant	4 weeks	Sep W2	Oct W1
16	Integration	5 weeks	Sep W3	Oct W3
17	Module: KPI Dashboard	3 weeks	Sep W3	Oct W1
18	Module: Template Gallery	3 weeks	Sep W4	Oct W2
19	Module: Export and Share	3 weeks	Oct W1	Oct W3
20	Module: User Profile and Settings	3 weeks	Oct W1	Oct W3
21	Module: Real-Time Collaboration	4 weeks	Oct W2	Nov W1
22	Unit Testing	5 weeks	Sep W4	Oct W4
23	Integration Testing	3 weeks	Oct W4	Nov W2
24	Bug Fixing	3 weeks	Nov W1	Nov W3
25	User Acceptance Testing (UAT)	2 weeks	Nov W3	Nov W4
26	Documentation	23 weeks	Jul W1	Dec W3
27a	Final Review	2 weeks	Dec W1	Dec W2
27b	Deployment	2 weeks	Dec W3	Dec W4

Project Milestones

Table 3.3: Project Milestones and Completion Criteria

Milestone	Target	Completion criterion
M1 — Requirements baselined	Jul W3	Functional and non-functional requirements approved by the guide
M2 — Literature survey complete	Jul W3	Comparable tools evaluated and the technology stack fixed
M3 — Design frozen	Aug W2	Interface layout and database schema approved
M4 — Engine proof of concept	Sep W1	A SimPy model executing in the browser with output verified against theory
M5 — Core modelling path working	Sep W1	A graph can be built, configured and executed end to end
M6 — Analytics operational	Oct W1	KPI dashboard rendering utilisation, wait time and throughput
M7 — Feature complete	Nov W1	All scheduled modules implemented and integrated

Milestone	Target	Completion criterion
M8 — Testing signed off	Nov W4	Unit, integration and acceptance testing passed
M9 — Deployed	Dec W4	Application live on the hosting platform and report submitted

Milestone M4 deserves emphasis. It was placed early specifically because it retires the project’s dominant technical risk, and every later milestone assumes that browser-hosted Python execution has already been demonstrated to work.

3.3 Gantt Chart and Activity Network Diagram

3.3.1 Gantt Chart

A Gantt chart is a scheduling instrument that presents project activities as horizontal bars positioned against a calendar, so that the duration of each activity and the overlap between activities can be read directly. It is used in software engineering for task scheduling, resource allocation and progress tracking; its principal value is that it decomposes a project into manageable activities, assigns each a realistic timeframe and makes scheduling conflicts visible early enough to be corrected.

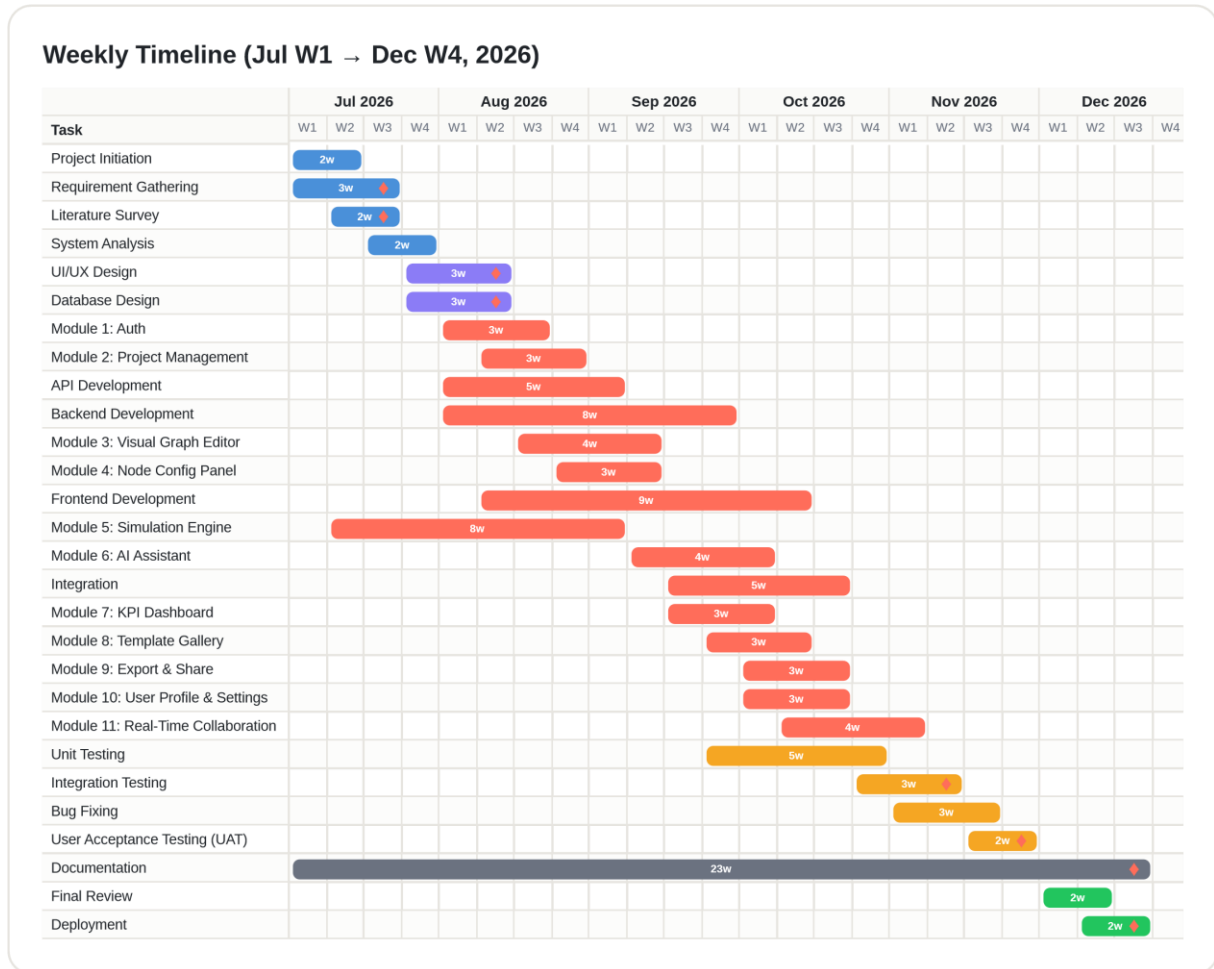


Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026)

The chart is grouped by phase, each shade corresponding to a stage of the life cycle. The first four bars represent the initiation and analysis phase, which occupies July and establishes the objectives, requirements and technology assessment on which the design depends. The next two bars mark the design phase, comprising interface and database design across roughly three weeks from late July. The development phase dominates the timeline: alongside API and backend development sit the two foundational modules, authentication and project management, followed by the visual graph editor and node configuration panel that constitute the modelling surface the user actually manipulates.

The simulation engine bar is the one to read most carefully: it begins in the second week of July and runs concurrently with backend development for the reason given in Section 3.1. The AI assistant follows once the engine has stabilised, and the remaining modules are delivered through September and October, each after its own prerequisite. The testing bars deliberately overlap the development bars, because testing begins before all modules are finished. The

single long bar is documentation, which spans almost the whole period, and the last two bars are the final review and deployment in December.

3.3.2 Activity Network Diagram

An activity network diagram is a behavioural model that shows the flow of activities through the project, including sequencing, decision points and parallel execution. Unlike a Gantt chart, which answers *when* work happens, it answers *in what logical order*, subject to which decisions and along which concurrent paths. Drawn in UML notation, it uses activity nodes, control flows, decision nodes, fork and join bars and start and end markers.

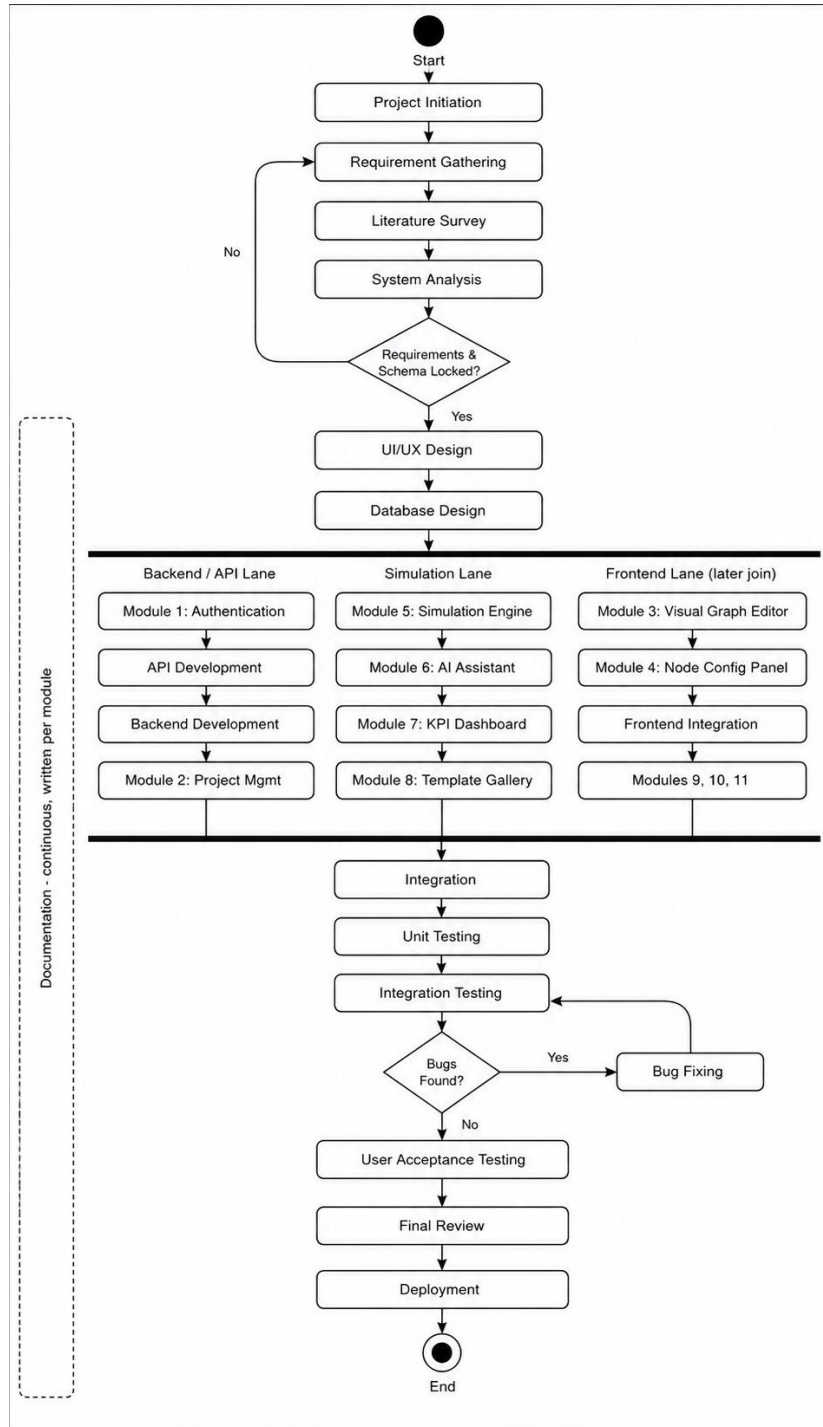


Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle

The flow begins with project initiation, requirement gathering, literature survey and system analysis. A decision node then asks whether requirements and the database schema are locked; if further refinement is needed, control returns to requirement gathering, and only once both are settled does the flow advance to design. Development is expressed as a fork into three concurrent lanes: the backend and API lane (authentication, API development, backend development, project management); the simulation lane (simulation engine followed by the AI

assistant, KPI dashboard and template gallery); and the frontend lane (graph editor, configuration panel, frontend integration, export, profile and collaboration). A join bar reunites the three lanes for system integration.

Testing begins with unit testing and proceeds to integration testing, where a second decision node asks whether defects were found. If so, control passes to bug fixing and returns to integration testing, repeating until the build is clean, after which the flow advances to user acceptance testing, the final review and deployment. Documentation runs as a continuous supporting activity alongside every phase.

3.3.3 Critical Path

The critical path — the chain of dependent activities that determines the earliest possible completion date — runs through Requirement Gathering, System Analysis, Database Design, API and Backend Development, Integration, Integration Testing, Bug Fixing, User Acceptance Testing, Final Review and Deployment. Any slippage along this chain delays delivery, whereas activities off it, such as the Template Gallery or Profile and Settings, carry float and can absorb short delays. The Simulation Engine track would have been critical had it been scheduled after the backend; running it in parallel is what removes it from the critical path and protects the final date.

The Gantt chart and the activity network diagram are complementary. Read together, they describe both the calendar and the control flow of the development strategy adopted for JustCmul8.

3.4 Resource Planning

Resource planning identifies the people, equipment, software and services required, and when. Because JustCmul8 is built on an open-source stack and free-tier cloud services, the planning effort concentrates on human effort and on making sure that the two team members' time is used where it has the greatest effect.

3.4.1 Human Resources

Table 3.4: Human Resources and Responsibilities

Resource	Role	Responsibilities
Ms. Jinal Gujar	Project Guide	Review of requirements, design and drafts; validation of milestones; academic guidance

Resource	Role	Responsibilities
Khushi Singh (261713)	Developer, Analyst and Documenter	Requirements and design documents; Platform and Data Layer; configuration panel; AI assistant; KPI dashboard; digital twin; report and share; testing support
Mohit Gupta (261715)	Developer, Architect and Tester	System architecture; graph editor; simulation engine; AI optimiser; Monte Carlo; templates; real-time collaboration; verification harness; deployment
Both members	Team	Requirement analysis, database design, integration, testing, user acceptance and the project report

3.4.2 Hardware, Software and Service Resources

Table 3.5: Hardware, Software and Service Resources

Category	Resource	Cost basis
Hardware	Two development laptops (8–16 GB RAM, SSD); college laboratory machines for cross-machine collaboration testing	Owned / provided by the college
Development software	VS Code, Git, GitHub, Node.js, Postman, Python and SimPy, Chrome DevTools	Free
Frameworks and libraries	Next.js, React, TypeScript, Tailwind CSS, React Flow, Zustand, Recharts, Pixi.js, Pyodide	Open source
Backend services	Supabase (database, authentication, realtime)	Free tier
AI service	Google Gemini API	Free tier / metered usage
Hosting	Vercel (Next.js hosting and preview deployments)	Free tier
Project management	GitHub issues and project board	Free tier

3.4.3 Effort Allocation

The nominal effort available is two team members over six months, or twelve person-months.

The allocation across phases below reflects the schedule of Section 3.2.

Table 3.6: Planned Effort Allocation by Phase

Phase	Weeks	Share of effort	Principal activity
Initiation and Analysis	4	10%	Requirements, literature survey, technology assessment
Design	3	10%	Interface design, database design, architecture
Development	18	45%	Twelve modules, API and backend, integration
Testing	9	20%	Unit, integration, security, acceptance testing

Phase	Weeks	Share of effort	Principal activity
Documentation	continuous	10%	Module documents and this report
Deployment	4	5%	Final review, configuration, release

3.5 Proposed Software Development Model

An **iterative and incremental model** combining the discipline of the waterfall model with the responsiveness of agile practice was adopted. The choice followed from two properties of this particular project.

The first is that the requirements were reasonably stable from the outset. The problem domain is well established, and the comparable tools surveyed in Section 1.3 defined the expected feature set. A pure agile approach, which derives its value from accommodating requirements that emerge during development, would have offered little advantage, and the formal documentation an academic project must produce benefits from the stage discipline of waterfall. The second is that the technical risk was concentrated in a single component: whether SimPy could execute inside a browser through Pyodide was an open question when planning began, and almost everything else depended on the answer. A strict waterfall would have postponed that discovery until implementation, by which time the design would have been fixed around an assumption that might not hold. An iterative prototyping loop was therefore inserted early, treating the engine as a spike to be proved before dependent modules were designed in detail.

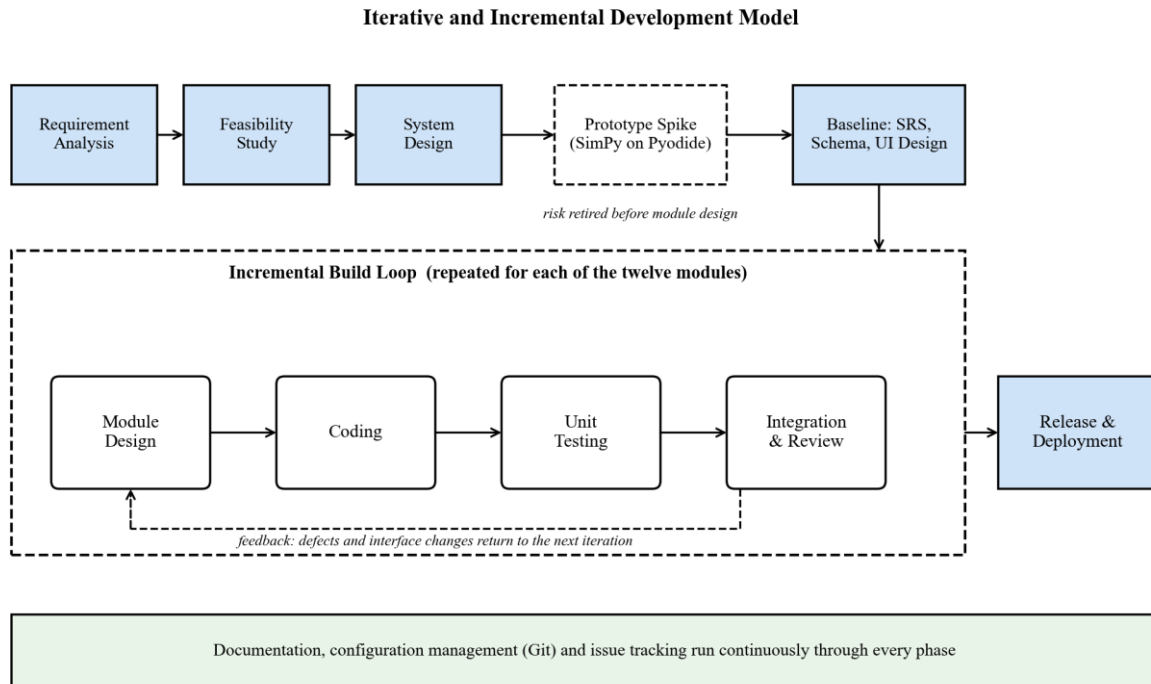


Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8

In practice the analysis and design phases proceeded in waterfall fashion, producing the requirement specification and schema before implementation began, while implementation proceeded iteratively, one module at a time, each module passing through its own cycle of design, coding, testing and integration before the next was begun. Testing was continuous rather than terminal, and documentation was written alongside each module.

Table 3.7: Phases of the Adopted Life-Cycle Model

Phase	Activities performed	Outcome
Requirement analysis	Elicitation of needs, study of comparable tools, identification of functional and non-functional requirements	Approved requirement specification (Section 2.1)
Feasibility study	Assessment of technical, economic, operational, schedule, legal and ethical viability	Confirmation that the browser-hosted approach was achievable
System design	Architecture, database schema, interface layout and UML modelling	Design documents and diagrams (Chapter 5)
Prototyping (iterative)	Spike implementation of the Pyodide and SimPy execution path, verified against analytical results	Retirement of the dominant technical risk at milestone M4
Implementation (incremental)	Module-by-module construction across three parallel streams, each with its own design-code-test cycle	Twelve delivered modules (Chapter 4)
Testing	Unit, structural, analytical, integration, security and acceptance testing alongside development	Verified system (Chapter 7)

Phase	Activities performed	Outcome
Deployment	Final review, environment configuration, database policies, release	Live application on the hosting platform
Documentation	Continuous recording of analysis, design, implementation and test outcomes	This report and module documents

Table 3.8: Why the Iterative and Incremental Model Was Preferred

Model	Strength	Weakness for this project
Waterfall	Clear stages and documentation	Defers discovery of the engine risk until late implementation
Pure Agile / Scrum	Adapts to changing requirements	Requirements were stable; formal documentation is required
Spiral	Explicit risk analysis in each loop	Heavyweight for a two-person team
Iterative and Incremental (adopted)	Early risk retirement, testable increments, disciplined documentation	Requires interface stability between modules — managed by the worker boundary

One consequence of this model deserves recording. Because the engine was built before several of the modules that consume it, its result format was extended as those modules arrived — for example, when Monte Carlo analysis needed per-run statistics and the digital twin needed a per-entity journey trace. Both extensions were inexpensive because the engine stayed isolated behind a worker boundary, which is precisely the benefit the layered architecture was intended to provide.

4. SOFTWARE / HARDWARE MODULES

JustCmul8 is composed of twelve software modules, each owning a distinct responsibility, exposing a defined interface to the others, and developed and tested as a unit before integration. This chapter describes each module — what it does, how it was implemented and which requirements of Section 2.1 it satisfies — together with the individual contribution of each team member. The chapter closes with the hardware considerations that follow from executing simulations on the client.

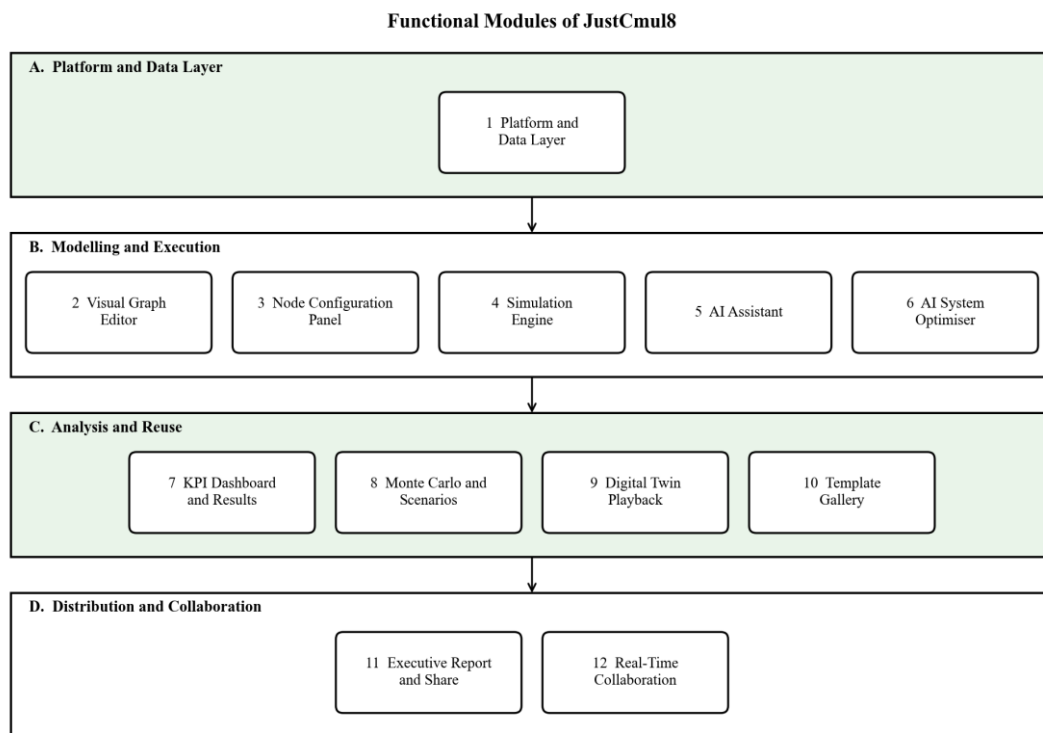


Figure 4.1: Module Structure of JustCmul8

Table 4.1: Module Overview and Requirement Coverage

No.	Module	Primary responsibility	Requirements
1	Platform and Data Layer	Authentication, project lifecycle, cloud persistence, profile	FR-01 to FR-06
2	Visual Graph Editor	Canvas-based construction of the simulation graph	FR-07 to FR-09
3	Node Configuration Panel	Parameter and cost entry with per-type validation	FR-10, FR-11
4	Simulation Engine	Code generation and in-browser SimPy execution	FR-12 to FR-15

No.	Module	Primary responsibility	Requirements
5	AI Assistant	Natural-language generation of candidate models	FR-16, FR-17
6	AI System Optimiser	Bottleneck diagnosis and one-click corrective fixes	FR-18
7	KPI Dashboard and Results	Metric computation, verification, visualisation, cost and ROI	FR-19 to FR-21
8	Monte Carlo and Scenario Comparison	Replicated runs, confidence intervals, baseline comparison	FR-22
9	Digital Twin Animated Playback	Animated 2D entity flow with transport controls	FR-23
10	Simulation Template Gallery	Reusable starter models across six domains	FR-24
11	Executive Report and Share	PDF, CSV, JSON, notebook export and share links	FR-25, FR-26
12	Real-Time Collaboration	Live presence, cursors and graph-operation broadcast	FR-27

4.1 Module 1 — Platform and Data Layer

Purpose. This module establishes who a user is, what they own and where that work is kept. Every other module depends on it, since a project cannot be created, shared or persisted until an identity exists to own it. It consolidates authentication, project management, cloud storage and account settings into a single foundation layer built on Supabase.

Functional responsibilities.

- Registration and login by email and password, and one-click Google or GitHub OAuth sign-in.
- Session protection: authenticated sessions persist across visits and protected routes are guarded by middleware before any project data is fetched.
- Password reset through an emailed link, in-session password update, secure logout and account deletion that cascades through all owned data.
- Creation of a project in one of the six domains, optionally seeded from a template; renaming and deletion of projects.
- Automatic persistence of the graph to cloud storage as the user edits, with no explicit save action.
- A dashboard listing every project with its domain and last-modified time, and a settings page for the profile display name, avatar and default simulation domain.

Implementation. Authentication is delegated to Supabase Auth; the application never handles a plaintext password beyond the submitting form. Sessions are refreshed by Next.js middleware, which also guards protected routes. The decisive design point is that authorisation is not enforced in the client: PostgreSQL Row-Level Security policies attached to every table evaluate the requesting user’s identity at query time, so a modified client cannot read another user’s rows even if it constructs the request by hand. A database trigger creates a profile row whenever a new user registers, and every child table is declared with cascading deletion so that removing an account removes its data.

Requirements covered	Principal source files	Developed by
FR-01 to FR-06	app/login, signup, forgot-password, update-password, auth/callback, dashboard, settings; middleware.ts; lib/supabase; api/account/delete; supabase/schema.sql	Khushi Singh (261713)

4.2 Module 2 — Visual Graph Editor

Purpose. The graph editor is the surface on which modelling happens and the module most responsible for the platform’s no-code claim. It allows a process to be expressed as a directed graph through direct manipulation alone.

Functional responsibilities.

- A drag-and-drop canvas onto which nodes are placed from a palette and freely repositioned.
- Fifteen node types spanning the six simulation domains, each with its own icon, colour and connection handles.
- Creation and deletion of directed edges between node handles, with real-time edge re-routing as nodes move.
- Zoom, pan and fit-to-view navigation, selection and deletion of nodes and edges.
- Structural validation before execution, rejecting graphs that cannot run and naming the offending node.
- Automatic layout of generated and template graphs using a strategy chosen per domain.

Implementation. The canvas is built on React Flow, which supplies the viewport transform, handle-based connection model and edge routing that would otherwise have consumed a disproportionate share of the schedule. Custom node components render each of the fifteen types. Labels are drawn from a domain registry, so a user modelling a bank sees “Teller” where

a user modelling a factory sees “Machine”, although both are the same underlying resource node. Automatic layout uses Dagre with a strategy selected per domain, because a single generic layout produced diagrams that were technically correct but unintuitive.

Requirements covered	Principal source files	Developed by
FR-07 to FR-09	components/workspace/NodeCanvas.tsx, NodePalette.tsx; lib/simulation/layoutEngine.ts, simTypeRegistry.ts	Mohit Gupta (261715)

Table 4.2: The Fifteen Node Types Supported by the Simulation Engine

Node type	Behaviour	Typical domains
source	Generates entities at intervals drawn from a chosen distribution	All six
queue	A waiting line of finite or infinite capacity under FIFO, LIFO or priority discipline	All six
resource	A capacity-limited server that entities seize, hold for a sampled service time, then release	All six
service	A single processing step with its own duration	Manufacturing, logistics
decision	Routes entities down one of several outbound edges by assigned probabilities	All six
sink	Terminates an entity’s journey and records its completion statistics	All six
container	Holds a level of bulk material or liquid, sampled at event boundaries	Liquid / material flow
store	A buffer of distinguishable items (SimPy Store, priority and filter variants)	Manufacturing, logistics
event_trigger	Monitors a condition and fires an event when it is met	Network, manufacturing
priority_resource	A preemptive resource: a higher-priority arrival displaces a lower one	Human queues, healthcare
channel	A transmission medium with propagation delay	Network and signal flow
broadcaster	Duplicates an entity to every outbound edge (one-to-many fan-out)	Network and signal flow
any_of	Waits on several concurrent events and proceeds when the first completes (SimPy AnyOf)	Network, manufacturing
all_of	Waits until every concurrent event has completed (SimPy AllOf)	Manufacturing, logistics

Node type	Behaviour	Typical domains
interrupter	Interrupts an in-progress process, modelling breakdowns and abandonment	Manufacturing, human queues

The first six node types form the classical discrete-event vocabulary and suffice for most models. The remaining nine extend the engine to cases the classical set cannot express: continuous levels, typed buffers, preemption, propagation delay, fan-out and event synchronisation. The two synchronisers correspond directly to SimPy's `AnyOf` and `AllOf` constructs, which is what allows a generated script to express concurrent waiting without custom scheduling code.

Table 4.3: The Six Simulation Domains and Their Vocabularies

Domain	Typical scenarios	Example node labels
Human Queues	Banks, clinics, ticket counters, reception desks	Teller, Counter, Waiting Area, Customer Arrival
Vehicle Traffic	Junctions, toll plazas, fuel stations, car washes	Approach, Signal, Toll Booth, Pump
Liquid / Material Flow	Tanks, pipelines, mixing and bottling plants	Reservoir, Pipe, Valve, Mixer
Manufacturing Lines	Assembly lines, machining cells, inspection bays	Machine, Buffer, Inspection, Assembly
Logistics / Warehousing	Depots, sorting hubs, order fulfilment	Inbound Dock, Picking, Packing, Dispatch
Network / Signal	Packet routing, message buses, sensor networks	Transmitter, Router, Link, Receiver

4.3 Module 3 — Node Configuration Panel

Purpose. Placing a node states what kind of thing it is; configuring it states how it behaves. This module supplies the parameter entry that turns a topology into an executable model, through ordinary form controls rather than a scripting field.

Functional responsibilities.

- A side panel that opens for the selected node and presents only the fields relevant to that node's type.
- Entry of arrival rates, service-time distributions and their parameters, capacities, queue disciplines, routing probabilities and propagation delays.

- Selection among the five supported probability distributions, with the parameter fields changing to match.
- Entry of the cost parameters — resource hourly rate, holding cost and waiting-penalty cost — consumed by the Cost and ROI analysis.
- Validation of every value against the node type, and immediate propagation of each change to the underlying model.

Implementation. Configuration is stored as a JSON object scoped to the node type's schema, which keeps the database schema stable as new node types are introduced while still permitting type-specific validation. Validation runs at the point of entry and again before execution. Messages name the offending node and state the corrective action, because reporting a raw engine exception to a non-technical user would defeat the purpose of a no-code interface.

Requirements covered	Principal source files	Developed by
FR-10, FR-11	components/workspace/NodePropertiesPanel.tsx; lib/simulation/types.ts	Khushi Singh (261713)

4.4 Module 4 — Simulation Engine

Purpose. This module is the technical core of the project. It converts the visual graph into an executable discrete event simulation and runs it entirely within the browser using a genuine Python engine.

Functional responsibilities.

- Compilation of the node-and-edge graph into a Python SimPy script reflecting the configured parameters.
- Execution of that script client-side through the Pyodide WebAssembly runtime, with no simulation computation on the server.
- A Web Worker execution path so that the interface remains responsive for the duration of the run.
- Acceptance of the simulated duration and time unit (seconds, minutes, hours, days) before a run, and streaming of periodic state snapshots and log events to the interface while the run proceeds.
- A live Python code inspector exposing the generated script, with export as a Jupyter notebook.

Implementation. Execution proceeds in four stages. The graph is validated structurally; the code generator walks it and emits a SimPy script in which each node becomes a resource, store or process and each edge becomes a routing decision; the script is passed to the worker, where Pyodide executes it; and the resulting event trace is returned to the main thread for analysis. Resources are created with the matching SimPy primitive — `Resource`, `PriorityResource`, `Store`, `PriorityStore`, `FilterStore` or `Container` — and the generated script is deliberately readable. It is exposed through the code inspector and can be exported as a notebook, so that a student can see the Python their diagram produced and re-run it independently, which turns the platform into a teaching aid for SimPy and makes any published result verifiable.

Requirements covered	Principal source files	Developed by
FR-12 to FR-15	lib/simulation/codeGenerator.ts, pyodideEngine.ts, pyodideWorker.ts, clientEngine.ts, jupyterExport.ts; components/workspace/CodeInspectorPanel.tsx	Mohit Gupta (261715)

4.5 Module 5 — AI Assistant

Purpose. This module lowers the conceptual barrier rather than the technical one. A user who does not know how a process should be represented as a graph can describe it in plain English and receive a candidate model instead.

Functional responsibilities.

- Conversion of a natural-language scenario description into a candidate graph of nodes and edges with plausible parameters.
- Validation of the generated graph — valid node types and no orphaned edges — before it is offered to the user.
- Preview and explicit acceptance, so the live project is never modified without consent.
- Conversational help on simulation concepts and on the user's own model in non-technical language, with chat history kept per project.
- Automatic fall-back to an alternative AI model when the preferred one is unavailable.

Implementation. Requests are proxied through Next.js API routes to Google Gemini, which keeps the API key on the server and out of the client bundle. The system prompt describes the available node types and their parameter schemas together with the current graph; supplying the node registry in the prompt is what constrains the model to emit graphs the engine can

execute. A model router keeps a priority list of models, classifies failures (unavailable, rate-limited, empty response) and temporarily marks an unhealthy model so that subsequent requests skip it. Generated graphs are validated by graph operations before display and arranged by the layout engine.

Requirements covered	Principal source files	Developed by
FR-16, FR-17	components/workspace/AIChatPanel.tsx; app/api/ai/chat and generate; lib/ai/systemPrompt.ts, graphOps.ts, modelRouter.ts, textFormat.ts	Khushi Singh (261713)

4.6 Module 6 — AI System Optimiser

Purpose. Where Module 5 helps a user build a model, this module helps them act on one. After a run it inspects the computed metrics, explains which node is constraining the system and why, and proposes a specific change that can be applied in a single action.

Functional responsibilities.

- Post-run inspection of utilisation, waiting times and flow through each node.
- Diagnosis of the constraining node, expressed in terms a non-specialist can act on.
- Proposal of a concrete corrective change — more capacity at a resource, rerouting at a decision node, a larger buffer.
- A one-click Apply Fix action, applied as an ordinary graph edit.
- Retention of the pre-change graph so the effect can be compared in Module 8.

Implementation. The optimiser is grounded in analytics output rather than in the raw event trace, so its recommendations refer to computed utilisation and queue statistics instead of impressions. A flow-analysis step first determines which node it is structurally sensible to change — raising the capacity of a node that is not the constraint achieves nothing — and only the resulting candidate is put to the language model for explanation. The proposal is always presented for approval: the optimiser advises but never acts unilaterally.

Requirements covered	Principal source files	Developed by
FR-18	app/api/ai/optimize/route.ts; lib/ai/flowAnalysis.ts, graphOps.ts	Mohit Gupta (261715)

4.7 Module 7 — KPI Dashboard and Results

Purpose. An event trace is not an answer. This module converts the raw output of a run into the performance indicators on which a decision rests, and presents them so that the constraining node is evident without further analysis.

Functional responsibilities.

- Computation and display of utilisation, average and percentile waiting times, throughput and queue length for each node.
- Automatic identification and prominent flagging of the bottleneck node.
- A Flow and Balance Check that applies Little’s Law to confirm the run is consistent, and a composite 0–100 system health score.
- Interactive tabs: Executive Summary, Flow and Balance Check, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person / Item Journey, Live Activity Log and Cost and ROI.
- A Cost and ROI panel converting the run into monetary terms, so a proposed change can be judged on whether it pays for itself.

Implementation. Analytics are computed in TypeScript modules with no React dependency, which makes them unit-testable in isolation. The system health score is a weighted composite: completion rate (35%), utilisation balance (30%), queueing stability judged by Little’s Law (20%) and tail latency (15%). The bottleneck is the resource or queue node with the highest average waiting time in the run. Percentiles are computed from the real per-entity samples rather than estimated, and a conservation check confirms that work-in-progress equals arrivals minus completions.

Requirements covered	Principal source files	Developed by
FR-19 to FR-21	lib/simulation/analyticsEngine.ts, resolveKpiMetrics.ts; components/workspace/SimResultsPanel.tsx, AdvancedResultsDashboard.tsx, results-dashboard/*	Khushi Singh (261713)

Table 4.4: Key Performance Indicators and Their Computation

Metric	Definition	Input data
Utilisation	Fraction of run time for which a resource was occupied	Accumulated busy-seconds per resource
Average waiting time	Mean interval between joining a queue and being served	Queue entry and service-start times per entity

Metric	Definition	Input data
Percentile waiting time	50th, 90th, 95th and 99th percentiles of the waiting distribution	Real per-entity waiting samples
Throughput	Entities completing the model per unit of simulated time	Sink completion counts and run duration
Average queue length	Time-weighted mean number of entities waiting	Periodic depth snapshots
Little's Law check	Compares L with λW and reports the discrepancy and stability verdict	Arrivals, completions, work-in-progress, mean time in system
System health score	0–100 composite: completion 35, utilisation balance 30, stability 20, tail latency 15	Outputs of the above
Total system cost	Resource, holding and waiting-penalty costs over the run	Cost parameters from Module 3

4.8 Module 8 — Monte Carlo and Scenario Comparison

Purpose. A single stochastic run is one observation, not an estimate. This module repeats the run and reports the range within which the true behaviour lies, then allows two configurations to be compared against that measure of uncertainty.

Functional responsibilities.

- Execution of a model over 10, 20, 50 or 100 replications, each an independent run.
- Computation of the mean, standard deviation and 95% confidence interval for every metric across replications.
- Rendering of intervals as bands so that significance is visible rather than inferred.
- Side-by-side comparison of a baseline scenario against a modified one, with the difference reported per metric.
- Progress reporting during a long analysis, so the user is never left without feedback.

Implementation. The runner executes replications through the same engine as a single run, aggregates the results with the standard-error formula of Section 1.1.2, and exposes a scenario-comparison function that pairs the baseline retained before an applied fix against the current configuration. Because the aggregation is pure TypeScript it is simple to test and independent of the interface.

Requirements covered	Principal source files	Developed by
FR-22	lib/simulation/monteCarlo.ts, monteCarloRunner.ts; components/workspace/MonteCarloPanel.tsx	Mohit Gupta (261715)

4.9 Module 9 — Digital Twin Animated Playback

Purpose. Numbers describe congestion; animation shows it. This module replays a completed run as a moving two-dimensional picture of entities passing through the model, which communicates where a system is failing far more immediately than a chart.

Functional responsibilities.

- Animated playback of entities flowing through the nodes and edges of the model.
- Highlighting of queue congestion so that accumulating lines are visible as they build.
- Transport controls in the manner of a video player: play, pause, step and speed selection from $0.5\times$ to $10\times$.
- Scrubbing to an arbitrary point in the run without re-executing the model.

Implementation. Playback is rendered with Pixi.js over WebGL rather than with DOM elements, because animating several hundred moving entities through the DOM degrades unacceptably. The animation is driven from the recorded entity journeys rather than by re-executing the model, so scrubbing and speed changes are immediate and what is shown corresponds exactly to the run whose metrics are on screen.

Requirements covered	Principal source files	Developed by
FR-23	components/workspace/DigitalTwinCanvas.tsx, PlaybackControls.tsx; lib/simulation/timelineSelectors.ts	Khushi Singh (261713)

4.10 Module 10 — Simulation Template Gallery

Purpose. An empty canvas is an obstacle to a first-time user. This module supplies working models across all six domains so that a user can begin from something that already runs and modify it — a considerably gentler introduction than construction from nothing.

Functional responsibilities.

- A gallery of starter graphs grouped by domain, each with a description.
- Instantiation of a template, pre-populating a project with its nodes, edges and default parameters.

- Creation of a fully independent copy, so edits can never propagate back to the source template.
- Free modification of all parameters after instantiation.

Implementation. Templates are declared in the simulation type registry alongside the node palette and layout strategy for each domain, keeping a domain’s definition in one place, so adding a domain does not require changes scattered across components. Instantiation copies the template’s nodes and edges into the new project rather than referencing them, which satisfies the independence requirement by construction.

Requirements covered	Principal source files	Developed by
FR-24	components/workspace/TemplateGallery.tsx; lib/simulation/simTypeRegistry.ts	Mohit Gupta (261715)

4.11 Module 11 — Executive Report and Share

Purpose. Work that cannot leave the platform has limited value. This module moves models and results outward, whether for archival, for analysis elsewhere or for presentation to someone who will never log in.

Functional responsibilities.

- One-click generation of a PDF executive report containing a written summary, a canvas snapshot and KPI tables.
- Export of the project graph as JSON and of run results as CSV.
- Export of the generated Python script as a Jupyter notebook for independent reproduction.
- Generation of a read-only public share link that never confers edit access.
- Revocation of a share link, after which the URL displays an explanatory notice.

Implementation. PDF generation runs client-side using jsPDF with html-to-image capturing the canvas, so no document-rendering service is required and the architecture’s zero-server-compute property is preserved. Share links are opaque random tokens. Public access goes only through two narrow database functions — one returns a single shared project’s name, domain and graph, the other reports whether a token is active, revoked or invalid — so anonymous visitors are never granted direct table access. The read-only guarantee is therefore enforced by the database rather than by hiding controls in the interface.

Requirements covered	Principal source files	Developed by
FR-25, FR-26	lib/simulation/pdfReportGenerator.ts, reportGenerator.ts, jupyterExport.ts; components/workspace/ShareExportModal.tsx, ShareViewer.tsx, ShareLinkUnavailable.tsx; app/api/projects/[id]/share and export; app/share/[token]	Khushi Singh (261713)

4.12 Module 12 — Real-Time Collaboration

Purpose. This module provides the multi-user behaviour that teams now expect of any modern design tool: seeing who else is in the workspace and what they are doing, in the manner of Canva or Figma.

Functional responsibilities.

- Live presence: an avatar row showing who is currently in the project.
- Live cursors: each collaborator's pointer drawn on the canvas in their own colour, labelled with their name.
- Broadcast of graph operations — node and edge changes and node-data edits — to the other connected clients.
- Awareness of selection and editing: which node a collaborator has selected or is editing.
- Access to the project channel authorised at the database, so that only entitled users can join.

Implementation. Collaboration is built on Supabase Realtime channels over WebSockets. Each project has a private channel named after its identifier; before joining, the client confirms that it can read the project through Row-Level Security, and a realtime policy independently authorises the channel at the database. Presence uses the channel's presence state, while cursor positions and graph operations are sent as broadcast messages. Cursor positions are ephemeral and never persisted, since they have no meaning after a session ends.

Requirements covered	Principal source files	Developed by
FR-27	lib/realtime/usePresence.ts; components/workspace/LiveCursor.tsx	Mohit Gupta (261715)

4.13 Hardware Modules and Considerations

JustCmul8 introduces no custom hardware, but the decision to execute simulations on the client rather than the server gives the user's own machine a role it would not otherwise have, and that role is worth stating explicitly.

Table 4.5: Hardware Roles in the Deployed System

Element	Role in the system	Consequence of the design
Client CPU	Executes the WebAssembly runtime and the entire simulation	Determines run duration; large models are slower on weaker machines
Client memory	Holds the Pyodide runtime, the graph, the event trace and the rendered canvas	Sets the practical ceiling on model size and replication count
Client GPU	Renders the digital-twin playback through WebGL	Enables smooth animation of many simultaneous entities
Network link	Carries persistence, collaboration and AI requests only	Simulation continues to run on an intermittent connection
Application server	Serves the application and a small number of API routes	Carries no simulation load, so hosting cost is independent of usage intensity
Database server	Managed PostgreSQL storing projects, runs, chats and shares	Sized by data volume rather than by computational demand

The last two rows express the economic argument of Section 2.2.2 in hardware terms. Because neither server participates in the computation, the infrastructure required to support a thousand users running simulations is essentially that required to serve a thousand users a static page, which is what makes the platform sustainable without a licence fee.

4.14 Individual Work Justification

The project was carried out by a team of two. Requirements analysis, architecture, database design, integration, testing and this report were joint activities. The twelve modules were divided equally, six to each member, with each member owning the design, implementation, unit testing and documentation of their modules and reviewing the modules of the other. The division is summarised below.

Table 4.6: Division of Work between Team Members

Module	Owner	Justification of contribution
1 Platform and Data Layer	Khushi Singh (261713)	Authentication flows, session middleware, dashboard, settings, database schema and Row-Level Security policies

Module	Owner	Justification of contribution
2 Visual Graph Editor	Mohit Gupta (261715)	React Flow canvas, custom node components, palette, layout engine and structural validation
3 Node Configuration Panel	Khushi Singh (261713)	Type-specific forms, distribution controls, cost parameters and value validation
4 Simulation Engine	Mohit Gupta (261715)	SimPy code generator, Pyodide worker, execution pipeline and the verification harness
5 AI Assistant	Khushi Singh (261713)	Chat interface, prompt design, graph generation, model router and history
6 AI System Optimiser	Mohit Gupta (261715)	Flow analysis, bottleneck diagnosis, one-click fix and cost-aware recommendation
7 KPI Dashboard and Results	Khushi Singh (261713)	Analytics engine, Little's Law check, health score, dashboard tabs and Cost and ROI
8 Monte Carlo and Scenario Comparison	Mohit Gupta (261715)	Replication runner, confidence intervals and scenario comparison
9 Digital Twin Playback	Khushi Singh (261713)	Pixi.js renderer, entity journeys, transport controls and timeline
10 Template Gallery	Mohit Gupta (261715)	Domain registry, six-domain template library and instantiation
11 Executive Report and Share	Khushi Singh (261713)	PDF report generation, JSON/CSV/notebook export, share links and read-only viewer
12 Real-Time Collaboration	Mohit Gupta (261715)	Presence channel, live cursors and graph-operation broadcast

Both members contributed to the specification, the design diagrams, the test cases and the report, and each acted as reviewer for the other's modules, which is what allowed the work to be integrated with few defects.

5. SYSTEM ANALYSIS AND DESIGN

This chapter presents the analysis and design of JustCmul8. It works from the outermost view of the system inwards: the context diagram establishes the boundary, the data flow diagrams decompose the processes within it, the entity relationship and class diagrams describe the data and the objects, and the behavioural diagrams describe how those objects interact at run time. The chapter closes with the physical concerns — the database tables and the deployment topology.

5.1 Context Diagram

The context diagram (data flow diagram level 0) fixes the boundary of the system. It represents JustCmul8 as a single process and shows the external entities that exchange data with it, without decomposing anything inside. Its purpose is to settle what belongs to the system and what does not before any internal structure is discussed.

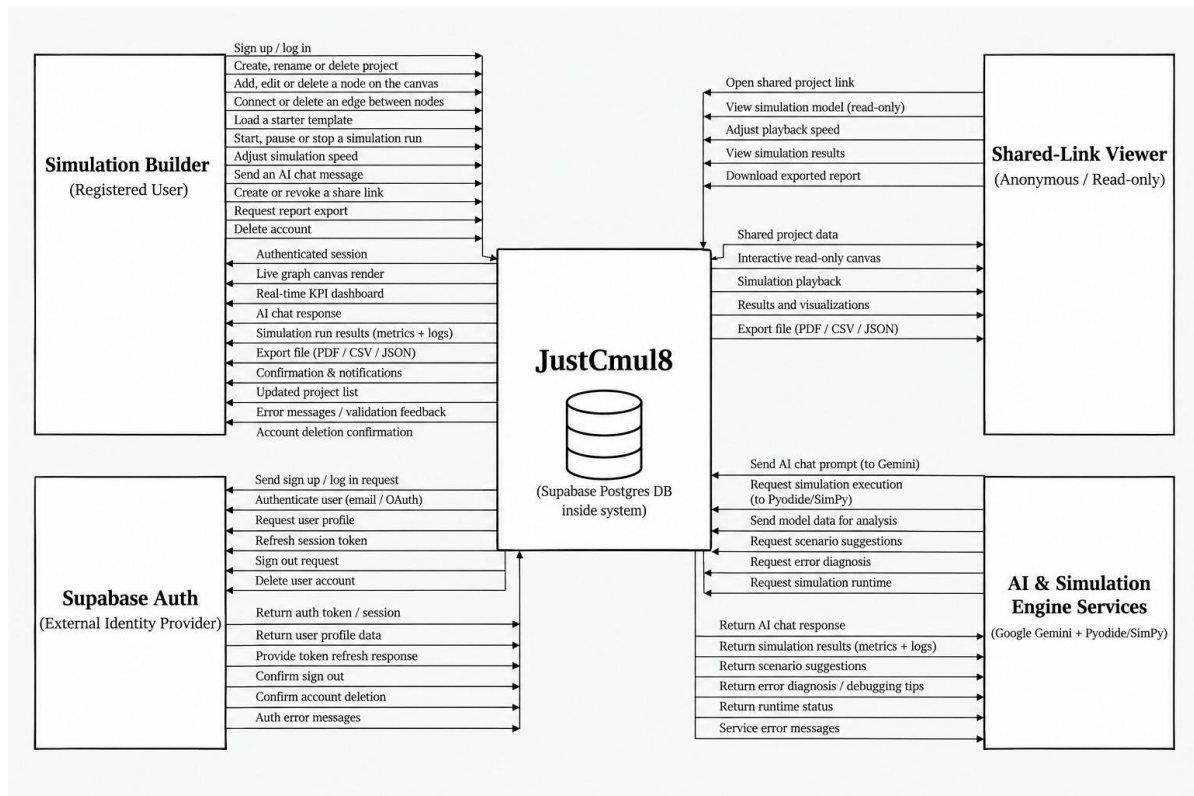


Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8

Four external entities interact with the system. The **Simulation Builder** is the registered user who creates and runs models; the inbound flows from this entity cover signing up and logging in, creating, renaming and deleting projects, adding, editing and deleting nodes and edges,

loading starter templates, starting and stopping runs, adjusting simulation speed, sending AI chat messages, creating and revoking share links, requesting report exports and deleting the account. The return flows carry the authenticated session, the live canvas rendering, the real-time KPI dashboard, AI responses, run results with metrics and logs, export files, confirmations, the updated project list and validation feedback.

The **Shared-Link Viewer** is an anonymous visitor holding a public share URL. This entity opens a shared project, views the model read-only, adjusts playback speed, views results and downloads an exported report. The asymmetry between the two user entities is deliberate: no flow from the Shared-Link Viewer modifies stored state.

Supabase Auth is the external identity provider. The system sends it sign-up and login requests, profile queries, session refreshes, sign-out requests and account-deletion instructions, and receives tokens, sessions, profile data and confirmations. Placing authentication outside the boundary records that JustCmul8 delegates credential handling rather than implementing it. The **AI and Simulation Engine Services** entity covers the Google Gemini API and the Pyodide and SimPy runtime delivered over a CDN. The PostgreSQL database is drawn inside the system boundary, since it is provisioned and governed by the application rather than consumed as an independent third party.

5.2 DFD, ERD, Class Diagram and State Transition Diagram

5.2.1 Data Flow Diagram — Level 1

The level-1 diagram decomposes the single process of the context diagram into the seven functional processes that constitute the system and introduces the data stores they read from and write to.

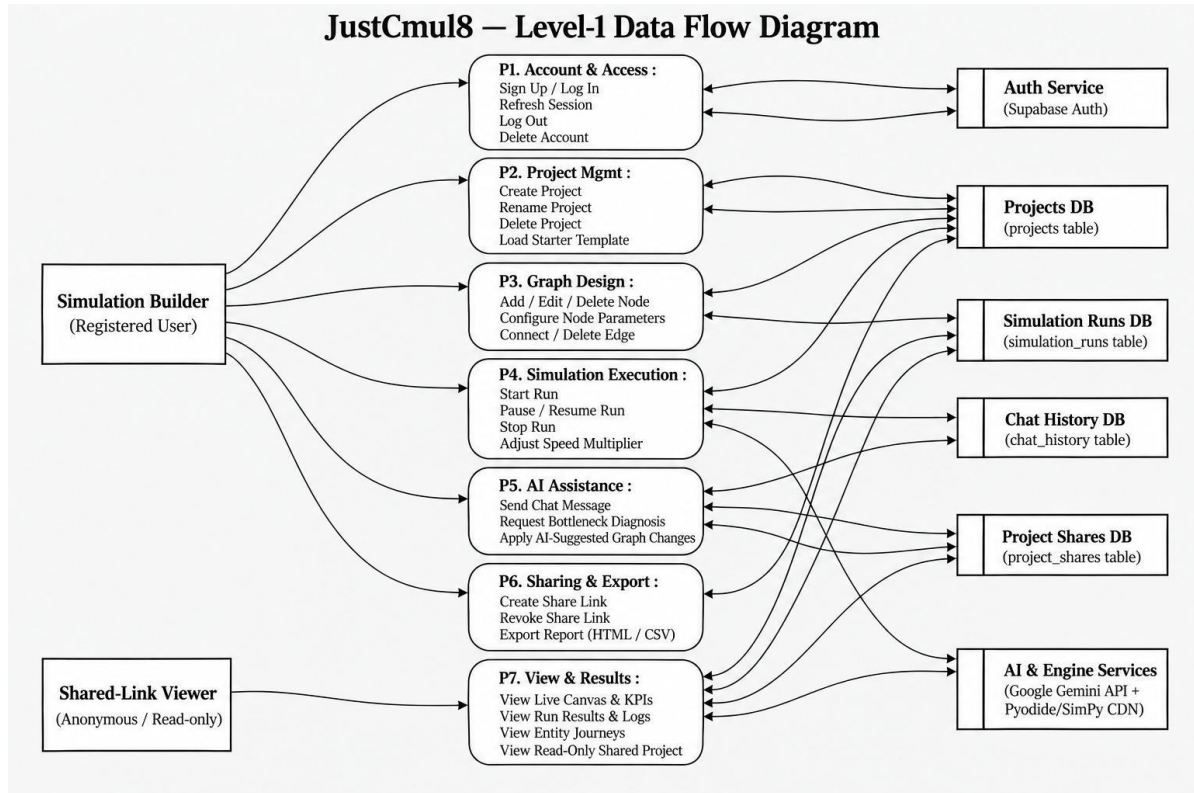


Figure 5.2: Level-1 Data Flow Diagram for JustCmul8

Process **P1, Account and Access**, handles sign-up, log-in, session refresh, log-out and account deletion, exchanging data with the external authentication service. **P2, Project Management**, covers project creation, renaming and deletion and the loading of starter templates, reading from and writing to the Projects store. **P3, Graph Design**, adds, edits and deletes nodes, configures node parameters and connects or removes edges, persisting each change to the same store.

P4, Simulation Execution, starts and stops runs and adjusts the speed multiplier; it reads the graph from the Projects store, drives the external engine services and writes run records to the Simulation Runs store. **P5, AI Assistance**, sends chat messages, requests bottleneck diagnosis and applies AI-suggested graph changes, exchanging data with the Gemini service and recording exchanges in the Chat History store. **P6, Sharing and Export**, creates and revokes share links and produces exports, writing to the Project Shares store. **P7, View and Results**, serves the live canvas and KPIs, run results and logs, entity journeys and the read-only shared project view; it is the only process reachable by the anonymous Shared-Link Viewer, which again enforces the read-only guarantee structurally rather than by convention.

5.2.2 Data Flow Diagram — Level 2

Process P4 is the most intricate in the system and is decomposed further at level 2. This is the path that converts a drawing into numbers, and it is where the architectural commitment described in Chapter 4 becomes visible as data flow.

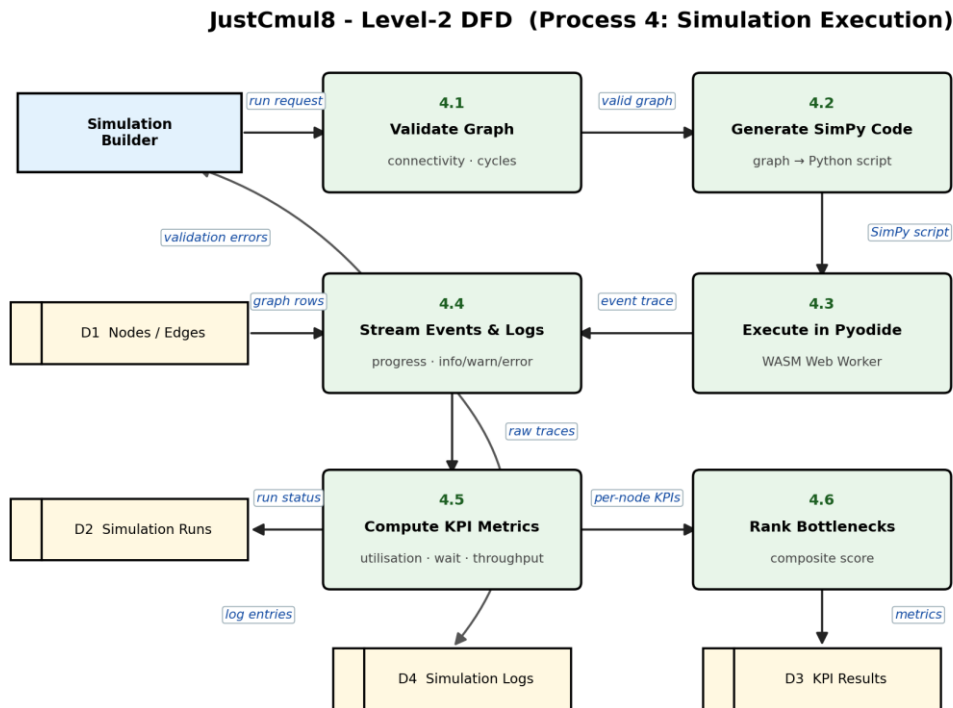


Figure 5.3: Level-2 Data Flow Diagram — Decomposition of Process P4 (Simulation Execution)

A run request from the Simulation Builder enters process 4.1, **Validate Graph**, which reads the node and edge data from the Projects store and checks connectivity. If validation fails, errors flow back to the user and the run does not proceed, which prevents an invalid model from reaching the engine at all. A valid graph passes to process 4.2, **Generate SimPy Code**, whose output flows to process 4.3, **Execute in Pyodide**, running inside a Web Worker. The event trace produced there flows to process 4.4, **Stream Events and Logs**, which forwards progress and log entries to the interface during the run.

Raw results then flow to process 4.5, **Compute KPI Metrics**, which derives utilisation, waiting times and throughput and updates the run record. The per-node metrics pass to process 4.6, **Rank Bottlenecks**. The ordering matters: bottleneck ranking depends on computed metrics rather than on the raw trace, which is why 4.6 follows 4.5, and it is the same dependency that grounds the optimiser of Module 6.

5.2.3 Entity Relationship Diagram

The entity relationship diagram describes how persistent data is organised and how the entities relate to one another. It is drawn in Chen notation, with rectangles for entities, ellipses for attributes and diamonds for relationships, annotated with cardinality.

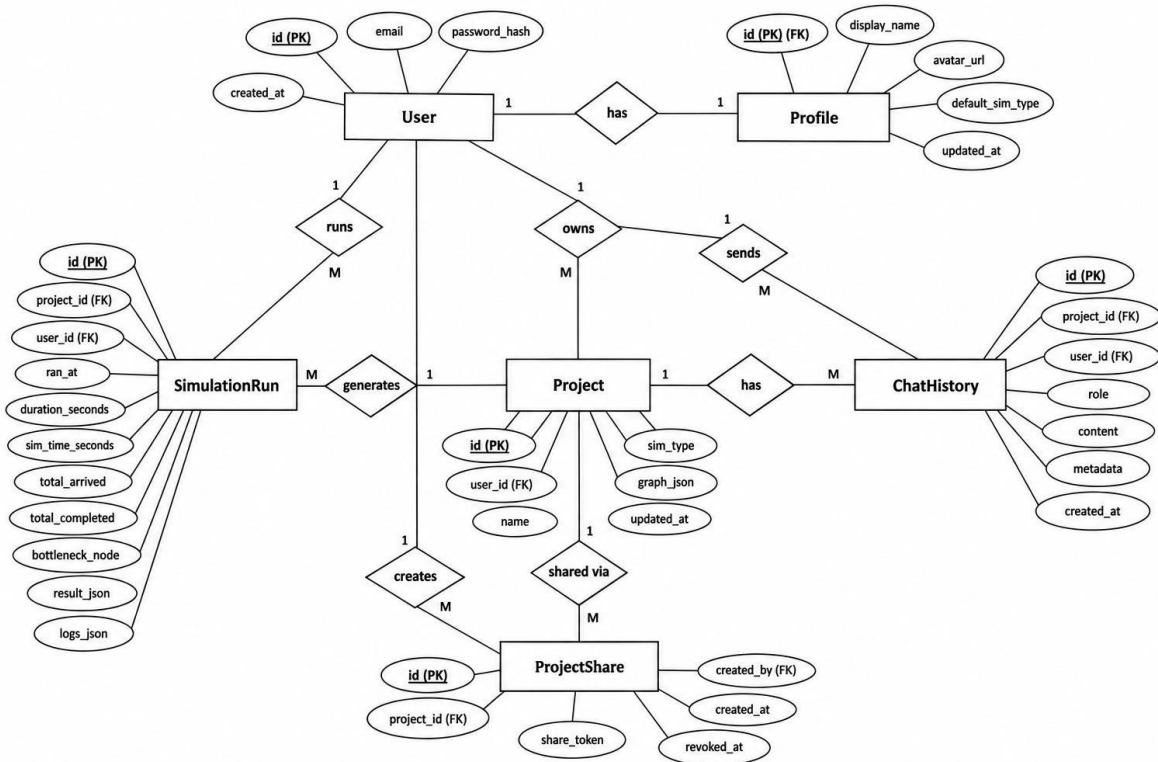


Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database

The **User** entity is central and stands in a one-to-one relationship with **Profile**, which holds the display name, avatar URL, default simulation type and update timestamp; separating the profile from the credential record keeps presentation data apart from authentication data. A User owns many **Projects**, each carrying its identifier, owner, name, simulation type, the graph (stored as a JSON document) and an update timestamp. A User also runs many **SimulationRuns**, sends many **ChatHistory** messages and creates many **ProjectShares**; each Project generates many runs, has many chat entries and is shared through many share records.

The **SimulationRun** entity holds the run timestamp, wall-clock duration, simulated duration, totals arrived and completed, the identified bottleneck node and the result and log payloads. Recording the bottleneck as an attribute of the run rather than recomputing it on demand means a historical run retains the diagnosis that was made at the time. **ProjectShare** holds an opaque share token and creation and revocation timestamps; retaining a revoked share as a row with a revocation timestamp, rather than deleting it, is what allows the system to tell a link that was

withdrawn from one that never existed and to show the viewer an explanatory notice instead of a bare error.

5.2.4 Class Diagram

The class diagram presents the object model implemented in TypeScript, showing each class with its typed attributes, operations and associations.

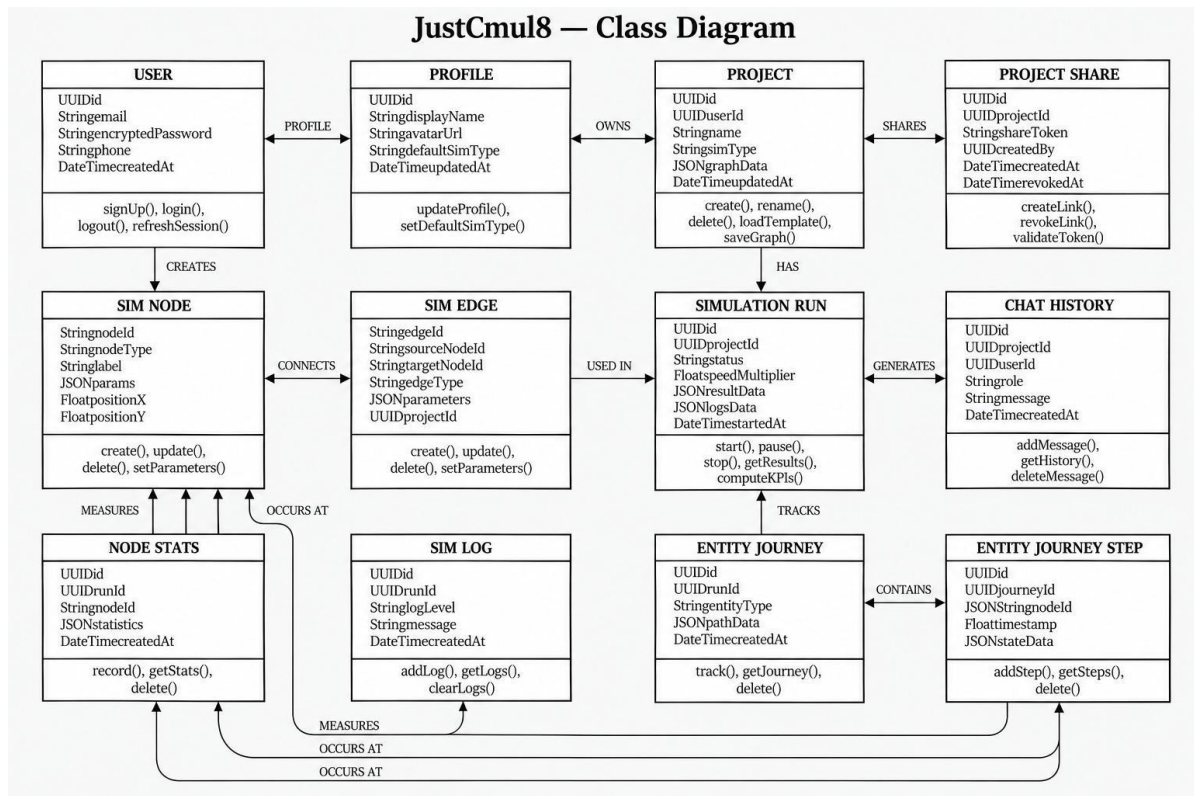


Figure 5.5: Class Diagram of the JustCmul8 Object Model

The upper row carries the account and project classes: **User**, **Profile**, **Project** and **ProjectShare**, with the associations “User has a Profile”, “User owns Projects” and “Project is shared through ProjectShare”. The middle row carries the graph and execution classes: **SimNode** holds the node identifier, type, label, a JSON parameter object and canvas coordinates; **SimEdge** joins two nodes; **SimulationRun** holds status, speed multiplier, result and log payloads; **ChatHistory** records each AI exchange. Storing parameters as JSON rather than as fixed columns is what allows fifteen node types with dissimilar parameters to share one class without a sparse schema.

The lower row carries the measurement classes: **NodeStats**, **SimLog**, **EntityJourney** and **EntityJourneyStep**. The journey and step classes exist to support the digital-twin playback:

because each step carries a timestamp and a node reference, the animation can be reconstructed and scrubbed without re-executing the simulation.

5.2.5 State Transition Diagram

The state transition diagram describes the lifecycle of a node during execution. Every node in a running model occupies exactly one of these states at any instant, and the transitions between them are what the event scheduler effects.

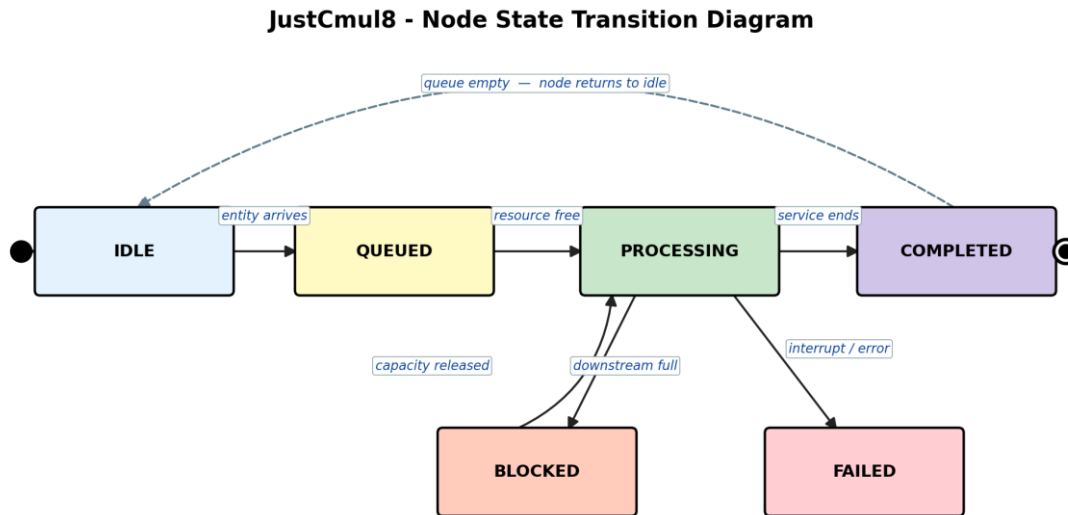


Figure 5.6: State Transition Diagram for a Node During Simulation Execution

A node begins in **IDLE**, having no entity to process. The arrival of an entity moves it to **QUEUED**, where the entity waits for capacity. When the associated resource becomes free the node enters **PROCESSING**, holding the entity for a service time sampled from the configured distribution. On completion the node moves to **COMPLETED** and the entity is routed onward; if no further entities are waiting, the node returns to **IDLE**.

Two exceptional transitions leave **PROCESSING**. If the downstream node has no capacity to accept the entity, the node moves to **BLOCKED** and remains there until capacity is released. Blocking is the mechanism by which congestion propagates backwards through a model, and representing it as a distinct state is what allows the analytics layer to distinguish a node that is slow from one that is merely obstructed by the node after it. The second exceptional transition leads to **FAILED**, entered when a process is interrupted — by an interrupter node, by preemption at a priority resource or by a runtime error. The state model maps directly onto SimPy semantics: **QUEUED** corresponds to an outstanding resource request, **PROCESSING**

to a timeout while the resource is held, BLOCKED to a pending put on a downstream store, and FAILED to a raised `Interrupt`.

5.3 Use Case Diagram

The use case diagram identifies the actors and the services the system offers to each. It expresses functionality from the outside, in terms of what an actor can accomplish, rather than from the inside in terms of how it is achieved.

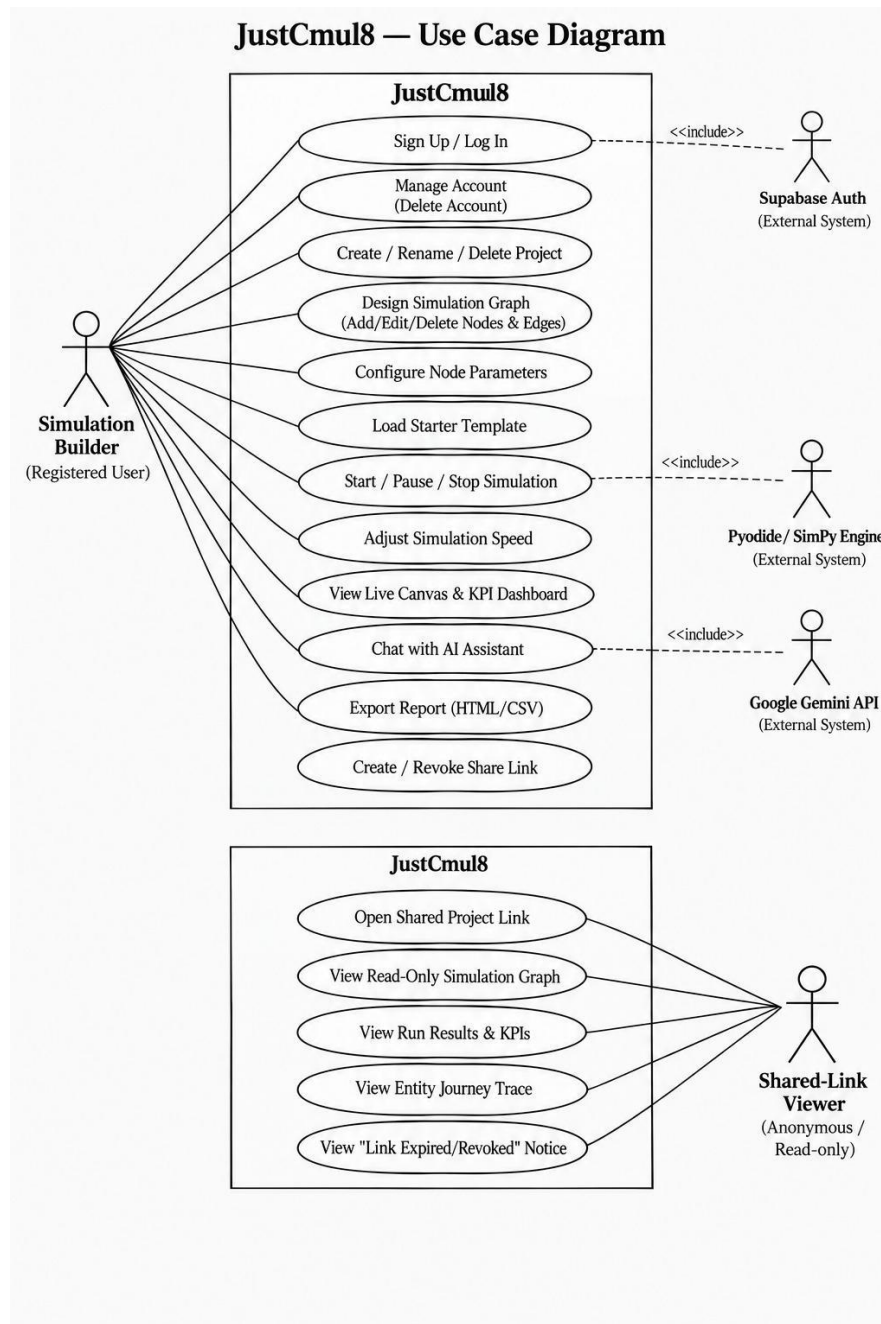


Figure 5.7: Use Case Diagram for JustCmul8

Two human actors appear. The **Simulation Builder**, a registered user, may sign up or log in, manage the account including deletion, create, rename and delete projects, design the simulation graph, configure node parameters, load a starter template, start and stop a simulation, adjust the speed, view the live canvas and KPI dashboard, chat with the AI assistant, export a report and create or revoke a share link. The **Shared-Link Viewer** is an anonymous actor whose use cases are drawn in a separate boundary: opening a shared project link, viewing the read-only graph, viewing results and KPIs, viewing the entity journey trace and viewing the notice displayed when a link has expired or been revoked. That last use case is easy to overlook and produces a poor experience when omitted, which is why it was modelled explicitly rather than treated as an error path.

Three external systems participate through include relationships: Supabase Auth is included by sign-up and log-in; the Pyodide and SimPy engine by starting and stopping a simulation; and the Google Gemini API by chatting with the assistant. The include stereotype is correct rather than extend because in each case the external interaction is mandatory for the use case to complete.

5.3.1 Use Case Specifications

Table 5.1: Use Case Specification — Run Simulation

Field	Description
Use case ID	UC-01
Use case name	Run Simulation
Primary actor	Simulation Builder
Secondary actor	Pyodide / SimPy engine
Precondition	The user is authenticated, a project is open and the graph contains at least one source and one sink
Main flow	1. The user sets the duration and time unit. 2. The user starts the run. 3. The system validates the graph structurally. 4. The system generates the SimPy script. 5. The worker executes the script through Pyodide. 6. Progress and log events stream to the interface. 7. The system computes KPIs and ranks the bottleneck. 8. The dashboard renders the results and the run is saved.
Alternative flow	3a. Validation fails: the system names the disconnected node, explains the fault and aborts before execution. 5a. The runtime raises an error: the error is shown with the relevant log entries. 5b. The user stops the run: execution halts.
Postcondition	On success a run record with KPI results and logs is persisted

Field	Description
Requirements	FR-12, FR-13, FR-14, FR-15, FR-19, FR-20

Table 5.2: Use Case Specification — Generate Model via AI

Field	Description
Use case ID	UC-02
Use case name	Generate Simulation Model from Natural Language
Primary actor	Simulation Builder
Secondary actor	Google Gemini API
Precondition	The user is authenticated and a project is open
Main flow	1. The user describes the scenario in plain English. 2. The system composes a prompt from the description, the node registry and the current graph. 3. The request is proxied through the server to Gemini. 4. The response is parsed into candidate nodes and edges. 5. The candidate graph is validated. 6. The graph is laid out and shown as a preview. 7. The user accepts and the graph is written to the project.
Alternative flow	5a. Validation fails on an unknown node type or orphaned edge: the candidate is rejected and the user is invited to rephrase. 7a. The user discards the preview: the project is unchanged. 3a. The preferred model is unavailable: the router falls back to the next model; if all fail, an explanatory message is shown and manual construction remains available.
Postcondition	Either the project contains the accepted graph or it is entirely unmodified
Requirements	FR-16, FR-17

Table 5.3: Use Case Specification — Share Project Read-Only

Field	Description
Use case ID	UC-03
Use case name	Share Project through a Public Link
Primary actor	Project Owner
Secondary actor	Shared-Link Viewer (anonymous)
Precondition	The user owns the project
Main flow	1. The owner requests a share link. 2. The system generates an opaque token and stores it against the project. 3. The link is presented for copying. 4. A viewer opens the link without authenticating. 5. The system validates the token and renders the project read-only.

Field	Description
Alternative flow	5a. The token has been revoked: the viewer sees an explanatory notice. 5b. The token is unrecognised: access is refused. 2a. The owner later revokes the link: the revocation timestamp is set and later requests fall to 5a.
Postcondition	The viewer has read access to that project alone, with no ability to modify anything
Requirements	FR-26

Table 5.4: Use Case Specification — Collaborate in Real Time

Field	Description
Use case ID	UC-04
Use case name	Live Collaboration in a Project
Primary actor	Collaborator
Secondary actor	Supabase Realtime, other collaborators
Precondition	Two or more authenticated users with access have the same project open
Main flow	1. Each client verifies its access to the project and joins the project’s private channel. 2. Presence state and cursor positions are broadcast continuously. 3. A user modifies a node or edge. 4. The change is broadcast as a graph operation. 5. Every other client applies the operation and re-renders.
Alternative flow	1a. The user has no access to the project: the channel is not joined. 5a. A client disconnects: its presence indicator is removed.
Postcondition	Connected clients display the same graph and can see one another’s presence
Requirements	FR-27

5.4 Architecture Design

JustCmul8 follows a layered architecture in which each layer depends only on the layer beneath it. The unusual feature of the design is the execution layer: in a conventional web application the computational work sits on the server, whereas here it sits on the client, and almost every consequence discussed in this report follows from that inversion.

JustCmul8 — Multi-Tier Architecture Design

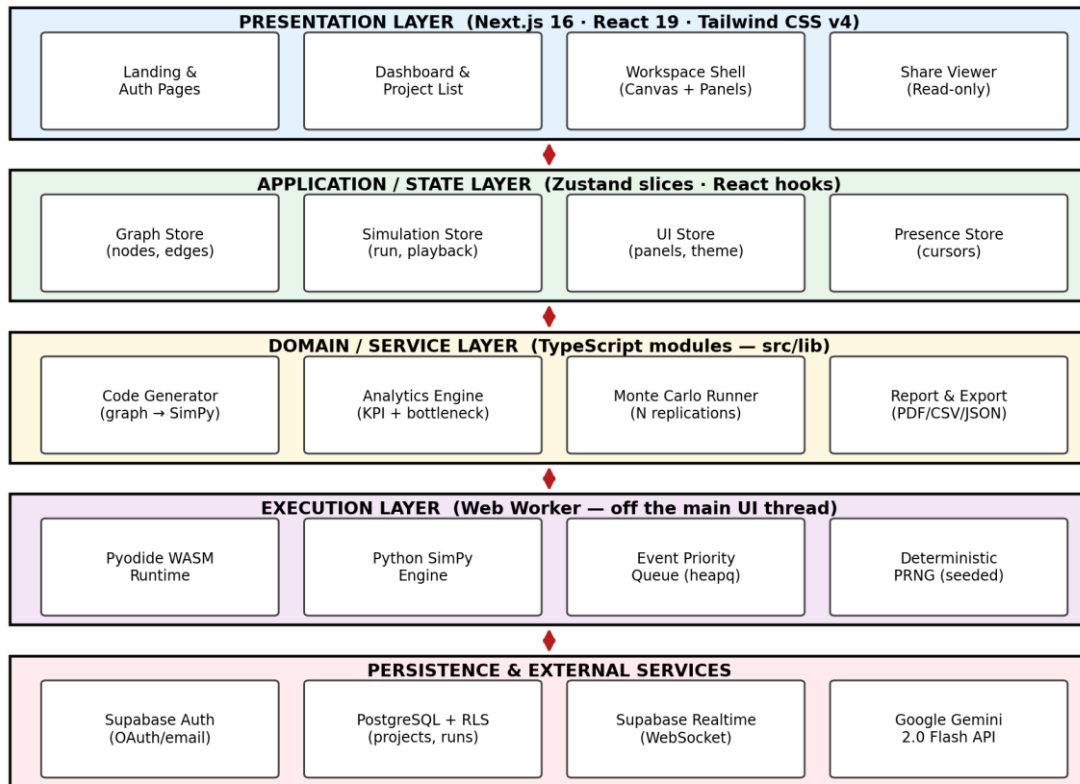


Figure 5.8: Multi-Tier Architecture of JustCmul8

The **presentation layer**, built with Next.js 16, React 19 and Tailwind CSS, contains the landing and authentication pages, the dashboard, the workspace shell that hosts the canvas and panels and the read-only share viewer. It holds no business logic; it renders state and dispatches intentions.

The **application and state layer** holds the Zustand slices — the graph store, the simulation store, the interface store and the presence store. Separating these into slices means a change to playback position does not cause components subscribed only to the graph to re-render, which matters when the canvas carries hundreds of nodes.

The **domain and service layer** contains the TypeScript modules that implement the system’s actual logic: the code generator that translates a graph into SimPy, the analytics engine that computes KPIs, the Monte Carlo runner and the report and export modules. These are plain modules with no React dependency, which is what makes them unit-testable in isolation.

The **execution layer** runs in a Web Worker, off the main thread. It contains the Pyodide WebAssembly runtime and the SimPy engine, whose event priority queue orders scheduled

events. Its isolation is the mechanism by which the requirement that the interface never freeze is met.

The **persistence and external services layer** comprises Supabase Auth, PostgreSQL with Row-Level Security, Supabase Realtime and the Gemini API. Arranging authorisation at this lowest layer, rather than in the presentation layer where it is most visible, is what makes it unbypassable: a request that reaches the database carries the user's identity and is filtered there regardless of what the client believes it is entitled to.

5.5 Activity, Component and Collaboration Diagrams

5.5.1 Activity Diagram — End-to-End User Workflow

Chapter 3 presented an activity network of the development lifecycle. The diagram below models a different flow: the path a user takes through the running system, from authentication to export. It is divided into swimlanes for the Simulation Builder, the system, Supabase Auth, the Pyodide and SimPy engine and the Gemini API, so that responsibility for each activity is unambiguous.

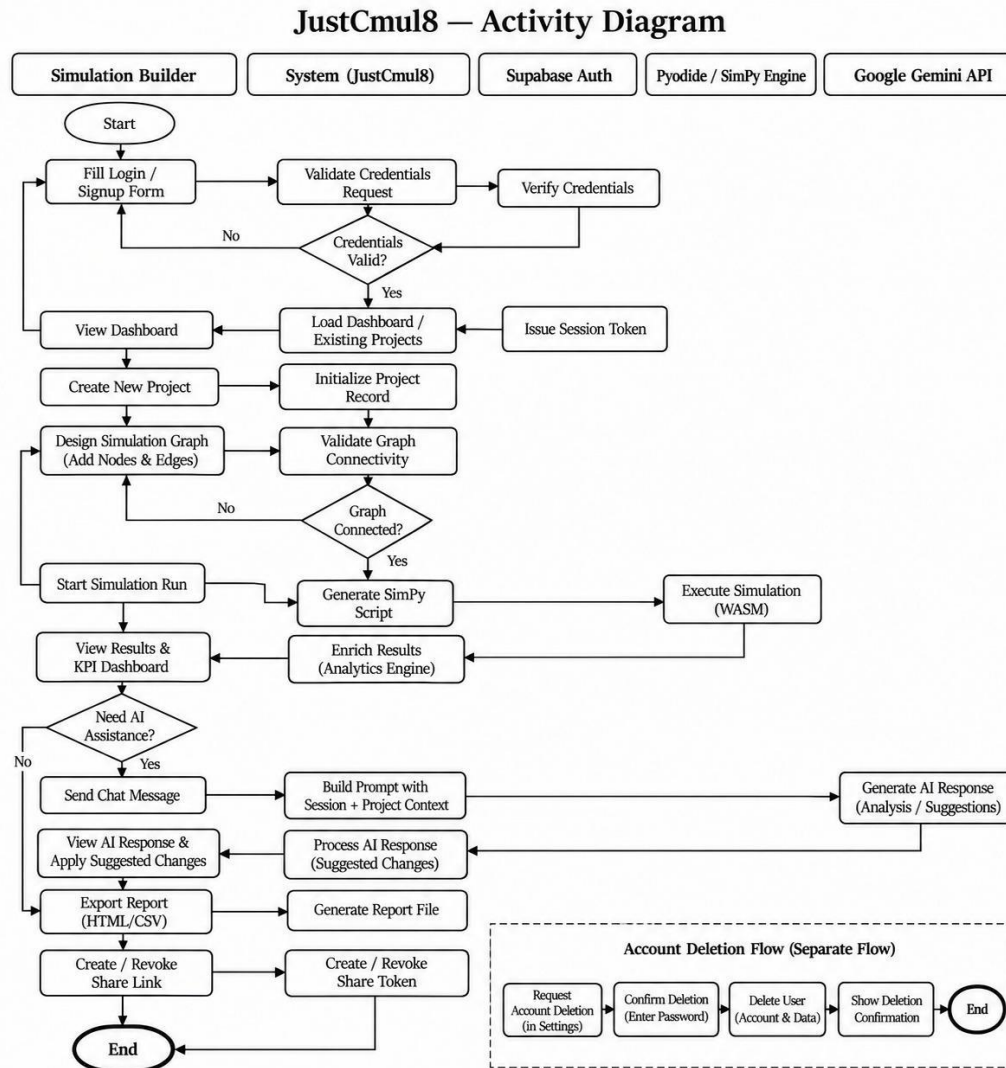


Figure 5.9: Activity Diagram of the End-to-End User Workflow

The flow opens with the user completing the login or sign-up form. The system asks Supabase Auth to verify the credentials; a decision node tests whether they are valid, and if not control returns to the form. On success a session is issued and the dashboard loads with the user's projects. The user creates a project and designs the graph, and the system validates connectivity; a second decision node returns control to the design activity if the graph is not connected, which is the loop that prevents an unrunnable model from reaching the engine.

Once the graph is valid the user starts the run. The system generates the SimPy script and hands it to the engine lane, where it executes as WebAssembly. Results return to the system lane, where the analytics engine enriches them, and the user views the KPI dashboard. A third decision node asks whether AI assistance is wanted; if so the system builds a prompt carrying the project context, Gemini responds with analysis or suggestions, and the user may apply

them, returning control to the design activity. Otherwise the flow proceeds to exporting a report and creating or revoking a share link before reaching the end node. Account deletion is drawn as a separate flow, since it is not part of the modelling workflow and must not be reachable accidentally from within it.

5.5.2 Component Diagram

The component diagram shows the implementation units of the system and the dependencies between them, grouped by the execution environment in which each unit runs.

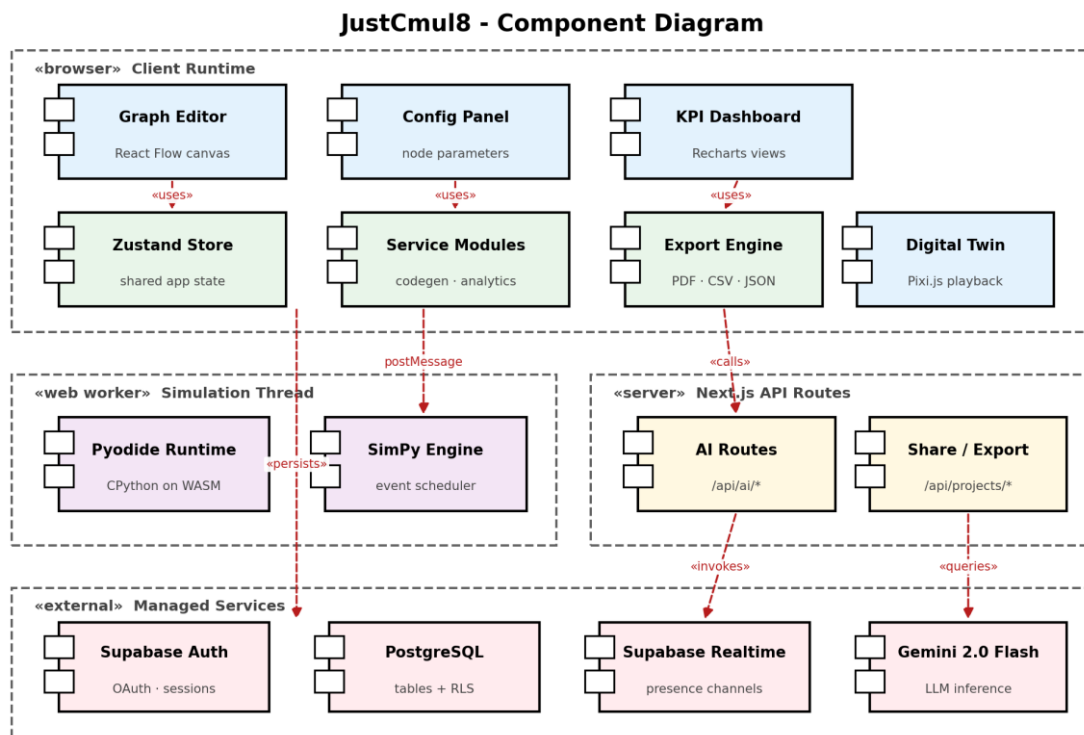


Figure 5.10: Component Diagram of the JustCmul8 Implementation

The **browser client runtime** contains the interface components — the graph editor built on React Flow, the configuration panel, the KPI dashboard built on Recharts and the Pixi.js digital twin — together with the Zustand store, the service modules implementing code generation and analytics, and the export engine. The interface components depend on the store rather than on one another, which allows a panel to be replaced or repositioned without touching the others. The **simulation thread** is a separate group because it executes in a Web Worker; it contains the Pyodide runtime and the SimPy engine and communicates with the client by message passing alone, so a long simulation cannot block rendering. The **server** group contains the Next.js API routes — the AI routes and the share and export routes — which exist for the

narrow set of operations that must not run on the client, chiefly holding the Gemini API key. The **external** group contains the managed services.

5.5.3 Collaboration Diagram

The collaboration diagram shows the objects that participate in a single simulation run and the sequence of messages passed between them, numbered in order of occurrence.

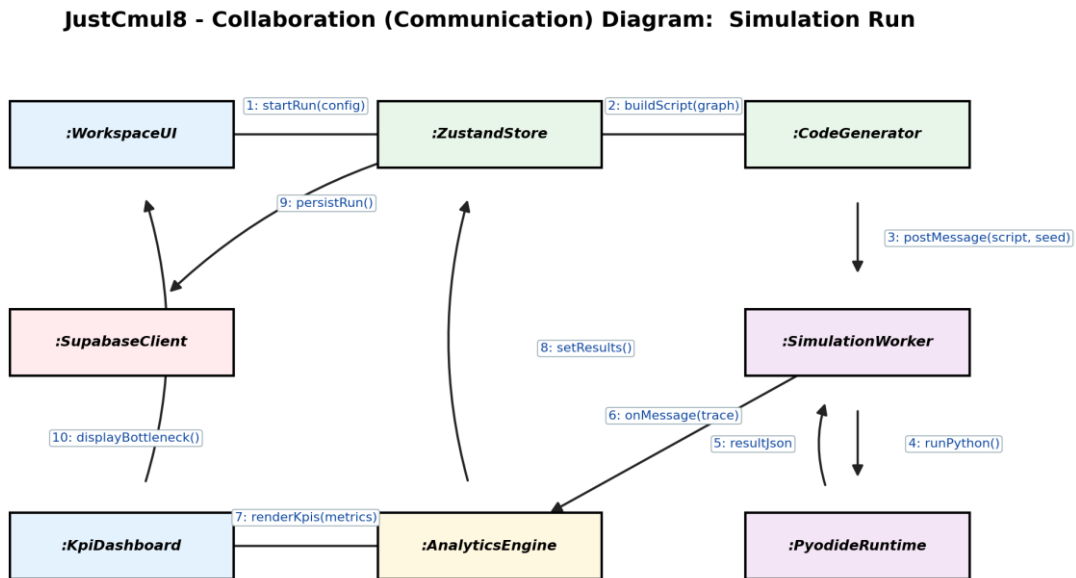


Figure 5.11: Collaboration Diagram for the Simulation Run Interaction

The interaction begins when the workspace interface calls `startRun` on the store with the run configuration (message 1). The store calls `buildScript` on the code generator, passing the current graph (2), and the generated script is posted to the simulation worker (3), a message that crosses the thread boundary. Within the worker, `runPython` is invoked on the Pyodide runtime (4), which returns the result payload (5). The trace is forwarded to the main thread, where the analytics engine receives it (6), renders KPIs to the dashboard (7) and writes the computed results into the store (8). The store persists the run through the Supabase client (9) and the dashboard displays the identified bottleneck (10). Two properties are worth noting: the store mediates between the interface and the services, so any component observing it sees a consistent view of the run, and persistence occurs after analysis so that a run is stored together with its computed metrics in a single write.

5.6 Table Design

The database is a managed PostgreSQL instance. All primary keys are UUIDs generated by the database, all foreign keys are declared with cascading deletion where the child cannot outlive the parent, and every application table carries a Row-Level Security policy. Structures whose shape varies by type — the graph, result payloads and log collections — are stored as JSONB, which keeps the relational schema small and stable while remaining queryable. The graph (nodes with their parameters and positions, and edges) is stored as a single JSONB document on the project, which gives fast loading and export.

5.6.1 Tables

Table 5.5: Table: auth.users (managed by Supabase Auth)

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique user identifier, referenced by every application table
email	text	UNIQUE	Login email address
encrypted_password	text	—	Salted hash of the password; never stored in plain text
raw_user_meta_data	jsonb	—	Registration metadata, including the full name
created_at	timestamptz	DEFAULT now()	Account creation timestamp

Table 5.6: Table: public.profiles

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, FK → auth.users(id) ON DELETE CASCADE	Shares the user's identifier (one-to-one)
display_name	text	—	Name shown in the interface and on collaborator cursors
avatar_url	text	—	Profile image location
default_sim_type	text	DEFAULT 'human_queue'	Domain preselected for new projects
updated_at	timestamptz	DEFAULT now()	Last modification timestamp

Table 5.7: Table: public.projects

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique project identifier
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owning user
name	text	NOT NULL	Display name
sim_type	text	NOT NULL, DEFAULT 'human_queue'	One of the six simulation domains
graph_json	jsonb	DEFAULT {nodes: [], edges: []}	The graph: nodes, parameters, positions and edges
updated_at	timestampz	DEFAULT now()	Last modification timestamp

Table 5.8: Table: public.simulation_runs

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique run identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project that was simulated
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	User who initiated the run
ran_at	timestampz	DEFAULT now()	Run timestamp
duration_seconds	integer	—	Wall-clock execution time
sim_time_seconds	numeric	—	Simulated duration modelled
total_arrived	integer	—	Entities generated
total_completed	integer	—	Entities that reached a sink
bottleneck_node	text	—	Node identified as the constraint in this run
result_json	jsonb	—	Per-node metrics and aggregate KPIs
logs_json	jsonb	—	Sampled log events

Table 5.9: Table: public.chat_history

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique message identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project the exchange concerns

Column	Data type	Constraints	Description
user_id	uuid	FK → auth.users(id) ON DELETE CASCADE	User who sent the message
role	text	NOT NULL, CHECK IN ('user', 'assistant')	Originator of the message
content	text	NOT NULL	Message text
metadata	jsonb	—	Model identifier and any graph payload the reply produced
created_at	timestamptz	DEFAULT now()	Message timestamp

Table 5.10: Table: public.project_shares

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique share record
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project being shared
share_token	text	NOT NULL, UNIQUE, random 128-bit hex default	Opaque token embedded in the public URL
created_by	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owner who created the link
created_at	timestamptz	DEFAULT now()	Creation timestamp
revoked_at	timestamptz	NULL while active	Revocation timestamp

5.6.2 Access Policies, Functions and Indexes

Table 5.11: Row-Level Security Policies

Table	Operations	Policy condition
projects	SELECT, INSERT, UPDATE, DELETE	auth.uid() = user_id
simulation_runs	SELECT, INSERT, DELETE	auth.uid() = user_id
chat_history	SELECT, INSERT, DELETE	auth.uid() = user_id
project_shares	ALL	auth.uid() = created_by
profiles	SELECT, INSERT, UPDATE	auth.uid() = id
realtime.messages	SELECT	Channel topic must be “project:” followed by a project the user owns

Beyond tables and policies, the schema defines database objects that carry behaviour. `get_shared_project(token)` is a security-definer function that returns the name, domain

and graph of one project when the supplied token is active; `get_share_status(token)` returns *invalid*, *revoked* or *active*. Anonymous visitors are granted execute permission on these two functions only, never direct access to any table. `handle_new_user()` is a trigger function that inserts a profile row whenever a new authentication user is created. Indexes on `projects(user_id)`, `projects(updated_at DESC)`, `chat_history(project_id)`, `chat_history(user_id)` and `project_shares(share_token)` keep the dashboard and share lookups fast.

5.6.3 Referential Integrity

- `profiles.id` → `auth.users.id` (one-to-one, cascading)
- `projects.user_id` → `auth.users.id` (cascading)
- `simulation_runs.project_id` → `projects.id` (cascading); `simulation_runs.user_id` → `auth.users.id` (cascading)
- `chat_history.project_id` → `projects.id` (cascading); `chat_history.user_id` → `auth.users.id` (cascading)
- `project_shares.project_id` → `projects.id` (cascading); `project_shares.created_by` → `auth.users.id` (cascading)

Cascading deletion was applied wherever a child row has no meaning without its parent, which is what makes account deletion a single operation that genuinely removes the user's data rather than orphaning it.

5.7 Architecture / Deployment Diagram

The deployment diagram shows the physical nodes on which the system executes and the protocols connecting them.

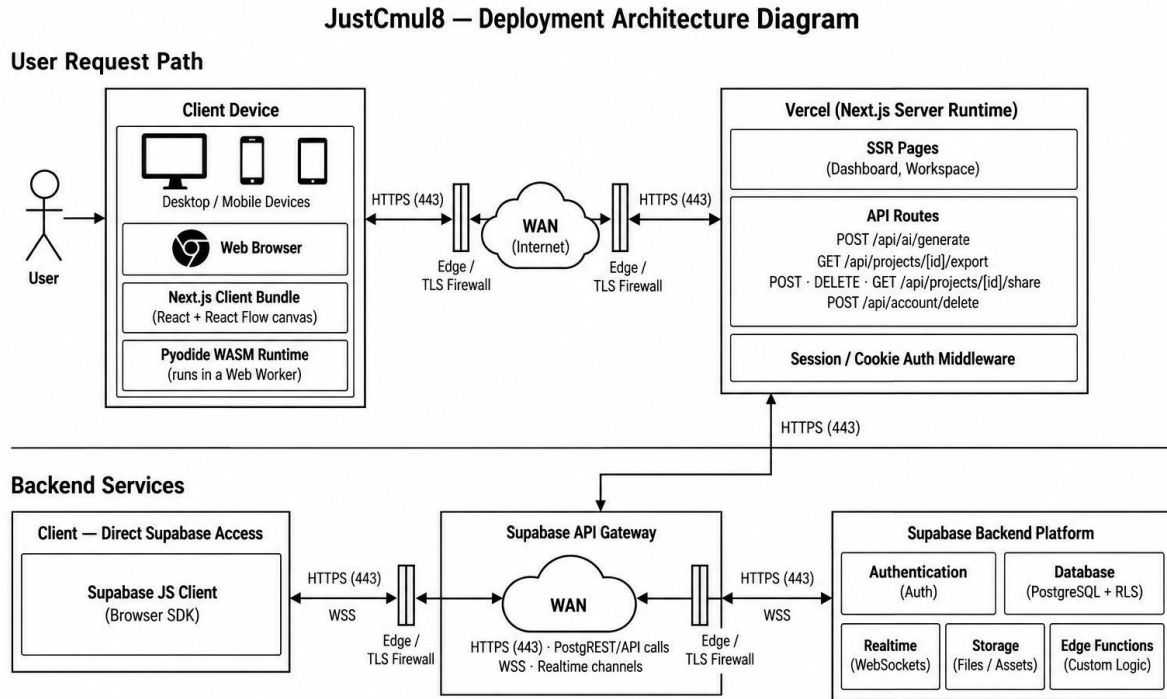


Figure 5.12: Deployment Architecture of JustCmul8

The **client device** hosts a web browser running the Next.js client bundle, which contains the React application and the React Flow canvas, alongside the Pyodide WebAssembly runtime executing in a Web Worker. The diagram makes the central architectural point visible: the simulation runtime is deployed on the user’s own device, not in the data centre.

Requests travel over HTTPS through edge TLS protection and the public internet to the **Vercel** platform, which runs the Next.js server runtime. That runtime serves the dashboard and workspace pages, exposes the API routes — the AI generation, chat and optimisation endpoints, the project export endpoint, the share endpoints and the account deletion endpoint — and applies session middleware. The **backend services tier** contains the Supabase API gateway, reached both from the Next.js server and directly from the browser through the Supabase client; behind it sit authentication, PostgreSQL with Row-Level Security and the realtime WebSocket service.

Permitting the browser to reach the database directly, rather than proxying every read through the application server, is safe precisely because Row-Level Security is enforced within PostgreSQL. The server is reserved for the operations that genuinely require a secret — principally holding the Gemini API key — which keeps the deployment small and, combined

with client-side simulation, keeps the running cost of the platform effectively independent of how heavily it is used.

6. SYSTEM IMPLEMENTATION AND METHODOLOGY

This chapter describes how the design of Chapter 5 was turned into working software: the methodology followed, the technologies and the algorithms at the heart of the platform, the implementation of each module and of the database, the design of the user interface and the most instructive fragments of code.

6.1 Development Methodology

Implementation followed the iterative and incremental model of Section 3.5. The engine was proved first as a prototype spike, verified against analytical queueing results, and only then were the modules that consume it built. Each module then passed through its own cycle of design, coding, unit testing and integration.

- **Version control** — Git with a GitHub remote; work was committed in small, described increments so that the history reads as a record of the project.
- **Code review** — every module was reviewed by the team member who did not write it before being merged.
- **Type safety** — TypeScript across the whole code base, so that mismatches between a node's parameters and the code that consumes them are caught at compile time.
- **Separation of concerns** — analytics, code generation, Monte Carlo and export are plain TypeScript modules with no interface dependency, which makes them independently testable.
- **Verification first** — the simulation engine is checked against closed-form queueing results by a separate native harness before any interface work depends on its output.
- **Continuous documentation** — module documents were written alongside the code and later consolidated into this report.

6.2 Technologies Used

The technology stack was summarised with its justification in Section 2.3.3. The paragraphs below describe how each technology is used in the implementation.

- **Next.js 16 and React 19** provide routing, server-side rendering of the landing and authentication pages, and the API routes that proxy the AI service and issue share tokens. The workspace itself is a client-rendered single-page application.
- **TypeScript** is used for all application and service code; the SimPy program is generated as Python text by a TypeScript module.
- **React Flow** implements the canvas, node handles, edges, viewport and selection; **Dagre** arranges generated and template graphs.
- **Pyodide** loads CPython compiled to WebAssembly inside a Web Worker; the generated script is executed with `runPythonAsync`, and results are returned by callback functions registered in the Python global namespace.
- **SimPy** supplies the event scheduler, resources, stores, containers and interrupts on which the generated model is built.
- **Zustand** holds application state in slices; **Recharts** draws the dashboard charts; **Pixi.js** renders the digital twin; **Framer Motion** and **Lenis** provide transitions and smooth scrolling on the landing page.
- **Supabase** provides authentication (email and OAuth), PostgreSQL with Row-Level Security, and Realtime channels for presence and broadcast.
- **Google Gemini** is accessed through a model router that maintains a priority list of models and a short-lived health record for each.
- **jsPDF and html-to-image** generate the executive report and capture the canvas in the browser.
- **Tailwind CSS v4** styles the interface consistently through utility classes and design tokens.

6.3 Algorithms and Techniques Used

6.3.1 Graph Representation and Validation

The simulation model is a directed graph and is represented in memory as an **adjacency list**: a map from each node identifier to the list of its outbound neighbours. Simulation graphs are sparse — a node typically connects to one or two successors — so an adjacency matrix, whose size grows with the square of the node count regardless of how many edges exist, would be wasteful. The list also matches the access pattern of the engine: routing an entity requires the successors of one particular node, which the list provides directly. Before a run, the interface traverses the graph from its nodes with a visited set and reports any node that cannot be reached,

so that a disconnected model is refused with a message naming the offending node instead of failing inside the engine.

Table 6.1: Complexity of Graph Operations on the Adjacency List

Operation	Complexity	Where it is used
Add node / add edge	$O(1)$	Placing and connecting nodes on the canvas
Retrieve successors	$O(\deg v)$	Routing an entity onward
Graph traversal	$O(V + E)$	Connectivity validation before a run
Space occupied	$O(V + E)$	Whole-graph representation

6.3.2 Event Scheduling

The event queue is the central structure of any discrete event simulator: it holds events scheduled for future simulated times, and the engine repeatedly removes the earliest and processes it. SimPy implements it as a **binary min-heap** keyed on event time, so insertion and extraction each cost logarithmic time in the number of queued events. A sorted list would make extraction constant but insertion linear; an unsorted list inverts the problem. The heap balances both, and ties in event time are broken by a monotonically increasing counter so that events scheduled for the same instant are processed in the order they were created. JustCmul8 obtains this behaviour from SimPy rather than re-implementing it, which is precisely the leverage the architecture is designed to exploit.

6.3.3 Random Variate Generation

Variability is introduced by sampling inter-arrival and service times from the five distributions of Table 1.2. In the generated script the exponential distribution uses `random.expovariate(1/mean)`, the normal distribution uses `random.gauss` truncated at zero (a negative service time has no physical meaning), the uniform distribution spans $\text{mean} \pm 50$ per cent, and the deterministic distribution returns the mean unchanged. Routing at decision nodes uses cumulative-probability sampling: a uniform variate is drawn and compared against the running sum of the configured route probabilities. Source nodes can distribute arrivals across several successors in round-robin fashion.

6.3.4 Metric Computation

Utilisation is computed by accumulating busy-seconds: whenever the number of busy servers changes, the interval since the last change multiplied by the previous busy count is added to a

running total, and utilisation is that total divided by capacity times run duration. **Waiting time** is the interval between an entity joining a queue and starting service, and percentiles are computed from the real per-entity samples. To keep memory bounded on long runs, entity logs are **reservoir-sampled**, retaining a uniform random sample of whole entity journeys rather than a prefix of the run, which would bias tail percentiles.

The **Little's Law check** compares the time-averaged number of entities in the system with the arrival rate multiplied by the mean time in the system, reports the percentage discrepancy and classifies the run as steady-state, transient or accumulating backlog. The **bottleneck** is the resource or queue node with the highest average waiting time. The **system health score** is a weighted composite of four components, as defined in Table Table 4.4:

$$\text{Health} = \text{Completion (35\%)} + \text{Utilisation balance (30\%)} + \text{Stability (20\%)} + \text{Tail latency (15\%)}$$

where a maximum utilisation above 88 per cent or 95 per cent progressively reduces the utilisation component, an unstable or inconsistent Little's Law result reduces the stability component, and a 99th-percentile wait more than five times the median reduces the tail component.

6.3.5 Monte Carlo Confidence Intervals

For each metric the mean and sample standard deviation across replications are computed, and the 95 per cent confidence interval is formed as:

$$CI = \bar{x} \pm 1.96 \times (s / \sqrt{n})$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and n the number of replications. Because the half-width falls with the square root of n , quadrupling the replications halves the interval, which is why the interface offers presets of 10, 20, 50 and 100 rather than an unbounded field.

6.3.6 AI Model Routing

The AI features are served by a **model router** rather than a hard-coded model. The router holds a priority list of Gemini models, classifies each failure (unavailable, rate-limited, empty response), and marks a failing model unhealthy for a short time-to-live so that subsequent requests skip it and go straight to the next candidate. This gives the assistant the resilience of automatic fail-over without any action from the user, and means that a model retired by the provider does not take the feature down. Generated graphs are validated before display, and every recommendation is presented for approval.

6.3.7 Presence and Broadcast

Collaboration uses two channel mechanisms. **Presence** tracks who is in the project: each client publishes a small payload (identifier, name, colour, selected and edited node) and receives the merged state of everyone else. **Broadcast** carries transient messages — cursor positions and graph operations — to the other clients, which ignore messages that originate from themselves. Cursor colours are derived deterministically from the user identifier, so the same person always appears in the same colour.

6.4 Module-wise Implementation

The table below relates each module to its implementation footprint. Line counts are for the source files of the module and are drawn from the delivered code base.

Table 6.2: Module-wise Implementation Summary

Module	Principal implementation	Approx. lines
1 Platform and Data Layer	Auth pages, dashboard, settings, middleware, Supabase clients, account deletion route	≈ 3,850 (app/)
2 Visual Graph Editor	NodeCanvas (951), NodePalette, layoutEngine, simTypeRegistry (715)	≈ 1,900
3 Node Configuration Panel	NodePropertiesPanel (1,109), shared types (514)	≈ 1,600
4 Simulation Engine	codeGenerator (783), pyodideEngine/Worker, clientEngine, CodeInspectorPanel	≈ 1,300
5 AI Assistant	AIChatPanel (962), chat and generate routes, systemPrompt, graphOps (739), modelRouter (370)	≈ 2,750
6 AI System Optimiser	optimize route (182), flowAnalysis (268)	≈ 450
7 KPI Dashboard and Results	analyticsEngine (806), resolveKpiMetrics, SimResultsPanel (1,047), AdvancedResultsDashboard, eight result tabs	≈ 4,400
8 Monte Carlo and Scenarios	monteCarlo (132), monteCarloRunner, MonteCarloPanel (778)	≈ 970
9 Digital Twin Playback	DigitalTwinCanvas (366), PlaybackControls, timelineSelectors	≈ 510
10 Template Gallery	TemplateGallery (135), templates in simTypeRegistry	≈ 850
11 Executive Report and Share	pdfReportGenerator (235), reportGenerator, jupyterExport, share modal, viewer, share/export routes	≈ 700 (excl. API routes)
12 Real-Time Collaboration	usePresence (143), LiveCursor	≈ 160

6.5 Database Implementation

The schema is defined in a single SQL script that is idempotent — every statement uses `IF NOT EXISTS` or a guarded block — so it can be re-run safely against an existing database. Tables are created first, Row-Level Security is enabled on each, policies are added inside guarded blocks, and indexes, functions and the profile trigger complete the script. The excerpt below shows the projects table and its policies; the remaining tables follow the same pattern.

Listing 6.1: Projects table and Row-Level Security policies

```
CREATE TABLE IF NOT EXISTS public.projects (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  name text not null,
  sim_type text not null default 'human_queue',
  graph_json jsonb default '{"nodes": [], "edges": []}':jsonb,
  updated_at timestamptz default now()
);

ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view their own projects"
  ON public.projects FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can update their own projects"
  ON public.projects FOR UPDATE
  USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

Public sharing is implemented without ever granting anonymous users access to a table. A share token is looked up inside a security-definer function that returns only the fields needed to render the project:

Listing 6.2: Read-only public access through a security-definer function

```
CREATE OR REPLACE FUNCTION public.get_shared_project(token text)
RETURNS TABLE (name text, sim_type text, graph_json jsonb) AS $$
  SELECT p.name, p.sim_type, p.graph_json
  FROM public.projects p
  JOIN public.project_shares s ON s.project_id = p.id
  WHERE s.share_token = token AND s.revoked_at IS NULL
  LIMIT 1;
$$ LANGUAGE sql SECURITY DEFINER STABLE;

GRANT EXECUTE ON FUNCTION public.get_shared_project(text)
  TO anon, authenticated;
```

6.6 User Interface Design

The interface follows the principles of **plain language**, **low cognitive load** and **familiar interaction**. Vocabulary is chosen for the user's domain, not for simulation theory: a bank

model offers a “Teller” and a “Waiting Line”, a factory model a “Machine” and a “Buffer”, and the result tabs are named for what they show — *Executive Summary*, *Traffic Heatmap*, *Line Buildup Timeline*, *Station Counters*, *Person / Item Journey*, *Live Activity Log* and *Cost and ROI*. The workspace is arranged in three regions: a node palette on the left, the canvas in the centre and a contextual panel on the right that changes between node configuration, AI chat, results and Monte Carlo analysis depending on what the user is doing.

The landing page communicates the proposition — simulation without installation, licence or code — and leads to registration. The visual identity uses a restrained neutral palette with a single accent, smooth scrolling and motion for polish, and a responsive layout so that the interface remains usable on tablets and phones. Errors are written as instructions, naming the offending node and stating the remedy.

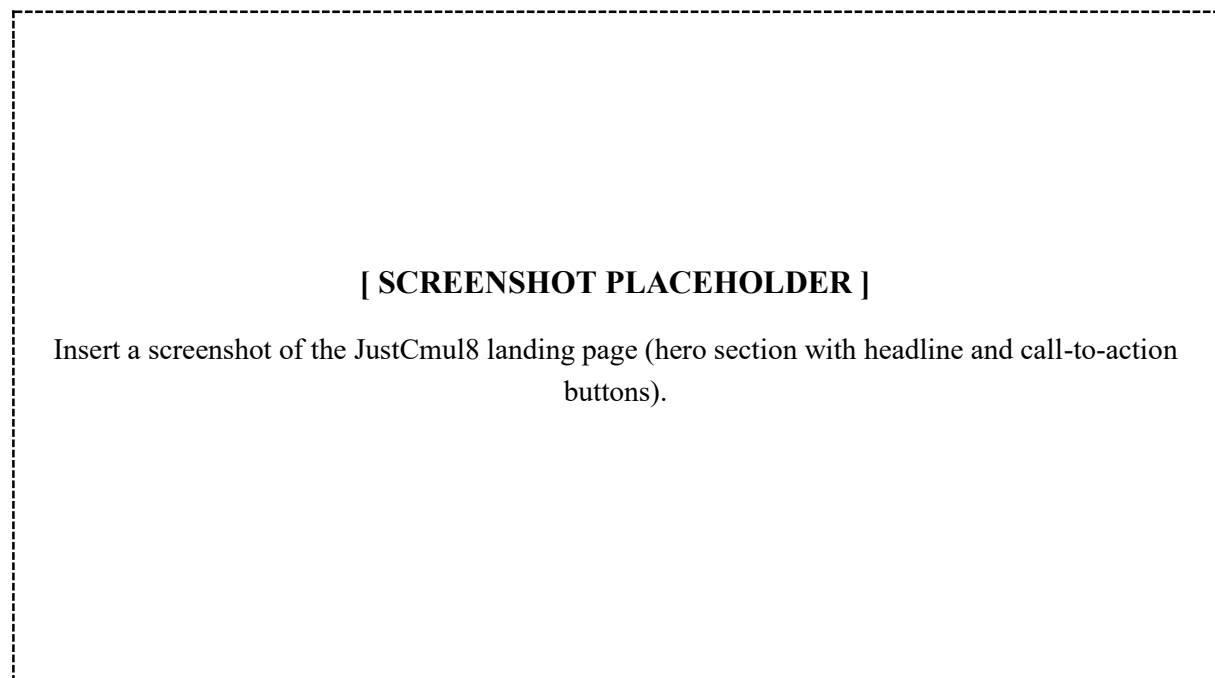


Figure 6.1: Landing Page

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Login / Sign-up page showing email fields and the Google and GitHub sign-in buttons.

Figure 6.2: Registration and Login Screens

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Project Dashboard listing the user's projects with domain, last-modified time and the New Project control.

Figure 6.3: Project Dashboard

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Workspace: node palette on the left, canvas with a connected model in the centre, toolbar with Run controls at the top and a side panel on the right.

Figure 6.4: Workspace with the Visual Graph Editor

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Node Configuration Panel for a Resource node (capacity, service-time distribution, cost parameters).

Figure 6.5: Node Configuration Panel

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Template Gallery showing starter models grouped by the six domains.

Figure 6.6: Template Gallery

6.7 Important Code Snippets

This section reproduces the fragments of code that best show how the system works. Each is taken directly from the delivered source and accompanied by a short explanation.

6.7.1 Distribution Sampling in the Generated SimPy Script

The generated program contains a single sampling function that every source and resource calls. It shows how the five distributions of Table Table 1.2 are realised, including truncation of the normal distribution at zero.

Listing 6.3: Distribution sampler in the generated script (codeGenerator.ts)

```
def sample(dist_type, mean, std=None):
    if dist_type == "exponential":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
    elif dist_type == "uniform":
        return random.uniform(mean * 0.5, mean * 1.5)
    elif dist_type == "normal":
        s = std if std is not None else mean * 0.3
        return max(0, random.gauss(mean, s))
    elif dist_type == "deterministic":
        return max(0, mean)
    elif dist_type == "poisson":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
```

6.7.2 Resource Service with Busy-Time Accounting

This fragment is the heart of the model: an entity waits, seizes a server, is served for a sampled time and releases it. Busy-time is accumulated whenever the busy count changes, which is what makes the reported utilisation exact rather than sampled. The comment records a design correction: the next hop is returned to the caller so that the server is released before the entity moves on, preventing the server from being held for the rest of the entity's journey.

Listing 6.4: Resource service and utilisation accounting (excerpt)

```
def do_resource_service(env, eid, node_id, cfg, s, resources,
                       queued_at, queue_stats=None):
    params = cfg["params"]
    wait_time = env.now - queued_at
    s["waits"].append(wait_time)
    if s["busyCount"] > 0:
        s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] += 1
    svc_time = sample(params.get("serviceDistribution", "exponential"),
                     params.get("serviceTimeMean", 1))
    yield env.timeout(svc_time)
    s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] -= 1
    s["entitiesOut"] += 1
    return get_next_target(node_id)    # caller releases the server first
```

6.7.3 Running Python in the Browser

The worker receives a start message, generates the script from the graph, registers two callbacks in Python's global namespace and runs the script. Because the callbacks are ordinary JavaScript functions, the Python code can stream ticks and deliver its result without any network traffic.

Listing 6.5: Executing the generated SimPy script in the Web Worker (pyodideWorker.ts)

```
case "start": {
    const { python } = generateSimPyScript(
        params.graph, params.durationSeconds, params.tickIntervalSeconds);

    pyodide.globals.set("emit_sim_tick", (tickJson: string) => {
        if (!stopped) emit(JSON.parse(tickJson));
    });
    pyodide.globals.set("emit_sim_result", (resultJson: string) => {
        if (!stopped) emit(JSON.parse(resultJson));
    });

    await pyodide.runPythonAsync(python);    // SimPy runs in WebAssembly
    break;
}
```

6.7.4 Confidence Interval Computation

Listing 6.6: Monte Carlo 95% confidence interval (monteCarlo.ts, abbreviated)

```
export function computeConfidenceInterval(samples: number[]): RunStatistic {
  const n = samples.length;
  const mean = samples.reduce((a, b) => a + b, 0) / n;
  const variance =
    samples.reduce((a, b) => a + (b - mean) ** 2, 0) / (n - 1);
  const stdDev = Math.sqrt(variance);
  const marginOfError = 1.96 * (stdDev / Math.sqrt(n));
  return {
    mean,
    ciLower: Math.max(0, mean - marginOfError),
    ciUpper: mean + marginOfError,
    stdDev, samples,
  };
}
```

6.7.5 System Health Score

Listing 6.7: Composite system health score (analyticsEngine.ts, abbreviated)

```
const completionScore = Math.min(1, completed / arrived) * 35;

let utilScore = 30; // utilisation balance
if (maxUtil > 0.95) utilScore = 15; // severe bottleneck
else if (maxUtil > 0.88) utilScore = 22;

let stabilityScore = 20; // Little's Law consistency
if (!littlesLaw.isStable) stabilityScore = 5;
else if (littlesLaw.discrepancyPercent > 20) stabilityScore = 12;

let tailScore = 15; // tail latency
if (waitStats.p99 > waitStats.p50 * 5) tailScore = 8;

return clamp(Math.round(completionScore + utilScore +
  stabilityScore + tailScore), 10, 100);
```

Taken together, these fragments show the shape of the platform: a small, readable SimPy program produced from a diagram; a worker that runs it without touching the network; analytics that turn its output into verifiable indicators; and a database that enforces who may see what.

7. SYSTEM TESTING

A simulator can be entirely usable and still be wrong, which makes testing an unusually important part of this project. Interface testing can show that a button works; only comparison with independent mathematics can show that the numbers behind the button are right. The testing of JustCmul8 therefore has two complementary halves: verification of the **engine** against analytical results and invariants, and functional testing of the **application** around it. This chapter describes the strategy, the units under test, the design of the test cases, the approach and the results.

7.1 Testing Strategy

Testing ran alongside development rather than following it, for the reason given in Chapter 3: defects found in an early module are cheaper to correct while the modules that depend on it are still being written. The strategy is layered, with fast, narrow tests at the base and broader, slower checks above.

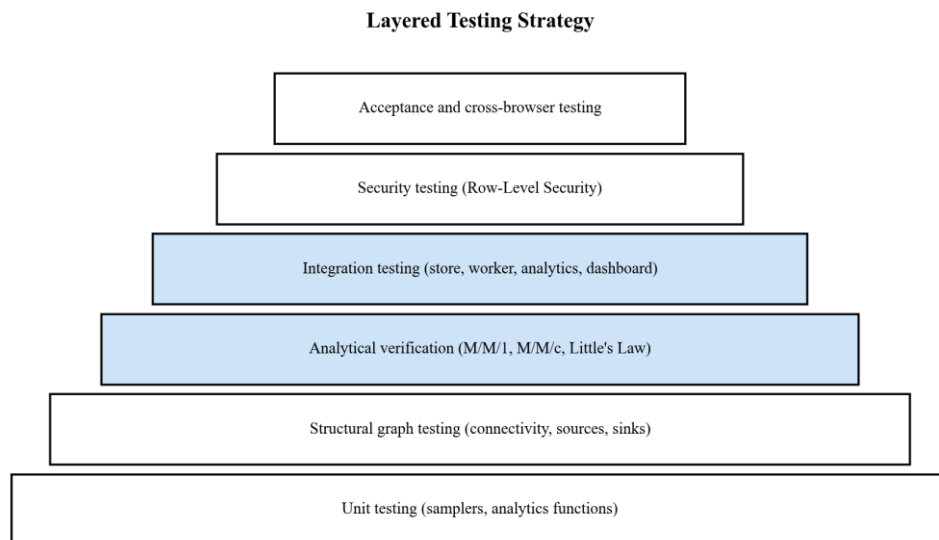


Figure 7.1: Layered Testing Strategy for JustCmul8

- **Unit testing** exercises the pure TypeScript modules — samplers and analytics functions — in isolation.
- **Structural graph testing** checks that invalid models — no blocks, no connections, unreachable nodes — are refused before execution with a message naming the fault.

- **Analytical verification** is the most valuable layer: the engine’s output for reference queues is compared with closed-form M/M/1 and M/M/c results, and with Little’s Law.
- **Invariant testing** applies conservation and flow-balance laws to a complex multi-stage model where no closed-form answer exists.
- **Integration testing** exercises the paths that cross module boundaries: store, code generator, worker, analytics and dashboard.
- **Security testing** attacks authorisation directly at the database, bypassing the interface.
- **Cross-browser and acceptance testing** checks identical behaviour across browsers and that a first-time user can obtain a result without a tutorial.

7.2 Testable Units

Table 7.1: Testable Units and the Techniques Applied to Them

Unit	Location	What is tested	Technique
Distribution samplers	codeGenerator.ts (generated sample), distributions.ts	Mean, spread and truncation of each of the five distributions	Statistical unit test
Code generator	lib/simulation/codeGenerator.ts	Every node type produces valid, executable SimPy	Execution of generated script
Analytics engine	lib/simulation/analyticsEngine.ts	Utilisation, waits, percentiles, Little’s Law, health score	Known-answer test; analytical oracle
Monte Carlo aggregation	lib/simulation/monteCarlo.ts	Mean, standard deviation and 95% interval	Known-answer test
Graph validation	NodeCanvas.tsx (validateGraphConnectivity)	Empty, unconnected and partially connected graphs	Boundary and negative testing
Graph operations	lib/ai/graphOps.ts	Validation of AI-generated graphs	Negative testing with malformed input
Model router	lib/ai/modelRouter.ts	Fail-over between models on error classes	Fault injection
Database policies	supabase/schema.sql	Row-Level Security and share functions	Direct-request security testing
Presence channel	lib/realtime/usePresence.ts	Join authorisation, presence, cursor broadcast	Multi-session integration test
Export modules	pdfReportGenerator.ts, jupyterExport.ts	Content of PDF, CSV, JSON and notebook exports	Output inspection

7.3 Test Case Design with Reference to the Project

Two families of test cases were designed. The first are **engine verification cases**, whose expected results are derived from queueing theory and which produce numerical outcomes. The second are **functional test cases** for the application, whose expected results are stated behaviours traceable to the requirements of Section 2.1.

7.3.1 Engine Verification Cases

The engine verification suite executes the generated SimPy script verbatim, five independent replications per benchmark, over a horizon of 36,000 simulated seconds, and compares the results with theory. Benchmark B1 is an M/M/1 queue at utilisation 0.8; B1v repeats it without an explicit queue node; B2 is an M/M/3 queue; B3 is a deliberately saturated queue; B4 checks time-unit handling; and B5 checks Little's Law. Deviation is measured against a stated tolerance.

Table 7.2: Engine Verification Test Cases and Results

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-01	B1 M/M/1	Utilisation ρ	0.8000	0.7979	-0.27%	$\pm 3.5\%$	Pass
E-02	B1 M/M/1	Queue wait W_q	4.000	4.005	+0.13%	$\pm 3.5\%$	Pass
E-03	B1 M/M/1	System time W	5.000	5.004	+0.08%	$\pm 3.5\%$	Pass
E-04	B1 M/M/1	Queue length L_q	3.200	3.188	-0.39%	$\pm 3.5\%$	Pass
E-05	B1 M/M/1	Percentiles from real samples (4,000 points)	real	real	—	—	Pass
E-06	B1 M/M/1	WIP conservation (arrived – completed)	3.8	3.8	—	—	Pass
E-07	B1 M/M/1	Source node arrivals reported	28,584	28,584	—	—	Pass
E-08	B1v (no queue)	Utilisation ρ	0.8000	0.7979	-0.27%	$\pm 3.5\%$	Pass
E-09	B1v (no queue)	Queue wait W_q	4.000	4.005	+0.13%	$\pm 3.5\%$	Pass
E-10	B1v (no queue)	System time W	5.000	5.004	+0.08%	$\pm 3.5\%$	Pass
E-11	B1v (no queue)	System WIP L	4.000	3.980	-0.51%	$\pm 3.5\%$	Pass
E-12	B2 M/M/3	Utilisation per server	0.8000	0.7983	-0.21%	$\pm 4.0\%$	Pass
E-13	B2 M/M/3	Queue wait W_q (Erlang-C)	1.079	1.101	+2.09%	$\pm 4.0\%$	Pass
E-14	B2 M/M/3	System time W	2.079	2.100	+1.03%	$\pm 4.0\%$	Pass
E-15	B3 saturated	Total arrivals	1,200	1,181	-1.55%	$\pm 5.0\%$	Pass
E-16	B3 saturated	Total completed	600	603.2	+0.53%	$\pm 5.0\%$	Pass
E-17	B3 saturated	Utilisation ρ	1.000	0.9968	-0.32%	$\pm 2.0\%$	Pass
E-18	B3 saturated	Final WIP within 600 ± 70	530–670	578.2	—	2σ	Pass
E-19	B3 saturated	Little's Law verdict (accumulating backlog)	backlog	backlog	—	—	Pass

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-20	B3 saturated	Health score degrades below 70	< 70	45.8	—	—	Pass
E-21	B4 time units	Run horizon identical in seconds, minutes, hours	3,600	3,600 / 3,600 / 3,600	—	—	Pass
E-22	B4 time units	Tick interval identical across units	3.6	3.6 / 3.6 / 3.6	—	—	Pass
E-23	B4 time units	Arrival-count spread across units	< 1%	0.000%	—	—	Pass
E-24	B4 time units	Tick granularity for a 5 s run	≤ 0.1 s	0.05 s (100 ticks)	—	—	Pass
E-25	B5 Little's Law	Discrepancy between L and λW	< 5%	2.41%	—	—	Pass
E-26	B5 Little's Law	Time-averaged true WIP	4.000	3.982	−0.46%	±8.0%	Pass
E-27	B5v (no queue)	Little's Law discrepancy	< 5%	2.41%	—	—	Pass
E-28	B5v (no queue)	Time-averaged true WIP	4.000	3.982	−0.46%	±8.0%	Pass

7.3.2 Invariant Cases for a Complex Multi-Stage Model

Benchmark B6 is a hospital-flow model with a triage station, a probabilistic decision node routing 70 per cent of patients to minor treatment and 30 per cent to major treatment, and two sinks, executed with and without patient abandonment (reneging) at an arrival rate of 1.2 per second over 7,200 simulated seconds. No closed-form answer exists, so the test asserts **invariants** — laws that any correct simulation must obey.

Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6)

ID	Invariant	Meaning
I1	Conservation residual ≥ 0 and below 5% of arrivals	Arrived = completed + reneged + dropped + stuck; nothing is created or lost
I2	Flow balance at every node	Entities in = entities out + current depth
I3	Decision split equals the configured probability	70/30 routing is honoured (observed 0.700 with abandonment on, 0.687 off)
I4	Total completed equals the sum over all sinks	Sink accounting is complete
I5	Throughput clears offered load ($\geq 95\%$)	Stable stations serve what they are offered
I6	Per-station utilisation approximates ρ	Triage ≈ 0.72 , minor ≈ 0.70 , major ≈ 0.72

7.3.3 Functional Test Cases

The functional test cases below cover every module. Each carries the requirement it verifies; the **Status** column is completed by the tester as the case is executed (Pass / Fail with date).

Table 7.4: Functional Test Case Design

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-01	Platform	Register with a valid, unused email and a compliant password	Account created, profile row inserted, session started	FR-01	
TC-02	Platform	Register with an already-registered email	Registration refused with a clear message	FR-01	
TC-03	Platform	Register with a weak password	Submission blocked; requirement shown	FR-01	
TC-04	Platform	Log in with correct credentials	Session issued; dashboard loads the user's projects	FR-01	
TC-05	Platform	Log in with a wrong password	Access refused without revealing which field was wrong	FR-01	
TC-06	Platform	Sign in with Google and with GitHub OAuth	Account linked; session established	FR-01	
TC-07	Platform	Open a workspace URL while logged out	Redirected to login before any data is fetched	FR-02	
TC-08	Platform	Request a password reset and set a new password	Email link works; new password accepted	FR-03	
TC-09	Platform	Create a project in a chosen domain	Project appears on the dashboard with the domain	FR-04	
TC-10	Platform	Rename and delete a project	Name changes; deletion removes project and its runs and chats	FR-04	
TC-11	Platform	Move a node, then reload the page	Position persisted without an explicit save	FR-05	
TC-12	Platform	Query another user's project by identifier	Row-Level Security returns no rows	NFR-05	
TC-13	Platform	Delete the account after confirmation	User row and all owned data removed	FR-06	
TC-14	Graph Editor	Place each of the fifteen node types	Every node renders with correct icon, handles and defaults	FR-07	
TC-15	Graph Editor	Connect two nodes and move one	Edge created and re-routes live	FR-08	
TC-16	Graph Editor	Delete a mid-graph node	Node and incident edges removed together	FR-07	
TC-17	Graph Editor	Run an empty canvas / an unconnected graph	Run refused; message explains the fault	FR-09	
TC-18	Graph Editor	Run a graph with a node unreachable from the rest	Run refused; the disconnected node is named	FR-09	
TC-19	Node Config	Enter capacity 0 or a negative value	Value rejected with a corrective message	FR-11	
TC-20	Node Config	Switch the service distribution	Parameter fields change to match	FR-10	
TC-21	Node Config	Enter routing probabilities not summing to one	Warning shown before the run	FR-11	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-22	Engine	Run a valid single-server model	Run completes; arrivals, completions and KPIs reported	FR-12	
TC-23	Engine	Run the same model with duration in seconds, minutes and hours	Equivalent horizons produce equivalent results	FR-13	
TC-24	Engine	Interact with the canvas during a long run	Interface remains responsive	FR-14	
TC-25	Engine	Stop a run part-way; change playback speed	Run halts; speed changes apply	FR-15	
TC-26	Engine	Open the code inspector and export the notebook	Readable SimPy shown; .ipynb downloads and runs	FR-25	
TC-27	AI Assistant	Describe “a clinic with two doctors and a waiting room”	Valid graph proposed with source, queue, resource, sink	FR-16	
TC-28	AI Assistant	Inject a response with an unknown node type	Candidate rejected by validation; not displayed	FR-17	
TC-29	AI Assistant	Generate, then discard the preview	Project remains unmodified	FR-17	
TC-30	AI Assistant	Make the preferred model unavailable	Router falls back to the next model	FR-17	
TC-31	AI Optimiser	Diagnose a deliberately under-provisioned server	Correct node identified with a capacity recommendation	FR-18	
TC-32	AI Optimiser	Apply the suggested fix	Graph updated as an ordinary edit	FR-18	
TC-33	KPI Dashboard	Three-stage line with one slow stage	Slow stage flagged as bottleneck	FR-20	
TC-34	KPI Dashboard	Open the Flow and Balance Check for a stable model	Little’s Law discrepancy shown with a steady-state verdict	FR-20	
TC-35	KPI Dashboard	Enter cost parameters and open Cost and ROI	Total cost and the return of a change are reported	FR-21	
TC-36	Monte Carlo	Run 20 replications	Mean, standard deviation and 95% interval per metric	FR-22	
TC-37	Monte Carlo	Apply a fix, then compare baseline and scenario	Per-metric difference shown with both intervals	FR-22	
TC-38	Digital Twin	Play back a completed run; scrub and change speed	Entities animate; position changes without re-running	FR-23	
TC-39	Templates	Instantiate one template from each domain	Canvas pre-populated; every template runs unmodified	FR-24	
TC-40	Templates	Instantiate, then modify the new project	Source template unchanged	FR-24	
TC-41	Export / Share	Export graph as JSON and results as CSV	Files download with correct content	FR-25	
TC-42	Export / Share	Generate the executive PDF report	PDF contains summary, canvas snapshot and KPI tables	FR-25	
TC-43	Export / Share	Open a share link in a private window	Project renders read-only with no editing controls	FR-26	
TC-44	Export / Share	Attempt a direct write using only a share token	Refused by the database	FR-26	
TC-45	Export / Share	Open a revoked share link	Explanatory notice shown instead of an error	FR-26	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-46	Collaboration	Open one project in two sessions	Presence and live cursors appear in both	FR-27	
TC-47	Collaboration	Try to join a project channel without access	Channel not joined	FR-27	
TC-48	Cross-browser	Run one model on Chrome, Edge, Firefox and Safari	Consistent metrics; canvas and playback render correctly	NFR-08	
TC-49	Responsiveness	Load the workspace at a phone viewport	Layout adapts; panels remain reachable	NFR-08	
TC-50	Profile	Change display name and default domain	Values persist and apply to new projects	FR-06	

7.4 Test Approach

- **Oracle-based testing.** For the engine, the expected value comes from mathematics rather than from a previous run: closed-form M/M/1 and Erlang-C results, Little’s Law and conservation laws. This avoids the trap of a test that merely records whatever the code currently does.
- **Replication and confidence.** Each analytical benchmark runs five independent replications and reports a 95% interval, so that a pass reflects the behaviour of the engine rather than the luck of one run.
- **Tolerance discipline.** Tolerances are stated in advance and differ by benchmark — tighter for utilisation, looser for waiting times, which have higher variance.
- **Invariant testing.** Where no closed-form answer exists, laws that must always hold are asserted instead.
- **Negative and boundary testing.** Zero and negative parameters, zero duration, empty graphs, unreachable nodes and malformed AI responses are supplied deliberately.
- **Adversarial security testing.** Authorisation is tested by issuing requests directly against the database, bypassing the interface: reading another user’s project, writing with only a share token, joining a channel without access.
- **Cross-environment testing.** An identical model is executed on several browsers and the metrics compared; layout is checked at phone widths.
- **User acceptance testing.** Participants matching the personas of Section 2.1.7 are asked to build and run a model without instruction; the objective is to confirm that a first result is reachable without a tutorial, which is the operative test of the no-code claim.
- **Defect management.** Every defect is raised as an issue in the project tracker, fixed, re-tested and linked to the change that closed it.

Entry and exit criteria. A module enters integration testing when its unit tests pass and it has been reviewed. The build exits testing when all engine verification cases pass, no high-severity defect remains open, and every functional test case has been executed and recorded.

7.5 Test Results and Validation

7.5.1 Engine Verification Results

All **28 of 28** analytical checks passed. For the M/M/1 benchmark the simulated utilisation was 0.7979 against a theoretical 0.8000, the mean queue wait 4.005 s against 4.000 s and the mean system time 5.004 s against 5.000 s; the 95% confidence interval of each estimate contained the theoretical value. For the three-server benchmark the simulated queue wait was 1.101 s against an exact Erlang-C value of 1.079 s, a deviation of 2.09 per cent, inside the ± 4 per cent tolerance. For the complex hospital-flow model all **22 of 22** invariants held, both with and without patient abandonment. Every deviation from theory was below 2.1 per cent.

Two observations from the exercise deserve record. First, the exact Erlang-C probability of waiting for three servers at an offered load of 2.4 is 0.6472, so the exact waiting time is 1.0787 s; the benchmark was scored against the exact value, not a rounded figure quoted in a planning brief. Second, the saturated benchmark confirmed that the analytics respond correctly to overload: the health score fell to 45.8 and the Little's Law verdict changed to *accumulating backlog*, so a failing system is reported as failing.

7.5.2 Edge-Case Probes

A set of edge-case probes recorded how the engine behaves at the boundaries of its inputs.

Table 7.5: Edge-Case Probes and Observed Behaviour

Probe	Observed behaviour
Zero service time	Run completes; 39 completed with utilisation 0.000, as expected
Zero duration	Rejected by the engine with a clear error
Zero arrival rate	Recorded for input-validation hardening; the sampler falls back to a default rate
Recurring schedule at time zero	A guard prevents an unbounded zero-delay loop
Very long run (28,714 entities)	Completed in 3.65 s; log retention capped at 20,000 events using an entity reservoir
Floating-point accumulation over 36,000 ticks	Simulated clock reached exactly 36,000.0
Multiple sinks	Both sinks counted (240 + 240 = 480 completed)

Probe	Observed behaviour
Two resources in series ($\rho = 0.5$ each)	Backlog of 2 over 1,768 arrivals; the first stage remains seized while the entity is in the second stage (blocking after service), which the model documents

7.5.3 Validation against Requirements

Validation asks whether the right system was built. The traceability below relates each group of requirements to the evidence that supports it.

Table 7.6: Requirements Traceability

Requirement group	Evidence
FR-01 – FR-06 Platform	Functional cases TC-01 – TC-13, TC-50; database policies and cascade rules in schema.sql
FR-07 – FR-11 Editor and configuration	Cases TC-14 – TC-21; graph validation module
FR-12 – FR-15 Engine	Engine cases E-01 – E-28; functional cases TC-22 – TC-26
FR-16 – FR-18 AI	Cases TC-27 – TC-32; graph validation and model router
FR-19 – FR-21 Analytics	Engine cases E-01 – E-28; invariants I1 – I6; cases TC-33 – TC-35
FR-22 Monte Carlo	Cases TC-36 – TC-37; confidence-interval unit test
FR-23 – FR-24 Playback and templates	Cases TC-38 – TC-40
FR-25 – FR-26 Export and share	Cases TC-41 – TC-45; security-definer share functions
FR-27 Collaboration	Cases TC-46 – TC-47; channel authorisation policy
NFR-03 Correctness	All 28 analytical checks within tolerance; 22 of 22 invariants
NFR-05 Access control	TC-12, TC-44, TC-47

8. RESULTS AND DISCUSSION

This chapter presents what the system produces and what the results mean. It begins with the outputs a user sees, then analyses the numerical results obtained from the verification exercise, and closes with an evaluation of performance.

8.1 System Output

The delivered interface is documented below screen by screen, in the order in which a user encounters the results of a run. Each entry states what the screen presents; a placeholder marks where the corresponding screenshot from the running application is to be inserted.

On completion of a run the **Executive Summary** presents the headline indicators — arrivals, completions, throughput, the flagged bottleneck and the 0–100 system health score — together with a plain-English diagnosis and suggested actions, so a non-specialist can read the answer without interpreting a chart.

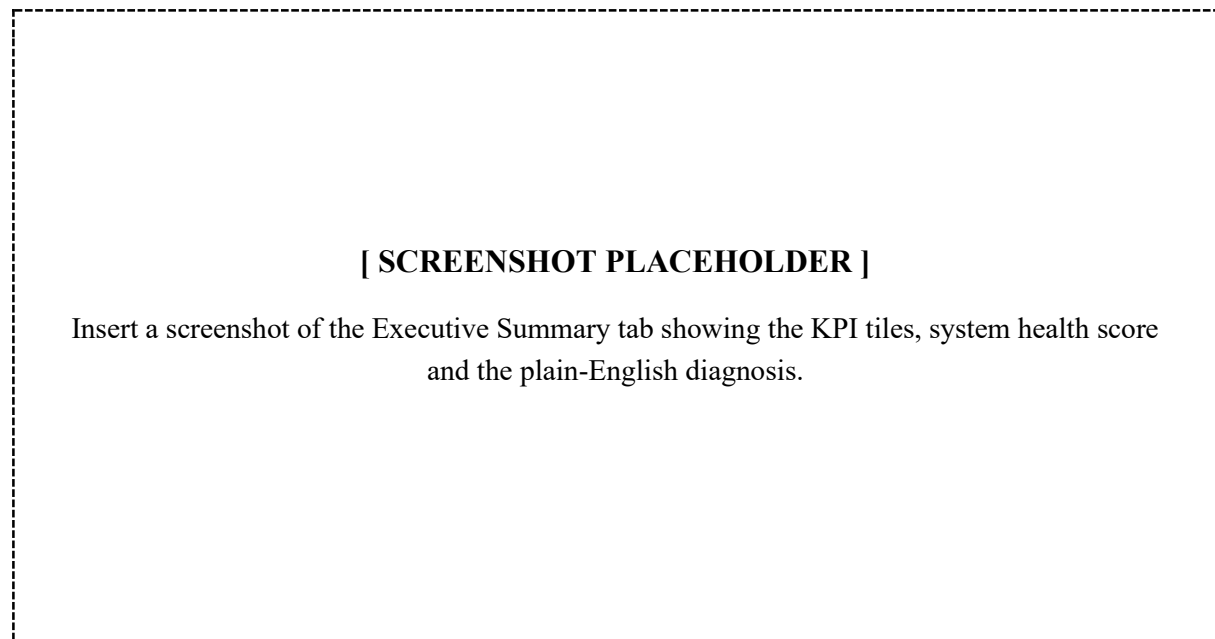


Figure 8.1: Executive Summary Tab of the Results Dashboard

The **Flow and Balance Check** applies Little’s Law and a conservation check to confirm that the run is internally consistent. It reports the discrepancy between L and λW and classifies the system as steady-state, transient or accumulating backlog.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Flow & Balance Check tab showing the Little's Law verification and stability verdict.

Figure 8.2: Flow and Balance Check (Little's Law Verification)

The **Traffic Heatmap** overlays the model with the load carried by each node and edge, so congested stations are visible at a glance.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Traffic Heatmap tab showing the model with congestion highlighted.

Figure 8.3: Traffic Heatmap

The **Line Buildup Timeline** plots queue length against simulated time, showing whether a queue is stable, cyclical or growing without bound.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Line Buildup Timeline tab showing queue length against simulated time.

Figure 8.4: Line Buildup Timeline

Station Counters list, for every node, the utilisation, average and percentile waiting times, throughput and queue depth.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Station Counters tab showing per-node utilisation, waiting time and throughput.

Figure 8.5: Station Counters

The **Person / Item Journey** tab follows a single entity through the model, listing each node it visited with the time waited and served, and is the tool for tracing an anomalous figure to the events that caused it.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Person / Item Journey tab tracing one entity through the model.

Figure 8.6: Person / Item Journey (Entity Tracer)

The **Cost and ROI** tab converts the run into money: resource cost over busy time, holding cost weighted by queue length and the waiting penalty. Where a change has been proposed, it reports whether the saving repays the cost.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Cost & ROI tab showing resource, holding and waiting cost.

Figure 8.7: Cost and Return-on-Investment Panel

The **Monte Carlo** panel runs 10, 20, 50 or 100 replications, reports progress as each completes and renders every metric as a mean with a shaded 95 per cent confidence band. The baseline-versus-scenario view places two configurations side by side.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Monte Carlo panel showing confidence-interval bands and the baseline-versus-scenario comparison.

Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands

The **Digital Twin** replays the run as moving entities on a WebGL canvas, with queue congestion highlighted and transport controls for play, pause, step, scrub and speed.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Digital Twin playback with entities moving and the transport controls visible.

Figure 8.9: Digital Twin Animated Playback

The **AI Assistant** accepts a plain-English description and returns a candidate graph for preview; the user must accept it before the project is altered. The same panel answers questions about simulation concepts and about the user's own model.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the AI Assistant panel with a plain-English prompt and the generated graph preview.

Figure 8.10: AI Assistant Generating a Model from a Description

The **AI Optimiser** reports which node is constraining the system and why, in terms of the computed statistics, and offers an Apply Fix action that performs the proposed change as an ordinary graph edit.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the AI Optimiser showing the diagnosis and the Apply Fix action.

Figure 8.11: AI System Optimiser Recommendation

The **Python Code Inspector** shows the SimPy script generated from the current graph, so any result can be reproduced independently; a single action exports it as a Jupyter notebook.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Python Code Inspector showing the generated SimPy script and the notebook export button.

Figure 8.12: Python Code Inspector

In a shared session each collaborator's **cursor** is drawn in their own colour with their name, and a presence row shows who is in the project.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of two browser windows in the same project showing live cursors and the presence avatars.

Figure 8.13: Real-Time Presence and Live Cursors

The **Export dialog** offers the graph as JSON, results as CSV, the script as a notebook and the full executive report as PDF, and manages the public share link with controls to create, copy and revoke it.

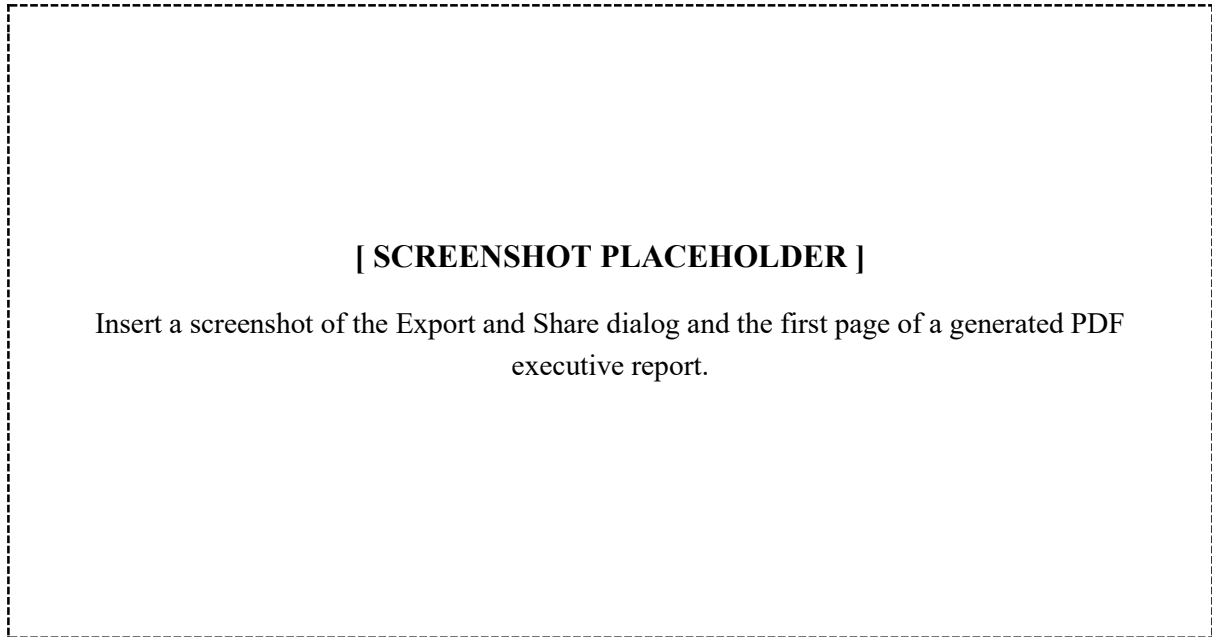


Figure 8.14: Export, Share and Executive PDF Report

A visitor opening a share link without an account sees the model read-only with every editing control absent. Where the link has been revoked, an explanatory notice replaces the project.

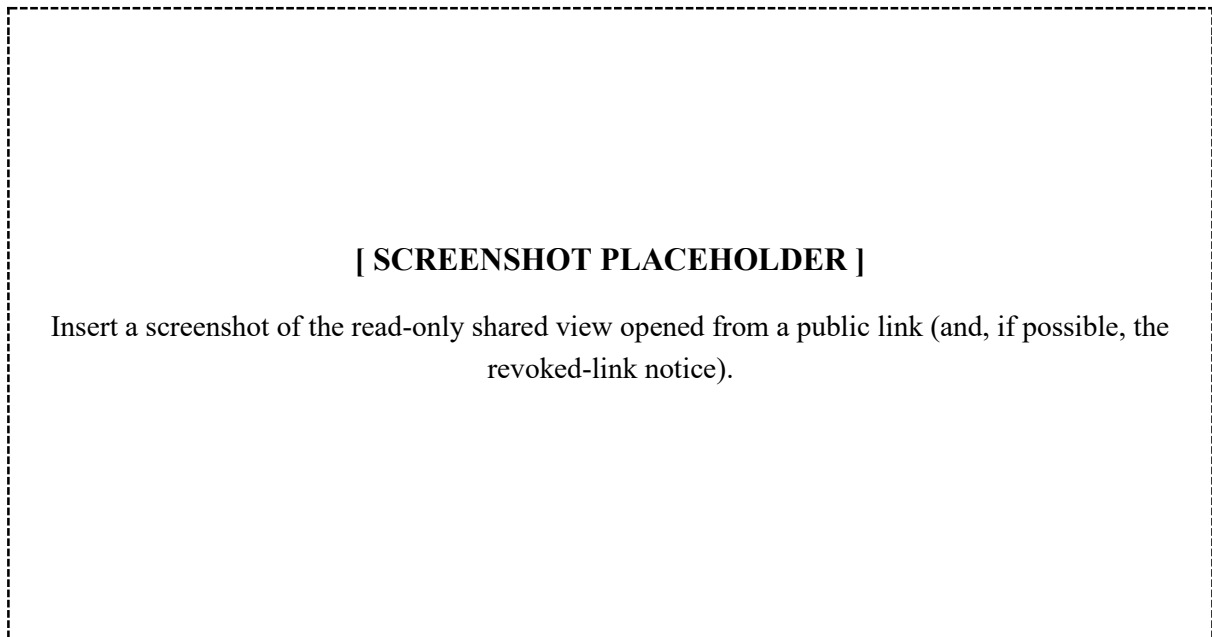


Figure 8.15: Read-Only Shared View

The **Settings** screen manages the display name and avatar, password change, the default simulation domain applied to new projects and account deletion, which requires confirmation and cascades through all owned data.

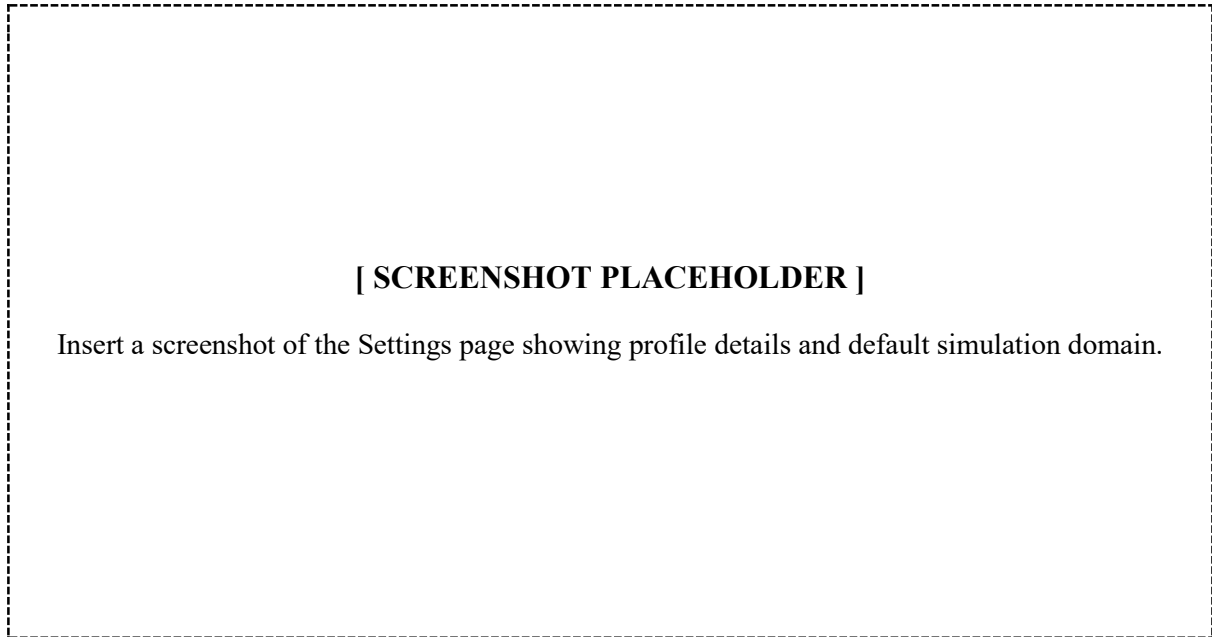


Figure 8.16: Profile and Settings

8.2 Result Analysis

8.2.1 Agreement with Queueing Theory

The central question for any simulator is whether it is right. The figure below compares the simulated result with the analytical prediction for the M/M/1 and M/M/3 reference queues. In every case the two bars are indistinguishable at the scale of the plot.

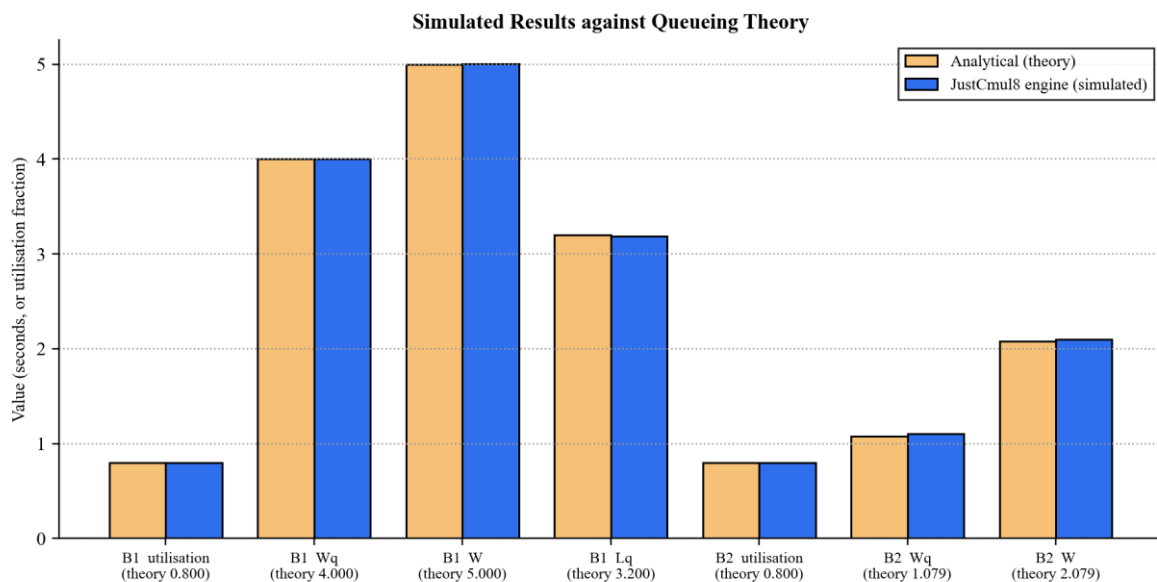


Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks)

The second figure presents the deviation of each of the fifteen principal checks as a percentage of the theoretical value, against the tolerance allowed. The largest deviation, +2.09 per cent for the M/M/3 queue wait, is roughly half of its tolerance; the M/M/1 deviations are below 0.6 per cent.

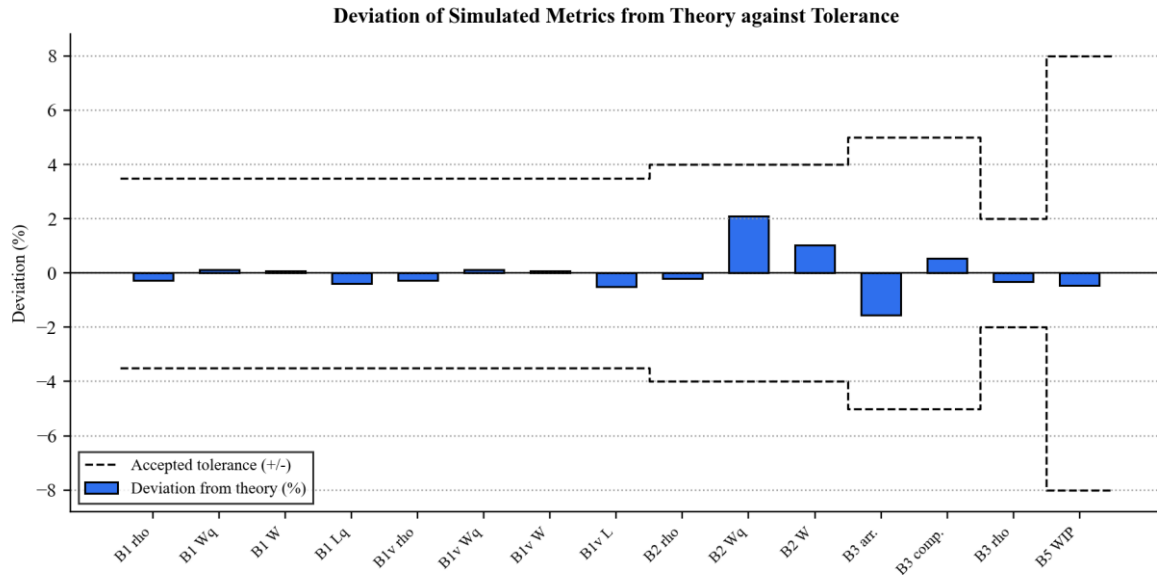


Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance

Table 8.1: Worked Example: M/M/1 Queue at $\rho = 0.8$ ($\lambda = 0.8$, $\mu = 1.0$, 36,000 s, 5 replications)

Metric	Formula	Theory	Simulated	95% CI ($\pm$)
Utilisation ρ	λ / μ	0.800	0.798	0.0056
Mean wait in queue W_q	$\rho / (\mu - \lambda)$	4.000 s	4.005 s	0.173
Mean time in system W	$1 / (\mu - \lambda)$	5.000 s	5.004 s	0.174
Mean queue length L_q	$\rho^2 / (1 - \rho)$	3.200	3.188	0.183

8.2.2 A Multi-Stage Model: Hospital Flow

Benchmark B6 shows the platform on a realistic scenario. Patients arrive at 1.2 per second, pass through a triage nurse, and a decision node sends 70 per cent to minor treatment and 30 per cent to major treatment; each station is stable in isolation with utilisation close to 0.7. The results with and without patient abandonment are summarised below.

Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds)

Measure	Abandonment on	Abandonment off
Patients arrived	8,683	8,541

Measure	Abandonment on	Abandonment off
Patients served	8,301	8,531
Patients who abandoned	374	0
Throughput clearance across seeds	0.956, 0.958, 0.954	0.999, 0.999, 1.000
Utilisation: triage / minor / major	0.705 / 0.690 / 0.697	0.708 / 0.682 / 0.726
Decision split to minor treatment	0.700	0.687
Little's Law discrepancy and verdict	3.2%, steady state	0.2%, steady state
System health score	91	93
Invariants satisfied	11 of 11	11 of 11

The result is exactly what a decision-maker needs to see. With abandonment on, 4.3 per cent of patients leave without being served, the system nevertheless remains stable, and the health score of 91 reflects a well-balanced but not flawless service. Without abandonment the service clears almost all of the offered load. The same model can be re-run under a proposed change — a third triage nurse, say — and compared through the Monte Carlo scenario view to see whether the improvement exceeds the noise.

8.2.3 The Platform Detects Failure as Well as Success

A useful analytics layer must recognise a system in trouble. In the saturated benchmark the arrival rate is twice the service rate: 1,181 entities arrive but only about 603 complete, utilisation is 0.997 and the work-in-progress grows to 578. The Little's Law verdict changes to *accumulating backlog*, and the health score falls to 45.8, well below the alert threshold of 70. The platform therefore distinguishes a healthy system from a failing one without the user needing to interpret raw numbers. A separate smoke test confirmed that a model using the extended node types — broadcaster, channel, store and container — executes without error and conserves flow (1,225 entities in and out at every stage).

8.3 Performance Evaluation

Performance was assessed on two levels. The first is the speed of the **engine**, measured by the verification harness, which executes the generated SimPy script in native CPython. The second is the responsiveness of the **application** in the browser, measured with the browser's developer tools.

8.3.1 Engine Throughput

Table 8.3: Engine Throughput Measured by the Verification Harness

Measurement	Workload	Observed
Analytical benchmark suite (B1–B5)	Five replications of each benchmark at a 36,000 simulated-second horizon; the M/M/1 benchmark generates $\approx 28,600$ entities per replication	159 s wall-clock for the whole suite
Single long run	28,714 entities through one resource	3.65 s wall-clock
Complex model	Hospital flow, 7,200 simulated seconds, 3 seeds, two configurations	Completed within the harness run
Clock accuracy	36,000 accumulated timeouts	Simulated clock exactly 36,000.0

A single run of nearly thirty thousand entities completes in under four seconds, and a working day for a typical service model in far less. Because discrete event simulation advances directly from event to event, run time depends on the number of events rather than on the simulated duration, and the heap-based event queue keeps the per-event cost logarithmic. The browser executes the same script through WebAssembly, which for CPU-bound Python is typically within a small multiple of native speed.

8.3.2 Application Performance in the Browser

The following measurements are taken on the target device with Chrome DevTools and Lighthouse and are recorded in the table below.

Table 8.4: Browser Performance Measurements (to be recorded)

Measurement	Target	Measured
Canvas interaction (drag / connect a node)	Immediate; no dropped frames	
First-run Pyodide download and start-up	Cached thereafter	
Run of a 10-node model, 1,000 simulated seconds	Interactive	
Run of a 100-node model, 1,000 simulated seconds	A few seconds	
Dashboard render after a run	Perceptibly immediate	
Peak browser memory for the 100-node run	Well below device limits	
Lighthouse performance score (landing page)	≥ 90	
Presence / cursor update between two sessions	Near real time	

8.3.3 Discussion

The measurements support the central claim of the project: that a genuine discrete event simulator can run acceptably inside a browser. Verification is the more important result. Agreement with queueing theory to within a few per cent, conservation of entities across a multi-stage model, and the correct classification of both healthy and overloaded systems establish that the platform is not merely fast enough but *correct*, which for a simulator is the property that matters. Three observations follow from the results and are carried into the roadmap of Chapter 10: memory rather than processing speed sets the ceiling on model size; Monte Carlo replications, being independent, are well suited to running in parallel; and the first-run download of the runtime is the first thing a new user experiences, which makes it the most valuable target for optimisation.

9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION

This chapter estimates the effort, schedule and cost that JustCmul8 would represent if it were developed commercially. Two independent models are applied — the Constructive Cost Model (COCOMO), which works from the size of the code, and Function Point Analysis (FPA), which works from the functionality delivered — and their outputs are compared with each other and with the effort actually invested. The chapter closes with a cost and benefit analysis of the platform for its users.

9.1 Effort Estimation

9.1.1 Basic COCOMO

The basic Constructive Cost Model estimates effort from size using coefficients determined by the project's development mode. JustCmul8 is classified as **organic**: a small team working in a well-understood application domain, with stable requirements and flexible constraints.

Table 9.1: COCOMO Coefficients by Development Mode

Mode	a	b	c	d	Applicable
Organic	2.4	1.05	2.5	0.38	Yes — small team, familiar domain, stable requirements
Semi-detached	3.0	1.12	2.5	0.35	No — assumes a medium team of mixed experience
Embedded	3.6	1.20	2.5	0.32	No — assumes tight hardware and regulatory constraints

With a measured size of 22.19 KSLOC (Section 9.2) the model gives:

$$\text{Effort } E = a \times (\text{KSLOC})^b = 2.4 \times (22.19)^{1.05} = 2.4 \times 25.93 = \mathbf{62.19 \text{ person-months}}$$

$$\text{Duration } D = c \times E^d = 2.5 \times (62.19)^{0.38} = 2.5 \times 4.80 = \mathbf{12.01 \text{ months}}$$

$$\text{Staffing } S = E / D = 62.19 / 12.01 = \mathbf{5.18 \text{ persons}}$$

$$\text{Productivity } P = \text{KSLOC} / E = 22.19 / 62.19 = \mathbf{0.357 \text{ KSLOC per person-month}}$$

Table 9.2: COCOMO Estimation Results

Parameter	Estimated value
Effort	62.19 person-months
Development duration	12.01 months

Parameter	Estimated value
Average staffing	5.18 persons
Productivity	0.357 KSLOC per person-month
Cost at ₹40,000 per person-month	₹24,87,600 ($\approx$ ₹24.88 lakh)

The monetary figure assumes a notional rate of forty thousand rupees per person-month, approximating an entry-level Indian software engineering salary including overhead. The rate is illustrative; the effort and duration are the substantive output of the model.

9.2 KSLOC Analysis

Size was measured from the delivered source tree, counting TypeScript, TSX and Python source files under the application's source directory and excluding dependencies, build output and generated artefacts.

Table 9.3: Source Lines of Code by Layer

Layer / directory	Contents	SLOC	Share
src/components	Workspace, landing, layout and shared interface components	11,544	52.0%
src/lib/simulation	Code generator, engine, analytics, Monte Carlo, export, registry	4,905	22.1%
src/app	Pages and API routes (auth, dashboard, workspace, share, AI, export)	3,848	17.3%
src/lib/ai	Model router, prompts, graph operations, flow analysis	1,654	7.5%
src/lib/realtime	Presence and broadcast hook	143	0.6%
src/lib/supabase, hooks	Database clients and helper hooks	59	0.3%
src/middleware.ts	Session refresh and route protection	39	0.2%
Total	91 source files (.ts, .tsx)	22,192	100%

Table 9.4: Measured Project Size

Measure	Value
Source files	91
Total lines of source code	22,192
Non-blank lines	20,286
Size in KSLOC	22.19

Measure	Value
Application pages	9 pages and 7 API routes (including the authentication callback)
Node types implemented	15
Simulation domains	6
Delivered modules	12
Database tables (application)	5, plus 2 access functions and 1 trigger

More than half of the code (52 per cent) is interface code, reflecting the richness of the workspace: a canvas editor, eight result tabs, a Monte Carlo panel, a WebGL digital twin and a landing page. The engine, analytics and export logic — the part that determines whether the numbers are right — occupies a further 22 per cent, and the AI layer 7.5 per cent. The database schema is small by design, because the graph is stored as a document and Supabase supplies authentication and authorisation.

9.3 Function Point Analysis

Function Point Analysis measures the functionality delivered rather than the code written, which makes it a useful cross-check on a size-based estimate. The IFPUG method counts five categories of component, weights each by complexity and adjusts the total by fourteen general system characteristics.

Table 9.5: Standard IFPUG Complexity Weights

Component type	Simple	Average	Complex
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

Table 9.6: Function Point Count by Component Type

Component type	Simple	Average	Complex	Count	Weighted
External Inputs — forms, edits and commands	6	8	4	18	74
External Outputs — dashboards, charts and exports	2	4	4	10	56

Component type	Simple	Average	Complex	Count	Weighted
External Inquiries — listings and retrievals	2	3	2	7	30
Internal Logical Files — application tables	3	0	2	5	51
External Interface Files — Auth, Gemini, Pyodide CDN	1	2	0	3	19
Unadjusted Function Points (UFP)				43	230

The eighteen external inputs comprise the authentication forms, project creation, renaming and deletion, graph edits, parameter configuration, template loading, run initiation, AI prompts, fix application, share-link creation and revocation, profile update and account deletion. The ten external outputs comprise the KPI dashboard, the bottleneck flag, the Monte Carlo interval charts, the digital-twin playback, the streamed log, the entity journey trace and the export formats. The five internal logical files are the database tables of Section 5.6, two of which (projects and runs) are classed as complex because they hold JSON documents.

Table 9.7: General System Characteristics and Degrees of Influence

No.	General system characteristic	Rating	Justification
1	Data communications	4	Client, server, database and AI service all communicate
2	Distributed data processing	5	Simulation executes on the client, not the server
3	Performance	4	Responsive canvas and immediate dashboard expected
4	Heavily used configuration	2	No constrained deployment hardware
5	Transaction rate	3	Moderate, rising during collaborative editing
6	Online data entry	5	All modelling is interactive and immediate
7	End-user efficiency	5	No-code operation is a primary objective
8	Online update	5	Automatic persistence on every change
9	Complex processing	5	Discrete event simulation, Monte Carlo statistics, AI
10	Reusability	3	Service modules reusable; interface is application-specific
11	Installation ease	5	Zero installation by design
12	Operational ease	4	Managed hosting with automatic deployment
13	Multiple sites	3	Browser-portable across platforms
14	Facilitate change	4	Registry-driven extension of domains and node types
	Total Degree of Influence (TDI)	57	

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI}) = 0.65 + 0.57 = \mathbf{1.22}$$

$$\text{Function Points } \text{FP} = \text{UFP} \times \text{VAF} = 230 \times 1.22 = \mathbf{281 \text{ function points}}$$

$$\text{Effort } E = \text{FP} / \text{productivity} = 281 / 13 = \mathbf{21.6 \text{ person-months}}$$

$$\text{Cost at ₹40,000 per person-month} = 21.6 \times 40,000 = \mathbf{₹8,64,000 (\approx ₹8.64 \text{ lakh})}$$

$$\text{Implied size } 22,192 \text{ SLOC} / 281 \text{ FP} = \mathbf{79 \text{ SLOC per function point}}$$

Industry productivity for a modern web application of this kind is in the region of thirteen function points per person-month. The implied 79 lines per function point is higher than the typical 50 to 60 for TypeScript, and the reason is visible in Table Table 9.3: a very large share of the code is presentation — the landing page alone runs to about 1,900 lines — which delivers polish and usability but adds few function points.

9.4 Comparison of Estimates and Cost–Benefit Analysis

Table 9.8: Comparison of Estimates against Planned Effort

Basis	Effort	Duration	Team	Indicative cost
COCOMO (basic, organic)	62.19 PM	12.01 months	5.18 persons	₹24.88 lakh
Function Point Analysis	21.6 PM	—	—	₹8.64 lakh
Actual project (planned)	12 PM	6 months	2 students	Nil cash cost — academic, open-source stack

The three rows differ substantially, and the gap is more informative than any single figure. Both models were calibrated against projects developed largely from first principles, and neither accounts for the leverage a modern framework stack provides. The clearest example is the simulation engine: writing a discrete event scheduler with resource contention, preemption, container levels and typed stores would plausibly account for a third of the COCOMO estimate on its own, yet all of it was obtained from SimPy, and the team’s effort went into the code generator that targets it. The same argument applies to React Flow, which supplied the canvas, and to Supabase, which supplied authentication, access control and the realtime transport. The twenty-two thousand lines written sit on top of a very large volume of code that was not. Function Point Analysis lands closer to the actual figure than COCOMO, consistent with its measuring delivered functionality rather than code volume.

The estimates therefore establish an upper bound on what the platform would have cost to build without such foundations and, in doing so, quantify the value of the open-source ecosystem on

which it rests. A commercially developed equivalent would plausibly fall between the two estimates.

9.4.1 Costs

Table 9.9: Development and Operating Costs

Item	Nature	Cost
Development effort	Two students, six months	Academic; no cash cost
Software licences	Entire stack is open source or free-tier	Nil
Hosting (front end)	Vercel free tier	Nil at academic scale
Database, authentication, realtime	Supabase free tier	Nil at academic scale; paid tiers scale with data volume
AI inference	Gemini API, metered per explicit request	Small and controllable
Simulation compute	Runs on the user's own device	Nil — does not grow with usage
Maintenance	Dependency upgrades, registry additions	Student / community effort

9.4.2 Benefits

Table 9.10: Benefits of the Platform

Benefit	Type	Beneficiary
Removes per-seat licence fees that commercial tools charge	Tangible, recurring	Institutions, small businesses
Removes installation and IT-support effort on every machine	Tangible	Laboratories, IT departments
No server compute cost that scales with the number of simulations	Tangible	The platform operator
Decisions tested on a computer instead of on the real system, avoiding costly mistakes	Tangible, decision-dependent	Small-business owners
Time to first result measured in minutes rather than days of training	Tangible (time)	Students, lecturers
Wider access to a decision-support technique	Intangible	Society, education
A transparent, reproducible engine that supports teaching of SimPy and queueing theory	Intangible	Educators, researchers

The cost–benefit balance is unusually favourable because the principal cost — computation — is transferred to hardware the user already owns, while the principal benefit — access to

trustworthy simulation — is delivered at no licence cost. Even a single commercial seat priced in the thousands of dollars per year would exceed the entire annual running cost of the platform at academic scale, and every additional user adds benefit without adding cost.

10. CONCLUSION AND FUTURE SCOPE

10.1 Achievements of the Project

JustCmul8 set out to remove three barriers that keep Discrete Event Simulation out of the hands of most of the people who could use it: the cost of commercial licences, the programming skill demanded by open-source alternatives and the installation that desktop distribution requires. The achievements of the project can be summarised as follows.

- **A complete no-code simulation platform** delivered as twelve integrated modules — from account creation and canvas modelling to analysis, sharing and collaboration — running entirely in a web browser.
- **A real engine, not an imitation.** Every model is compiled into a genuine Python SimPy program and executed by CPython compiled to WebAssembly in a Web Worker, so results come from the same library researchers use directly.
- **Verified correctness.** All 28 analytical checks against M/M/1, M/M/c and saturated-queue theory passed within tolerance, with every deviation below 2.1 per cent, and all 22 conservation and flow-balance invariants held on a multi-stage hospital-flow model.
- **Breadth of modelling.** Fifteen node types across six application domains, including preemptive resources, typed stores, containers, channels, broadcasters, event synchronisers and interrupters.
- **Decision support, not just numbers.** Bottleneck flagging, a Little’s Law consistency check, a composite health score, cost and return-on-investment analysis, and Monte Carlo confidence intervals with baseline-versus-scenario comparison.
- **AI-assisted workflow.** Plain-English model generation and one-click, cost-aware fixes, served through a health-aware router that fails over between models automatically.
- **Understandable output.** An eight-tab results dashboard, a WebGL digital-twin animation, an entity tracer and a traffic heatmap turn statistics into pictures.
- **Transparency and reproducibility.** A live Python code inspector and one-click Jupyter export make any result independently verifiable.
- **Security by design.** Row-Level Security on every table, opaque revocable share links served through narrow database functions, secrets kept on the server and cascading deletion of user data.

- **Sustainable economics.** Client-side computation and an open-source stack keep the running cost essentially independent of usage.

Academically, the project exercised material from across the programme rather than merely referencing it. Priority queues and graph traversal from data structures appear in the event scheduler and the validation pass; normalisation, referential integrity and access control from database management appear in the schema and its policies; the software development life cycle appears in the scheduling decisions of Chapter 3, including the decision to front-load the highest-risk component; and probability and statistics appear in the distribution samplers, the confidence intervals and the verification against queueing theory.

10.2 Conclusion

The decision that most shaped the result was to refuse to approximate the engine. It would have been considerably easier to write a rough event loop in JavaScript, and the platform would have looked much the same. Instead the graph is compiled into a genuine SimPy script, and the verification of Chapter 7 is what makes this more than an assertion: agreement with theory to within a few per cent shows that the generated script expresses the intended model and that the statistics are gathered correctly.

In practical terms the platform is usable today. A student can open a template and see queueing behaviour within a minute; a lecturer can demonstrate congestion live and circulate a share link afterwards; a small-business owner can describe a shop in plain English and read off a confidence interval before committing money. That these things require no licence, no installation and no code is the outcome the project intended. JustCmul8 demonstrates that professional-grade simulation can be made as accessible as a web page, and that accessibility need not be bought at the price of accuracy.

10.3 Future Scope

The architecture was designed for extension: a single registry defines each domain, the engine is isolated behind a worker boundary, and authorisation lives in the database. The roadmap below builds on those decisions rather than replacing them, and addresses each limitation recorded in Section 1.4.

Near-term enhancements

- **Parallel Monte Carlo** — distributing replications across several Web Workers to reduce a hundred-replication analysis by close to the core count of the machine, since the replications are independent.
- **Reproducible runs** — user-selectable random seeds, so any run can be repeated exactly and shared with its seed.
- **Pause and resume** — checkpointing so that a long run can be suspended and continued.
- **Streaming result aggregation** — discarding per-entity detail once aggregated, and retaining full traces only when playback is requested, to raise the ceiling on model size.
- **Role-based collaboration** — invitations by email with owner, editor and viewer permissions enforced by Row-Level Security, and version history with rollback.
- **Convergent conflict resolution** — replacing last-write semantics with a merge based on operational transformation or CRDTs, so that two users can edit the same node without either losing work.
- **Input-validation hardening** — surfacing every boundary condition, such as a zero arrival rate, as a friendly message before a run.
- **Progressive runtime loading** — fetching a reduced Pyodide payload to shorten the first-run start-up.

Medium-term extensions

- **Custom node types through a plugin interface**, executed safely inside the existing sandbox, so users can model domains not anticipated at design time.
- **Additional simulation paradigms** — an agent-based engine and a genuine continuous solver alongside SimPy, selected per project.
- **Search-based optimisation** — sweeping a parameter space to find the configuration that minimises cost subject to a service-level constraint, turning advice into optimisation.
- **Offline and classroom mode** — a progressive web application with local persistence for laboratories without reliable internet.
- **Native mobile companion** for viewing dashboards and playback.
- **A community template marketplace** where users publish and rate models for their domains.

Longer-term directions

- **Optional cloud execution** for models that exceed browser memory, retaining client-side execution as the default.
- **Parallel and distributed simulation** — partitioning very large models across workers or machines with synchronised clocks.
- **Calibration from real data** — fitting arrival and service distributions automatically from historical or sensor data, replacing estimated parameters with measured ones.
- **Live digital-twin integration** — connecting a running model to IoT feeds so that the simulation tracks the physical system it represents.
- **Institutional features** — single sign-on, organisation management and classroom cohorts for department-scale adoption.

None of these depends on revisiting the decisions taken in this project. The layered architecture, the worker boundary, the registry-driven definition of domains and node types and the enforcement of authorisation at the database were each chosen to make extension possible, and every item above builds on them.

REFERENCES AND BIBLIOGRAPHY

References are listed in IEEE citation style: textbooks and articles establishing the theoretical basis first, followed by standards and the technical documentation of the software on which the platform is built.

- [1] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [2] J. Banks, J. S. Carson, B. L. Nelson and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2010.
- [3] B. P. Zeigler, H. Praehofer and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. San Diego, CA, USA: Academic Press, 2000.
- [4] S. Robinson, *Simulation: The Practice of Model Development and Use*, 2nd ed. Basingstoke, U.K.: Palgrave Macmillan, 2014.
- [5] D. Gross, J. F. Shortle, J. M. Thompson and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [6] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.
- [7] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [9] G. E. P. Box and M. E. Muller, “A note on the generation of random normal deviates,” *The Annals of Mathematical Statistics*, vol. 29, no. 2, pp. 610–611, Jun. 1958.
- [10] A. B. Owen, *Monte Carlo Theory, Methods and Examples*. Stanford, CA, USA: Stanford University, 2013.
- [11] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.
- [12] A. J. Albrecht and J. E. Gaffney, “Software function, source lines of code, and development effort prediction: A software science validation,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 6, pp. 639–648, Nov. 1983.
- [13] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

- [14] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2015.
- [15] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.
- [16] G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [17] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [18] Team SimPy, “SimPy: Discrete event simulation for Python — documentation.” [Online]. Available: <https://simpy.readthedocs.io/> [Accessed: Sep. 2026].
- [19] Pyodide Development Team, “Pyodide: Python with the scientific stack, compiled to WebAssembly.” [Online]. Available: <https://pyodide.org/> [Accessed: Sep. 2026].
- [20] WebAssembly Community Group, “WebAssembly Core Specification,” W3C Recommendation, 2019. [Online]. Available: <https://www.w3.org/TR/wasm-core-1/> [Accessed: Sep. 2026].
- [21] Vercel Inc., “Next.js Documentation — App Router.” [Online]. Available: <https://nextjs.org/docs> [Accessed: Sep. 2026].
- [22] Meta Open Source, “React Documentation.” [Online]. Available: <https://react.dev/> [Accessed: Sep. 2026].
- [23] xyflow, “React Flow Documentation.” [Online]. Available: <https://reactflow.dev/> [Accessed: Sep. 2026].
- [24] Supabase Inc., “Supabase Documentation — Database, Auth, Realtime and Row Level Security.” [Online]. Available: <https://supabase.com/docs> [Accessed: Sep. 2026].
- [25] PostgreSQL Global Development Group, “PostgreSQL Documentation — Row Security Policies.” [Online]. Available: <https://www.postgresql.org/docs/> [Accessed: Sep. 2026].
- [26] Google, “Gemini API Documentation.” [Online]. Available: <https://ai.google.dev/docs> [Accessed: Sep. 2026].
- [27] Microsoft Corporation, “TypeScript Handbook.” [Online]. Available: <https://www.typescriptlang.org/docs/> [Accessed: Sep. 2026].
- [28] Mozilla Developer Network, “Using Web Workers.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API [Accessed: Sep. 2026].

- [29] International Function Point Users Group, Function Point Counting Practices Manual, Release 4.3.1. Princeton Junction, NJ, USA: IFPUG, 2010.